



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Diplomová práce

Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)

Bc. Milan Janoch



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Diplomová práce

Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)

Bc. Milan Janoch

Vedoucí práce

Ing. Martin Dostal, Ph.D.

© Bc. Milan Janoch, 2026.

Všechna práva vyhrazena. Žádná část tohoto dokumentu nesmí být reprodukována ani rozšiřována jakoukoli formou, elektronicky či mechanicky, fotokopírováním, nahráváním nebo jiným způsobem, nebo uložena v systému pro ukládání a vyhledávání informací bez písemného souhlasu držitelů autorských práv.

Citace v seznamu literatury:

JANOCH, Bc. Milan. *Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)*. Plzeň, 2026. Diplomová práce. Západočeská univerzita v Plzni, Fakulta aplikovaných věd, Katedra informatiky a výpočetní techniky. Vedoucí práce Ing. Martin Dostal, Ph.D.

ZÁPADOČESKÁ UNIVERZITA V PLZNI

Fakulta aplikovaných věd
Akademický rok: 2025/2026

ZADÁNÍ DIPLOMOVÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení: **Bc. Milan JANOCH**
Osobní číslo: **A24N0104P**
Studijní program: **N0613A140040 Softwarové a informační systémy**
Téma práce: **Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)**
Zadávající katedra: **Katedra informatiky a výpočetní techniky**

Zásady pro vypracování

1. Prostudujte současné možnosti generování kódu pomocí velkých jazykových modelů (LLM), se zvláštním zaměřením na oblast automatizovaného testování softwaru.
2. Analyzujte strukturu uživatelských příběhů a specifikací běžně používaných v nástrojích pro řízení vývoje softwaru a identifikujte klíčové prvky využitelné pro generování testovacích scénářů.
3. Navrhněte architekturu systému pro automatické generování end-to-end (E2E) testů z uživatelských příběhů s využitím LLM.
4. Implementujte funkční prototyp navrženého řešení využívající vybraný LLM a vhodný framework pro E2E testování.
5. Otestujte funkčnost systému na vzorových scénářích a vyhodnoťte kvalitu generovaných testů.
6. Kriticky zhodnoťte přínosy a limity řešení a navrhněte možnosti dalšího rozšíření.

Rozsah diplomové práce: **doporuč. 50 s. původního textu**
Rozsah grafických prací: **dle potřeby**
Forma zpracování diplomové práce: **tištěná/elektronická**

Seznam doporučené literatury:
dodá vedoucí diplomové práce

Vedoucí diplomové práce: **Ing. Martin Dostal, Ph.D.**
Katedra informatiky a výpočetní techniky
Konzultant diplomové práce: **Ing. Tomáš Rojík**
Emplifi Czech Republic, a.s.
Datum zadání diplomové práce: **10. září 2025**
Termín odevzdání diplomové práce: **14. května 2026**

L.S.

doc. Ing. Miloš Železný, Ph.D.
děkan

doc. Ing. Přemysl Brada, MSc., Ph.D.
vedoucí katedry

V Plzni dne 30. září 2025

Prohlášení

Prohlašuji, že jsem tuto diplomovou práci vypracoval samostatně a výhradně s použitím citovaných pramenů, literatury a dalších odborných zdrojů. Tato práce nebyla využita k získání jiného nebo stejného akademického titulu.

Dále prohlašuji, že během této práce jsem použil Gemini 3.1 Pro ke kontrole gramatiky a ke stylistické úpravě textu z důvodu zlepšení čitelnosti a srozumitelnosti textu. Po použití tohoto nástroje jsem pečlivě zkontroloval a upravil obsah podle potřeby a přebírám plnou odpovědnost za výsledný obsah práce.

Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona v platném znění, zejména skutečnost, že Západočeská univerzita v Plzni má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona.

V Plzni dne 24. dubna 2026

.....
Bc. Milan Janoch

V textu jsou použity názvy produktů, technologií, služeb, aplikací, společností apod., které mohou být ochrannými známkami nebo registrovanými ochrannými známkami příslušných vlastníků.

Abstrakt

Tato diplomová práce se zabývá využitím velkých jazykových modelů (LLM) pro automatizované end-to-end testování softwaru. Cílem práce je návrh a implementace systému, který na základě uživatelských příběhů generuje robustní automatizované testovací scénáře. Navržené řešení využívá architekturu založenou na autonomním AI agentovi s přístupem k nástrojům pro inspekci prohlížeče v reálném čase. Funkčnost systému byla evaluována pomocí devíti jazykových modelů (z rodin GPT, Claude a Gemini). Samotná evaluace zahrnovala jak generování testů pro plně funkční aplikaci, tak křížovou verifikaci a detekci záměrně zanesených chyb. Výsledky ukazují, že pokročilé modely s podporou reasoningu dokáží efektivně identifikovat defekty a generovat kód s vyšší sémantickou hloubkou než běžné nahrávací nástroje využívané v testování, přičemž dosahují časové úspory ve srovnání s manuálním psáním testů.

Abstract

This thesis explores the use of large language models (LLMs) for automated end-to-end software testing. The goal of this thesis is to design and implement a system that generates robust automated test scenarios based on user stories. The proposed solution utilizes an architecture based on an autonomous AI agent with access to real-time browser inspection tools. The system's functionality was evaluated using nine language models (from the GPT, Claude, and Gemini families). The evaluation itself included both test generation for a fully functional application and cross-verification and detection of intentionally introduced errors. The results show that advanced models with reasoning capabilities can effectively identify defects and generate code with greater semantic depth than conventional recording tools used in testing, while achieving time savings compared to manual test writing.

Klíčová slova

E2E testování • LLM • AI agent • user stories • automatizace testů

Poděkování

Rád bych poděkoval Ing. Martinu Dostalovi, Ph.D. za jeho neustálou podporu, věcné připomínky a cenné rady při tvorbě této diplomové práce. Děkuji také Ing. Tomášovi Rojíkovi za jeho pravidelné odborné konzultace, praktické rady, postřehy a poskytnutí přístupu k LLM modelům.

Obsah

1	Úvod	4
2	Teoretická část	5
2.1	Agilní vývoj a specifikace požadavků	5
2.1.1	Uživatelské příběhy	7
2.1.2	Akceptační kritéria	8
2.2	Automatizované testování softwaru	9
2.2.1	E2E testování	11
2.2.2	Přehled frameworků pro webovou automatizaci	12
2.2.3	Výzvy automatizace v agilním prostředí	13
2.3	Velké jazykové modely ve vývoji softwaru	14
2.3.1	Parametry a technické aspekty LLM	15
2.3.2	Srovnání LLM modelů a pricing	17
2.3.3	Limity generování	18
2.4	Agentní systémy a autonomní testování	19
2.4.1	Definice a charakteristika autonomních agentů	19
2.4.2	Architektura ReAct: Spojení uvažování a jednání	20
2.4.3	Model Context Protocol (MCP)	21
2.5	Shrnutí	23
3	Analýza a návrh systému	24
3.1	Analýza struktury a sémantiky vstupních artefaktů	24
3.2	Specifikace požadavků na systém	27
3.2.1	Vymezení rozsahu generovaných testů	27
3.2.2	Funkční požadavky (FR) na systém	27
3.2.3	Nefunkční požadavky (NFR) na systém	28
3.3	Koncepční architektura systému	29
3.3.1	Doménový model aplikace	29
3.3.2	Distribuce a perzistence dat	30
3.3.3	Architektura systémových komponent	32

3.3.4	Architektura agenta a exekuční smyčka	34
3.4	Návrh uživatelského rozhraní	36
3.5	Shrnutí	38
4	Použité technologie	39
4.1	TypeScript	39
4.2	React	39
4.3	NestJS	40
4.4	Vercel AI SDK	41
4.5	Playwright a Model Context Protocol (MCP)	42
4.6	Shrnutí	42
5	Implementace	43
5.1	Organizace projektu	43
5.2	Implementace backendové části	44
5.2.1	Doménové moduly	44
5.2.2	Implementace doménového modelu a sdílených DTO	46
5.2.3	Perzistence a konzistence dat	47
5.3	Implementace komunikace s LLM	49
5.3.1	Abstrakce poskytovatelů a Vercel AI SDK	49
5.3.2	Správa dostupných modelů	52
5.4	Implementace autonomního agenta	54
5.4.1	Integrace nástrojů přes Model Context Protocol	54
5.4.2	Řídící smyčka agenta a ochranné mechanismy	55
5.4.3	Návrh systémového promptu a řízení chování (Prompt Engineering)	57
5.4.4	Validační pipeline a samoopravná smyčka	59
5.5	Implementace klientské části	61
5.5.1	Architektura UI a správa stavu	61
5.5.2	Real-time streamování a vizualizace procesu	62
5.5.3	Prezentace vygenerovaného kódu a exekuce testů	64
5.6	Shrnutí	64
6	Testování a evaluace	65
6.1	Metodika evaluace LLM agenta a testovací prostředí	65
6.1.1	Architektura testované aplikace	66
6.1.2	Připravené testovací scénáře	67
6.2	Výsledky evaluace velkých jazykových modelů	68
6.2.1	Metodika exekuce a sběru dat	68
6.2.2	Analýza generování pozitivních scénářů	70

6.2.3	Křížová verifikace a detekce defektů	72
6.2.4	Ekonomické zhodnocení	75
6.2.5	Porovnání s ručně psanými testy	77
6.3	Testování softwarové implementace nástroje	79
6.4	Uživatelské testování a zpětná vazba	80
6.5	Shrnutí	81
7	Možná vylepšení	83
7.1	Správa testů a jejich úpravy	83
7.2	Automatizovaná izolace prostředí	83
7.3	Optimalizace uživatelské přívětivosti a výkonu	84
7.4	Integrace do firemních ekosystémů	84
7.5	Bezpečnostní mechanismy a rozšiřitelnost nástrojů	85
7.6	Shrnutí	85
8	Závěr	86
	Přehled zkratk	88
A	Specifikace promptů pro AI agenta	89
A.1	Systémový prompt	89
A.2	Prompt pro regeneraci testů	92
B	Ukázky klientského prostředí	94
C	Uživatelské příběhy použité k evaluaci	100
D	Uživatelské testování	105
E	Uživatelská dokumentace	112
F	Programátorská dokumentace	114
G	Obsah přílohy	116
	Bibliografie	117
	Seznam obrázků	122
	Seznam tabulek	123
	Seznam výpisů	124

V dnešním světě, kdy se software i díky nástupu umělé inteligence vyvíjí rychleji než kdy dříve, je zajištění kvality a spolehlivosti aplikací klíčovým faktorem pro úspěch. Testování hraje zásadní roli při odhalování chyb, zajištění důvěry uživatelů a zvyšování kvality softwaru. S nástupem velkých jazykových modelů (LLM) se objevují nové příležitosti pro automatizaci generování testů [1], což může výrazně zefektivnit proces testování a snížit náklady spojené s manuálním vytvářením testovacích scénářů. Hranice možností se stále posouvají nejen co se týče výkonností jednotlivých modelů (např. známých GPT, Claude) a společností přicházejících na trh, ale i co se týče přístupů k využití těchto modelů pro generování testů. I přes tento inovační potenciál je ale třeba myslet na rizika a výzvy spojené s rozšířeným využíváním LLM - a to nejen z pohledu softwarového (např. riziko halucinací, nedostatečná kvalita generovaných testů), ale i na cenovou a etickou stránku věci (např. náklady na využívání LLM, otázka ochrany dat a soukromí).

Cílem této diplomové práce je prozkoumat možnosti generování kódu pomocí velkých jazykových modelů v rámci automatizovaného testování a implementovat funkční prototyp, který má potenciál výrazně urychlit a zefektivnit proces testování softwaru pomocí end-to-end testů. Aby aplikace byla prakticky použitelná, bude zaměřená na generování testů na základě uživatelských příběhů, jež jsou běžně využívány v agilním vývoji softwaru [2] a poskytují přehledný a srozumitelný popis požadavků z pohledu uživatele. To umožní rozšířit možnosti psaní spustitelných testů i do oddělení jako je QA (Quality Assurance), které nemusí mít hluboké technické znalosti, ale jsou klíčové pro zajištění kvality softwaru. Důraz bude kladen na kvalitu vygenerovaných testů, jejich schopnost odhalovat chyby a přínos pro zefektivnění procesu testování.

Teoretická část

2

V této kapitole bude představen teoretický rámec nezbytný pro pochopení problematiky autonomního generování testů. Z důvodu potřeby transformace obchodních požadavků do funkčního kódu bude nejprve popsáno, jakým způsobem probíhá specifikace v agilním prostředí pomocí uživatelských příběhů a akceptačních kritérií. Následně budou rozebrány moderní přístupy k testování softwaru, existující frameworky pro webovou automatizaci a výzvy spojené s testováním v agilním prostředí.

Důležitou částí kapitoly je rozbor velkých jazykových modelů (LLM), které v současnosti slouží jako nástroj pro interpretaci požadavků a generování kódu. Budou vysvětleny jejich základní principy, technické parametry a limity spojené s psaním testovacích skriptů. V závěru kapitoly pak budou představeny autonomní agentní systémy, které propojují svět testování a umělé inteligence skrze pokročilé architektury a komunikační protokoly. Tento ucelený přehled metodik a technologií poslouží jako základ pro následný praktický návrh vlastního řešení.

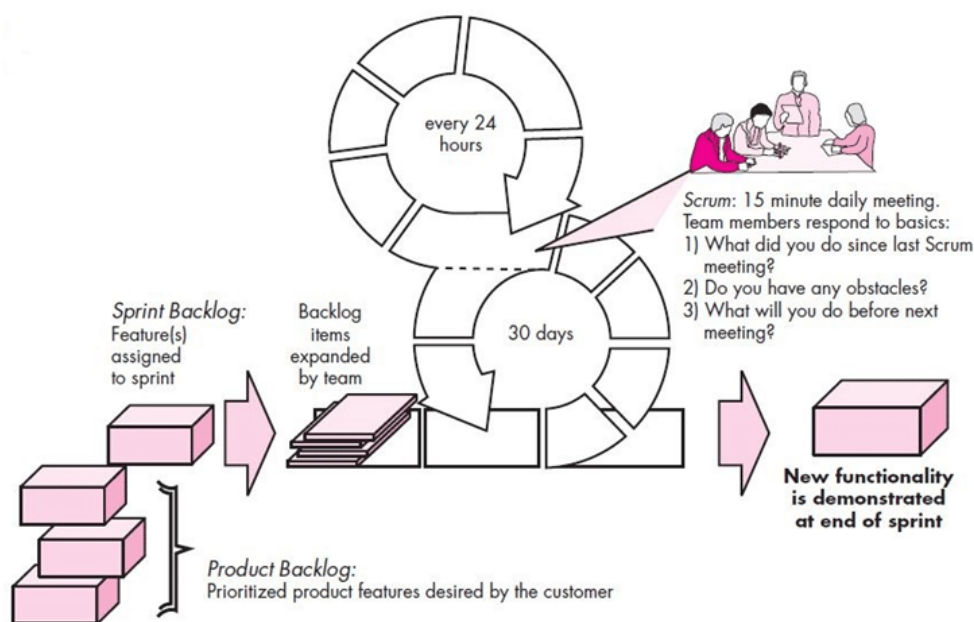
2.1 Agilní vývoj a specifikace požadavků

IT je jedním z nejrychleji se rozvíjejících odvětví, a to nejen z pohledu technologií, ale i přístupu k vývoji softwaru. Jak [3] uvádí, tento aspekt vedl k vývoji metodik (např. agilní metodiky či vodopádový model), které navrhují řízení v závislosti na vlastnostech produktu, organizačních strukturách či dynamickém prostředí. Tradiční konzervativnější vodopádový model (populární např. ve stavebnictví) se zaměřuje na důkladné plánování [4] a znalost požadavků před zahájením vývoje [5]. V rychle se měnícím prostředí je ale tento přístup často neefektivní, protože požadavky ze strany uživatelů se mohou rychle měnit a není možné je znát všechny předem. Tento problém řeší agilní metodiky, které kladou důraz na flexibilitu - díky tomu, že fungují na tzv. iterativním principu (software se vyvíjí v krátkých cyklech často přezdívaných jako sprinty), umožňují průběžné přizpůsobování se měnícím se požadavkům a rychlejší dodávání funkčních verzí softwaru [5].

Nejznámějším a současně nejrozšířenějším přístupem k agilnímu vývoji je Scrum. Scrum tým se zpravidla skládá z následujících rolí [4, 6, 7, 8]:

- **Vývojáři (Developers)** - jsou zodpovědní za vývoj softwaru a implementaci požadovaných funkcionalit (features).
- **Scrum Master** - zajišťuje dodržování Scrum procesů, podporuje tým při práci a pomáhá odstraňovat překážky, které by mohly bránit efektivnímu vývoji.
- **Product Owner** - je zodpovědný za správu produktového backlogu, komunikaci s vývojovým týmem a zainteresovanými stranami (stakeholdery) a za zajištění toho, aby vývoj směřoval k naplnění potřeb zákazníka.

Hlavní náplní práce Product Ownera je správa backlogu a příprava uživatelských příběhů (často známý pod rozšířenějším anglickým pojmem *user stories*), které specifikují požadované funkcionality z pohledu uživatele [8].



Obrázek 2.1: Vizualizace Scrum procesu a jeho klíčových fází [9].

Vzájemná interakce těchto rolí a posloupnost jednotlivých aktivit je znázorněna na obrázku 2.1. Celý proces probíhá v iterativních cyklech (tzv. sprintech), které zpravidla trvají 2-4 týdny (na zmiňovaném schématu je pro ilustraci uvedena historicky tradiční délka 30 dnů). Během nich tým postupuje od výběru prioritních požadavků zákazníka z produktového backlogu (*Product Backlog*), přes vytvoření

menšího backlogu pro daný sprint (*Sprint Backlog*), až po samotnou implementaci, která je prokládána každodenními krátkými (15-minutovými) schůzkami. Klíčovým výstupem každého sprintu je funkční přírůstek softwaru a zpětná vazba od zákazníka. A právě uživatelské příběhy tvoří základní pilíř pro specifikování těchto přírůstků.

2.1.1 Uživatelské příběhy

User story je nejpoužívanějším artefaktem v agilním vývoji [10] - jedná se o krátký, semi-strukturovaný popis požadavků napsán v přirozeném jazyce. Na ukázce 2.1 je uvedena standardní šablona, podle níž se uživatelské příběhy píší [10, 11, 12].

As a~<type of user / persona>
I~want <to perform some action / goal>
So that <some value / benefit is achieved>

Zdrojový kód 2.1: Standardní šablona uživatelského příběhu.

Každá user story by měla být napsána z pohledu konkrétního uživatele (*persona*), s jasně stanoveným cílem (*goal*) a dosaženou hodnotou (*value*). Hodnota je dosažena pouze tehdy, pokud byl plánovaný cíl úspěšně splněn [12].

Kvalita napsaných user stories je klíčová nejen pro porozumění vývojářů při implementaci, ale také pro následné testování a ověření, že plánovaný cíl byl skutečně dosažen. Jednou z metod, která pomáhá jasně definovat kritéria pro úspěšné splnění user story, je INVEST [13]. V tabulce 2.1 jsou uvedeny jednotlivé principy INVEST, které by měly být dodržovány pro psaní kvalitních uživatelských příběhů.

Princip	Popis
Independent	User story by měla být co nejvíce nezávislá na ostatních stories.
Negotiable	Požadavky by měly být otevřené pro diskusi a případné úpravy.
Valuable	Splnění user story musí přinést jasnou hodnotu koncovému uživateli.
Estimable	Musí být možné odhadnout náročnost implementace (např. čas nebo potřebné zdroje).
Small	User story by měla být dostatečně malá, aby bylo možné ji dokončit během jednoho sprintu.
Testable	Musí obsahovat definovaná akceptační kritéria, podle kterých lze ověřit splnění požadavku.

Tabulka 2.1: INVEST principy pro tvorbu kvalitních user stories [14].

2.1.2 Akceptační kritéria

Zatímco user story charakterizují potřeby zákazníka, akceptační kritéria (často označována zkratkou ACC z anglického Acceptance Criteria) jasně popisují Definition of Done (zkráceně DoD) pro konkrétní user story [15] co je nutno dodržet a udělat, aby byla user story považována za splněnou. Týkají se zejména funkcionality (tlačítko dělá to co má), kvalitativních charakteristik (systém reaguje do X milisekund) a obchodní logiky [16].

Akceptační kritéria mohou být zapsána různou formou - ať už prostý seznam, kde každý bod představuje právě jedno kritérium, nebo formou scénářů, které jsou popisovány pomocí jazyka Gherkin. Tento způsob testování (kdy jsou nejprve napsány testovací scénáře v přirozeném jazyce a následně se implementují samotné testy) se nazývá Behavior-Driven Development (BDD) [17].

Feature: User Login

Scenario: Successful login with valid credentials

Given the user is on the "/login" page

When the user enters "user@example.com" into the email field

And the user enters "password123" into the password field

And the user clicks the "Login" button

Then the user should be redirected to the "/dashboard" page

And a~welcome message "Welcome back" should be displayed

Zdrojový kód 2.2: Ukázka testovacího scénáře zapsaného pomocí klíčových slov jazyka Gherkin.

Hlavní motivací pro využití Gherkin scénářů je vysoká abstrakce - scénáře jsou psány v přirozeném jazyce (jak je vidět na ukázce 2.2), což umožňuje jednodušší komunikaci mezi technicky zdatnými i netechnickými členy týmu a zároveň poskytují jasný popis požadavků a očekávaného chování [18].

Pro prostý seznam akceptačních je důležité, aby používal jasný a konzistentní jazyk a zaměřoval se hlavně na testovatelnost, nikoliv na formát zápisu [16]. Ukázka takového seznamu je uvedena na ukázce 2.3 - každý bod by měl být jasně identifikovatelný a obsahuje jedno kritérium, které lze ověřit.

Acceptance Criteria for US_01: User Login

- **ac_01 (Success):** System grants access to the dashboard when valid credentials (email and password) are provided.
- **ac_02 (Success):** Navigation to the login page is possible via the "Sign In" button on the landing page.
- **ac_03 (Failure):** System displays an error message "Invalid credentials" if the password does not match the stored hash.
- **ac_04 (Failure):** Account is temporarily locked after five consecutive failed login attempts.
- **ac_05 (Robustness):** Input fields prevent the submission of SQL injection patterns or excessively long strings.

Zdrojový kód 2.3: Ukázka akceptačních kritérií zapsaných formou strukturovaného seznamu.

V praxi bývají akceptační kritéria (spolu s uživatelskými příběhy) nejčastěji obsažena v ALM nástrojích (Application Lifecycle Management) jako jsou Jira, Azure DevOps, Rally či GitHub Projects. Tyto nástroje nevyžadují specifický formát zápisu, ale umožňují flexibilitu - ať už jde o prostý seznam, Gherkin scénáře nebo kombinaci obojího. Např. studie *From bugs to benefits: Improving user stories by leveraging crowd knowledge with cruise-ac* [19] zaměřená na generování akceptačních testů vycházela z běžných user stories a jejich textových popisů, z něhož následně generovala Gherkin scénáře, z nichž až poté byly implementovány automatizované testy.

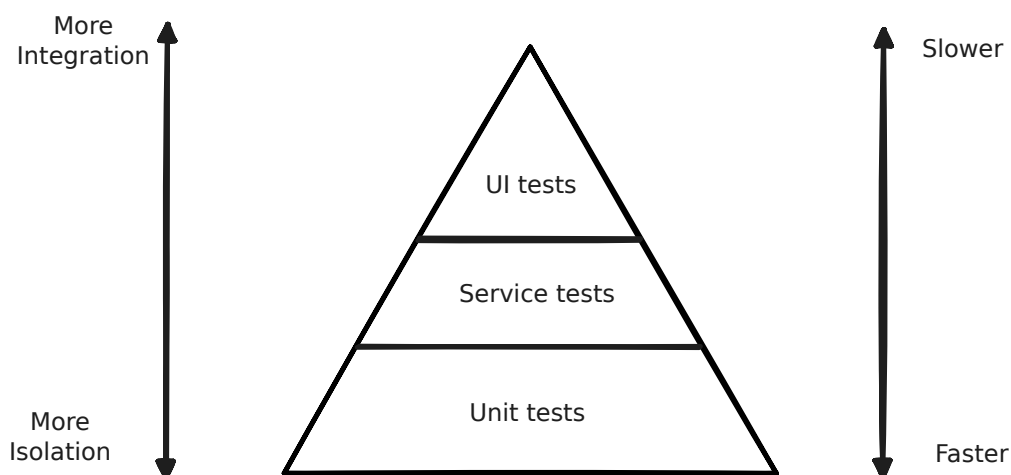
2.2 Automatizované testování softwaru

Softwarové testování lze definovat jako hledání chyb či chybějících požadavků v softwaru [20]. Dá se na něj nahlížet i z pohledu snižování rizika, což vede v posílení důvěry uživatelů a zlepšení kvality softwaru. Jak také *Software Testing Techniques: A Literature Review* [20] uvádí, testování je důležitou součástí QA - čím dříve jsou chyby odhaleny, tím levnější je jejich oprava. Testování lze rozdělit do několika tříd, jež jsou znázorněny v tabulce 2.2, která vychází z knihy [21].

Název testu	Popis
Testování podle scénářů	Tester dle předpřipravených scénářů manuálně prochází aplikaci a kontroluje očekávané chování jednotlivých testovacích kroků.
Průzkumné testování	Tester bez předem známých scénářů prozkoumává aplikaci a hledá chyby.
Regresní testy	Ověřují, zda nově přidané featury nerozbily existující funkce.
Jednotkové (unit) testy	Automatizované skripty, které izolovaně ověřují funkčnost nejmenších částí zdrojového kódu (funkce, metody).
Beta testy	Poskytnutí aplikace omezené skupině uživatelů.
Smoke testy	Rychlé testy ověřující sestavení aplikace a její stabilitu.
Konfirmační testy	Potvrzení, že aplikace po odstranění defektu funguje správně.

Tabulka 2.2: Druhy testů dle [21].

Na obrázku 2.2 je uvedena pyramida testování, která znázorňuje důležitost jednotlivých druhů testů v rámci automatizace testování [22].



Obrázek 2.2: Pyramida testování Mika Cohna.

Jednotkové testy jsou nejvíce izolované, píšou se rychleji, a tudíž jsou i levnější. Naproti tomu UI testy (User Interface - testy uživatelského rozhraní - např. end-to-end) vyžadují větší integraci a jejich vývoj je zpravidla dražší (testy se píšou déle a déle běží). Přestože jsou end-to-end testy dražší, představují nedílnou součást celé automatizace - bez těchto testů není zaručeno, že komponenty/moduly systému spolu správně komunikují a dávají tak koncovému uživateli očekávaný stav.

Při psaní testů se objevují speciální případy testů - a to testy chybně napsané či nestabilní testy. Jedná se o následující termíny [21]:

- **Falešně pozitivní (False Positive)** - Test, který selže, přestože testovaná funkcionální funguje správně.
- **Falešně negativní (False Negative)** - Test, který projde, i když testovaná funkcionální nefunguje správně.
- **Křehké (Fragile)** - Test, který selže kvůli změně v implementaci, přestože testovaná funkcionální zůstala nezměněna (např. změna názvu HTML elementu).
- **Nestabilní (Flaky)** - Test, který se chová nedeterministicky - může selhat nebo projít bez jakýchkoliv změn v kódu, což ztěžuje identifikaci skutečných problémů (např. problémy se síťovou komunikací).

Těmto problémům se při psaní testů snaží vývojáři předcházet - a to např. důrazem na determinismus testů, kontrolou správnosti testovaného chování na Code Review nebo využíváním nástrojů pro správu testů, které pomáhají identifikovat a řešit nestabilní testy.

2.2.1 E2E testování

End-to-end testování (zkráceně E2E) je typ testování, který ověřuje kompletní funkčnost systému z pohledu koncového uživatele. Využívá se přístupu tzv. metody **black-box** (černé skříňky), kdy tester nemá přístup k implementaci či struktuře systému, ale testuje systém jako jeden celek [21, 23]. V případě webových aplikací je možné nahlížet na způsoby tvorby end-to-end testů dvěma způsoby [23]:

- **Capture-Replay** - Tester nejdříve manuálně prochází aplikaci, zaznamenává své akce a následně se tyto akce převedou na automatizovaný skript.
- **Programmable Web Testing** - Testy se rovnou píšou jako zdrojový kód, který využívá frameworky pro webovou automatizaci.

Hlavní výhodou přístupu **Capture-Replay** je rychlost - testy lze vytvořit velmi rychle, nevýhodou je ale nízká kvalita a nestabilita testů, které jsou často náchylné k falešně negativním výsledkům (např. když se HTML elementy změní, program funguje stále správně, ale test selže).

Obě tyto perspektivy potřebují lokalizovat a interagovat s HTML elementy. Interakce s HTML elementy lze rozdělit do tří kategorií [23]:

- **Coordinate-based** - Využívání [X,Y] souřadnic pro interakci s elementy.
- **DOM-based** - Využití Document Object Modelu (DOM)/HTML struktury.

- **Vizuální** - Využití rozpoznání obrazu pro určení elementů na obrazovce.

Zatímco první kategorie je označována jako zastaralá [23], DOM-based přístup je náchylný na drobné změny v HTML struktuře, které mohou způsobit selhání testů a vyžadují častou údržbu. Vizuální přístup sice dokáže testovat to, co uživatel skutečně vidí, ale je náchylný na změny v rozložení stránky a režie spojená s rozpoznáváním obrazu může být vysoká. Právě snaha o řešení těchto problémů vedla k zrodu mnoha testovacích frameworků pro webovou automatizaci, které se snaží zjednodušit a zefektivnit tvorbu end-to-end testů.

2.2.2 Přehled frameworků pro webovou automatizaci

V oblasti webové automatizace existuje velké množství frameworků, které se navzájem liší svými vlastnostmi, podporovanými jazyky a přístupy k testování. Zde je uveden přehled těch nejpopulárnějších a nejčastěji používaných,

Selenium (WebDriver)

Jeden z nejrozšířenějších a nejstarších nástrojů. Podporuje širokou škálu jazyků (Java, Python, C#, JavaScript) a prohlížečů prostřednictvím WebDriverů [24]. Díky rozšířené komunitě, množství dostupných zdrojů a existujících knihoven je Selenium stále velmi populární volbou. Nevýhodou je složitější prvotní konfigurace a manuální správa testů, která může (při nesprávném použití) vést k nestabilním testům.

Robot Framework

Open-source framework založený na Pythonu, který využívá klíčová slova pro psaní testů (tzv. Keyword-Driven Testing). Architektura využívá knihovny SeleniumLibrary pro webovou automatizaci [25]. Silnou stránkou je jednoduché psaní testů - testy jsou psány čitelně v přirozeném jazyce, díky čemuž jim porozumí i netechničtí členové týmu. Se zvyšující se komplexností webových aplikací a testů může být nevýhodou nižší flexibilita a nutnost používat externí knihovny pro pokročilé funkce.

Playwright

Moderní testovací framework vyvinutý společností Microsoft. Podporuje více jazyků (JavaScript/TypeScript, Python, Java, .NET) a nabízí robustní API (Application Programming Interface) pro interakci s webovými stránkami [26]. Výhodou je rychlost testů a podpora pro více prohlížečů (Firefox, WebKit, Chromium) bez nutnosti externích driverů. Klíčovou funkcí je také Auto-waiting, který pomáhá eliminovat

křehkost testů. Nevýhodou je menší komunita a méně dostupných zdrojů, jelikož se jedná o relativně nový nástroj.

Cypress

Nástroj zaměřený na testování moderních webových aplikací, který podporuje pouze JavaScript a TypeScript. Běží přímo v prohlížeči [27], což urychluje vývoj testů a jejich případné debuggování. Disponuje také funkcí Time Travel, která umožňuje procházet test krok po kroku a vidět stav aplikace v každém kroku. Nevýhodou je omezená podpora napříč jazyky, prohlížeči a testování vícero tabů.

2.2.3 Výzvy automatizace v agilním prostředí

Studie *A conceptual model supporting decision-making for test automation in Agile-based Software Development* [28] identifikovala několik klíčových výzev, které se vyskytují při implementaci automatizace testů v agilním prostředí. Mezi tyto kritické oblasti patří zejména:

- Údržba a narůstající technický dluh
- Potřeba efektivního regresního testování
- Rozdíl mezi specifikací požadavků a výsledným kódem

Údržba a technický dluh

Jak vyplývá ze zmíněné studie, tento proces je časově náročný a náchylný k chybám. S rostoucím počtem uživatelských příběhů se manuální tvorba testů (testovacích případů) stává bottleneckem, což může vést k následnému nárůstu technického dluhu, kdy se testy stávají zastaralými a všechny scénáře nemusí být pokryty, což zvyšuje riziko chyb.

Regresní testování

Jelikož agilní vývoj vyžaduje rychlé a časté změny, je klíčové zajistit, aby po každé z nich bylo co nejdříve ověřeno na regresních testech, že nedošlo k narušení stávající funkcionality. Tento problém je možno částečně řešit automatickým generováním testů přímo z požadavků, aby se minimalizoval čas mezi implementací a testováním.

Rozdíl mezi specifikací a kódem

Tato výzva souvisí s jasností požadavků na produkt. Jelikož jsou požadavky napsány v přirozeném jazyce, může docházet k nedorozuměním či nejasnostem, které mohou vést k nesprávné implementaci či chybám. To má poté za následek:

- Nestabilní testovací orákulum - pokud mechanismus pro ověření výsledků není stabilní a předvídatelný, může docházet k velkému množství falešně pozitivních nálezů či křehkým testům.
- Náklady na opravu - opakovaná reimplementace funkcionalit vynucuje souběžnou údržbu testů, což výrazně snižuje návratnost (ROI) investic.
- Nízká testovatelnost - zaměření na testovatelnost od prvotních fází vývoje výrazně zvyšuje efektivitu automatizace.

2.3 Velké jazykové modely ve vývoji softwaru

Velké jazykové modely jsou rozsáhlé, předtrénované, statistické modely založené na neuronových sítích. Jedná se zejména o modely založené na architektuře Transformer, které obsahují miliardy parametrů a jsou trénovány na masivním množství textových dat. Díky tomuto objemu parametrů jsou schopni zachytit komplexní vzory a vztahy v jazyce (na základě statistické analýzy) a generovat text [29]. Navíc vykazují i tzv. *emergent abilities* - schopnosti jako je schopnost učit se v kontextu, plnit komplexní instrukce či řešit více krokové úlohy.

Evoluce jazykových modelů prošla 4 hlavními generacemi [30]:

- **Statistické jazykové modely (SLM)** - Vznik v 90. letech 20. století, založené na pravděpodobnostních modelech a n-gramových přístupech.
- **Neuronové jazykové modely (NLM)** - Slova mapovaná na vektory (embeddings), k predikci dalších slov využívány rekurentní neuronové sítě (RNN) a později LSTM (Long Short-Term Memory).
- **Předtrénované jazykové modely (PLM)** - Předtrénované na obrovském množství neoznačeného textu a využití fine-tuningu pro zaměření na konkrétní úlohy.
- **Velké jazykové modely (LLM)** - Současné modely s miliardami parametrů vycházející z PLM, které díky masivnímu nárůstu dat a výpočetnímu výkonu dokáží řešit mnoho úloh bez nutnosti dalšího doladování (*emergent abilities*).

Základem architektury Transformer, která byla představena v roce 2017 [31] a je převratná pro vývoj LLM, je tzv. mechanismus *Self-Attention* - ten umožňuje modelovat vliv každého slova v kontextu věty na ostatní slova, což pomáhá efektivněji zachytit a pochopit kontextové vazby.

V rámci architektury Transformer se využívají dva hlavní typy bloků - *Encoder* a *Decoder*. Zatímco *Encoder* bloky se zaměřují na zpracování vstupního textu

a jejich pochopení, Decoder bloky jsou zaměřené na generování výstupního textu na základě těchto reprezentací [30]. Na modely se pak nahlíží tak, jakou část této architektury využívají:

- **Encoder-only** - Modely využívající pouze encoder část architektury. Jsou vhodné zejména pro úlohy zaměřené na porozumění textu, například klasifikaci nebo analýzu sentimentu. Typickým příkladem je BERT.
- **Decoder-only** - Modely založené pouze na decoder části, optimalizované pro generování textu. Do této kategorie patří například rodina modelů GPT.
- **Encoder-Decoder** - Modely kombinující obě části architektury. Využívají se zejména pro úlohy transformace textu, jako je strojový překlad, sumarizace nebo otázky a odpovědi. Příkladem je model T5.

Je nutné podotknout, že i přes své schopnosti a potenciál jsou LLM stále jen nástrojem založené na predikci - nejsou schopné skutečnému porozumění, a proto může docházet k jevu zvanému halucinace - okamžik, kdy model generuje věcně nesprávný či nesmyslný text, který ale může působit věrohodně.

2.3.1 Parametry a technické aspekty LLM

Před rozбором principů fungování velkých jazykových modelů je nezbytné definovat klíčové parametry a mechanismy, které zásadně ovlivňují jejich chování, výkon a možnosti integrace. V následující části budou rozebrány tyto koncepty:

- **Tokenizace a zpracování dat:** Rozdělení textu na menší jednotky - základní metrika pro zpracování dat a účtování nákladů.
- **Context window:** Kapacita určující maximální množství vstupních a výstupních dat (tokenů) v rámci výpočetního cyklu.
- **Temperature:** Parametr řídící míru nedeterminismu a kreativity generovaného výstupu.
- **Tool Calling:** Mechanismus umožňující modelům autonomní interakci s externími systémy.
- **Prompt Caching:** Technika pro optimalizaci výkonu a snížení výpočetních nákladů.

Tokenizace a zpracování dat

Tokenizace je proces převedení vstupního textu na menší jednotky (tzv. tokeny), které mohou reprezentovat celá slova, části slov či jednotlivé znaky. To umožňuje modelům zpracovávat i neznámé výrazy pomocí algoritmů jako je Byte Pair Encoding (BPE) nebo SentencePiece [32].

Tyto tokeny se dále dělí na další kategorie uvedené v tabulce 2.3, které jsou i základní metrikou pro pricing jednotlivých poskytovatelů (providerů) v rámci používání jejich modelů.

Token	Popis	Příklad	Cena
Input	Data zasílaná modelu ke zpracování.	Uživatelský dotaz.	Nižší
Output	Sekvence tokenů generovaná modelem jako odpověď.	Odpověď modelu.	Vyšší
Reasoning	Externalizovaný výpočetní stav předávaný mezi cykly generování.	Mezikroky řešení.	Dle implementace
Cached	Data v mezipaměti pro opakované využití.	Části dlouhého dokumentu.	Nejnižší

Tabulka 2.3: Přehled běžně používaných druhů tokenů v LLM.

Přestože reasoning tokeny mohou připomínat lidské uvažování, model po dalším cyklu tuto znalost ztrácí - tyto tokeny mu tak slouží jako State over Tokens, kam si odkládá mezivýsledky, aby věděl, jak při dalším kroku pokračovat [33].

Context window

Kontextové okno definuje maximální počet tokenů, které je model schopen zpracovat v rámci jednoho výpočetního cyklu. Zatímco starší generace modelů operovaly v řádu nižších tisíců tokenů, současné modely již zvládají pracovat v řádu nižších milionů. Průkopníkem v této oblasti se stala společnost Google, která jako první u svých modelů řady Gemini ¹ představila kapacitu přesahující jeden milion tokenů. Podobné limity nyní postupně adoptují i další poskytovatelé, jako například Anthropic u modelů Claude v rámci platformy Bedrock ².

Temperature

Parametr využívaný modely k určení míry náhodnosti a diverzity generovaného výstupu [34]. Čím nižší je hodnota temperature, tím konzervativnější a determinističtější je výstup modelu. S vyššími hodnotami stoupá riziko halucinací, ale model

¹Dokumentace Google AI

²Dokumentace AWS Bedrock (Anthropic)

nabízí vyšší míru kreativity. Pro úlohy vyžadující přesnost a konzistenci se doporučuje nastavit hodnotu `temperature` na 0. Některé modely (např. v rámci Azure OpenAI) tento parametr neumožňují konfigurovat, protože je pevně definován na úrovni modelu ³.

Tool Calling

Tool Calling (volání nástrojů) je schopnost modelu autonomně využívat externí aplikace či služby (jako jsou firemní API, vyhledávače...), čímž překonávají svá omezení jako jsou např. zastaralé informace. Pro správné použití je nutné zajistit, aby model mohl správně identifikovat záměr uživatele, vybrat vhodný nástroj a správně nastavit vstupní parametry[35]. Tuto schopnost lze modelu naučit buď pomocí fine-tuningu na ukázkových interakcích, nebo technikou `in-context-learningu`, kdy se do promptu, který se zasílá modelu, vloží definice jednotlivých nástrojů, aby věděl, jak je využívat. Tato vlastnost není podporována u všech modelů, a proto je třeba předem zkontrolovat dokumentaci, zda je pro daný model Tool Calling dostupný.

Prompt Caching

Prompt Caching je inovativní technika, jež výrazně zrychluje inferenci modelu a snižuje výpočetní nároky tím, že recykluje interní výpočty modelu [36]. Vychází z toho, že uživatelské dotazy obsahují často stejné bloky zpráv (např. systémový prompt, specifikace nástrojů, přiložené dokumenty...), které se mezi dotazy nemění. Namísto přepočítávání těchto bloků si model ukládá jejich výpočty do mezipaměti a při dalším dotazu, který obsahuje identické bloky, si je pouze načte z mezipaměti a spojí, čímž se zkracuje doba zpracování dotazu a to bez jakéhokoli dopadu na kvalitu finálního výstupu. Tato technika je podporována zejména u novějších modelů a je nutné zkontrolovat dokumentaci, zda je pro daný model dostupná.

2.3.2 Srovnání LLM modelů a pricing

Výběr konkrétního modelu závisí na mnoha faktorech - ať už se jedná o výkon, ale i rovnováhu mezi cenou, rychlostí a dostupností funkcí (Tool Calling, Temperature, podporovaný vstup - text, obrázky). V tabulce 2.4 jsou uvedeny některé z nejnovějších modelů, které vyšly od druhé poloviny roku 2025.

Veškeré cenové údaje jsou převzaty z oficiálních dokumentací jednotlivých poskytovatelů.⁴ Je důležité poznamenat, že každý poskytovatel uplatňuje odlišnou cenovou strategii. Například, jak je patrné z tabulky 2.4, Google AI rozlišuje u některých modelů dvoustupňový pricing - sazba za milion tokenů se automaticky

³Dokumentace Microsoft k Azure OpenAI

⁴Podrobné specifikace dostupné v dokumentacích: Anthropic, Google AI, OpenAI, DeepSeek, Together AI a Moonshot.

Provider	Model	In (\$/1M)	Out (\$/1M)	Context
Anthropic	Claude 4.6 Sonnet	3.00	15.00	1M
	Claude 4.6 Opus	5.00	25.00	1M
OpenAI	GPT-5 (Standard)	1.25	10.00	400k
	GPT-5.3 Codex	1.75	14.00	400k
Google AI	Gemini 3.1 Pro	2.00/4.00	12.00/18.00	1M
	Gemini 3.1 Flash-Lite	0.25	1.50	1M
Ostatní	DeepSeek-V3.2	0.28	0.42	128k
	Llama 4 Scout	0.18	0.59	1M
	Moonshot Kimi K2	0.60	2.50	262k

Tabulka 2.4: Vybrané modely vydané od H2 2025 a jejich cenová politika.

navyšuje u dotazů, jejichž celkový kontext přesáhne hranici 200 000 tokenů. A ještě větší rozdíly jsou u přístupu ke cachování - zatímco např. OpenAI sází na plnou automatizaci a slevy pro krátkodobě identické začátky promptů, Anthropic a Google vyžadují explicitní definici ukládaných bloků, což umožňuje efektivnější a dlouhodobější uchování rozsáhlých dat.

Modely, které jsou veřejnosti dostupné (včetně jejich metadat o nástrojích, kontextových oknech apod.), je možné najít na stránkách `models.dev`, která tak slouží jako centrální repozitář, který usnadňuje vývojářům výběr a porovnání modelů pro jejich konkrétní potřeby.

2.3.3 Limity generování

Ačkoliv LLM prokazují schopnost psát zdrojové kódy, při generování testů na GUI (Graphical User Interface) narážejí na omezení spojená s absencí interaktivity s aktuálním prostředím. Hlavními výzvami jsou:

- **Halucinace** - Jazykové modely si mohou při nedostatku kontextu vymýšlet neexistující elementy/selektory. S tímto chováním se setkal Horínek ve své diplomové práci [37], kdy se modely snažily použít atribut `aria`, který ovšem daným frameworkem nebyl vůbec podporován.
- **Statický kontext** - Model při generování testu nevidí reálný stav DOMu v čase běhu a nedokáže reagovat na dynamické změny na stránce. Tento problém demonstruje Horínek [37] na případu, kdy model generoval výběr prvku z rozbalovací nabídky, ovšem tyto hodnoty se při každé inicializaci databáze měnily a pevně vygenerované selektory tak v době běhu selhávaly.

- **Absence zpětné vazby a zotavení z chyb** - Bez možnosti interakce model nemůže v průběhu testu ověřit, zda jeho předchozí kroky byly úspěšné. Na nutnost řešit tento problém při návrhu nástrojů pro generování GUI testů poukazuje ve své práci Holický [38]. Pokud je test vygenerován čistě staticky, se-bemenší nepřesnost nebo neočekávané chování uživatelského rozhraní vede k selhání celého skriptu. Bez implementace dodatečné zpětné vazby (např. předání chybové hlášky zpět modelu) se systém z takové chyby nedokáže zotavit a test zůstává nevalidní.

Tyto tři body ukazují, že generovat kód staticky pro moderní proměnlivé aplikace nemusí stačit. Pro překonání těchto limitů je potřeba opustit paradigma jednorázového promptu a přijít s řešením, které dokáže s aplikací aktivně interagovat, pozorovat výsledky akcí a následně je verifikovat. Tímto směrem se ubírají tzv. **agentní systémy** - systémy, které vidí do DOMu a dokážou dynamicky reagovat.

2.4 Agentní systémy a autonomní testování

2.4.1 Definice a charakteristika autonomních agentů

S narůstající komplexností úloh, které jsou na modely umělé inteligence kladeny, se objevuje potřeba systémů k migraci od tradičního jednorázového generování k interaktivním a adaptivním systémům, které dokáží reagovat na dynamické prostředí. Zatímco standardní neagentní přístup (tzv. *non-agentic workflow*) funguje na principu, který generuje výstup na základě jednorázového uživatelského dotazu a čeká na další lidský pokyn [39], agentní systémy představují zásadní změnu paradigmatu. V agentním pojetí se LLM stává jakýmsi “mozkem“, jenž proaktivně řídí provádění úkolů, samostatně se rozhoduje o dalším postupu a v případě potřeby iterativně reviduje své kroky bez nutnosti neustálých zásahů člověka [39, 40, 41].

Ve své základní podobě se skládá ze tří klíčových komponent [41]:

- **Model** - Konkrétní velký jazykový model, který řídí uvažování, rozhodování a celkovou exekuci workflow agenta.
- **Nástroje (Tools)** - Externí funkce nebo API, jenž agent využívá k interakci s vnějším prostředím.
- **Instrukce (Instructions)** - Explicitní pravidla definující agentovo chování - zmenšují prostor pro nejednoznačnosti což vede k efektivnějšímu chování a minimalizaci halucinací.

Pro plné fungování v rámci komplexnějších architektur je často zmiňují i další klíčové aspekty, které pomáhají agentům řešit náročnější úlohy. Mezi ně patří paměť

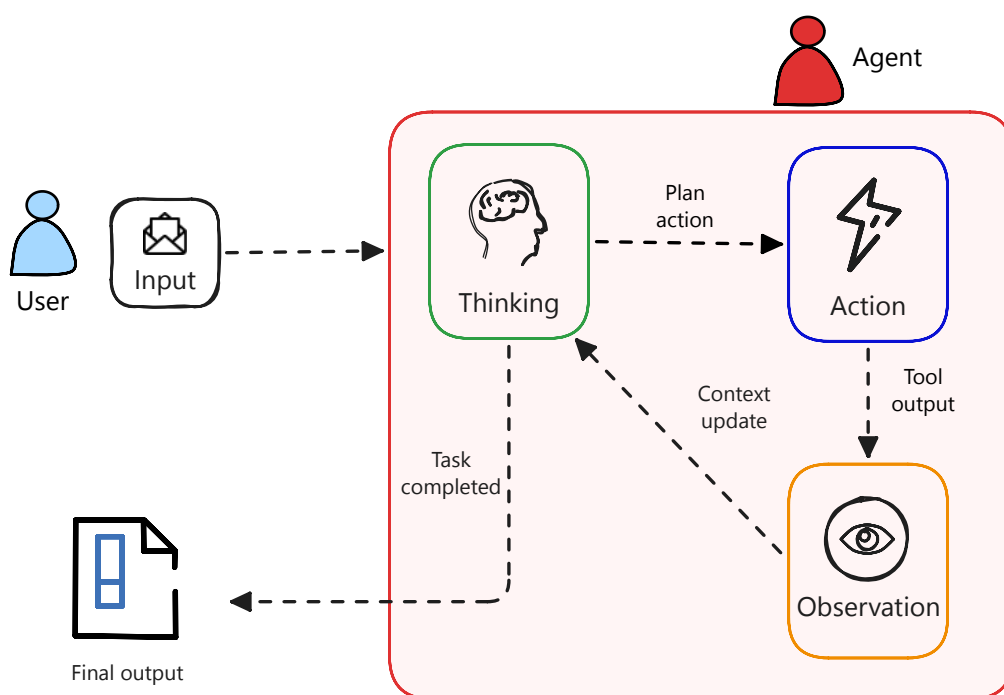
(Memory), která umožňuje agentovi uchovávat a využívat informace z předchozích interakcí, a dále plánování (Planning), jež umožňuje agentovi dekomponovat složitý problém na sadu menších proveditelných kroků [42].

2.4.2 Architektura ReAct: Spojení uvažování a jednání

Vzor ReAct (Reasoning and Acting) je klíčový rámec pro pochopení toho, jak autonomní agenti “přemýšlejí” a jednají [40]. Tento vzor umožňuje jazykovému modelu krok za krokem uvažovat o problému a konkrétními příkazy spouštět nástroje. Díky tomu model (místo klasického přístupu, kdy se vygeneruje celý výstup najednou) běží v cyklu, jenž mu umožní iterativně vytvářet, monitorovat a revidovat své kroky. Jádrem tohoto vzoru je opakující se smyčka, která se skládá ze tří fází [43]:

- **Thought (Myšlenka)** - Interní monolog, v němž model analyzuje aktuální stav. Na základě svých předchozích instrukcí a aktuálního kontextu hodnotí, co vidí, jaký by měl být další krok a jaké nástroje by měl k jeho dosažení použít.
- **Action (Akce)** - Konkrétní volání toolu, který model na základě předchozí úvahy zvolil.
- **Observation (Pozorování)** - Získání zpětné vazby z prostředí po provedení konkrétní akce - tou může být např. pozměněný stav DOMu, nová URL adresa či chybová hláška.

Celý tento proces je zobrazen na obrázku 2.3. Ten začíná přijetím vstupu od uživatele a pokračuje do agentovy iterativní smyčky, kde se střídají fáze uvažování, jednání a následného pozorování. Celý proces končí v okamžiku, kdy agent v rámci fáze myšlení dospěje k závěru, že úloha je splněna a vygeneruje výsledný výstup pro uživatele.



Obrázek 2.3: Vzor ReAct: Spojení uvažování a jednání pro autonomní agenty.

Pro QA a automatizované testy představuje tento vzor zásadní posun - díky explicitním krokům je celý proces generování plně transparentní, interpretovatelný a auditovatelný. Uživatel přesně vidí úvahu, která agenta vedla k výběru určitého lokátoru, což výrazně usnadňuje ladění a zvyšuje tak spolehlivost celého systému.

2.4.3 Model Context Protocol (MCP)

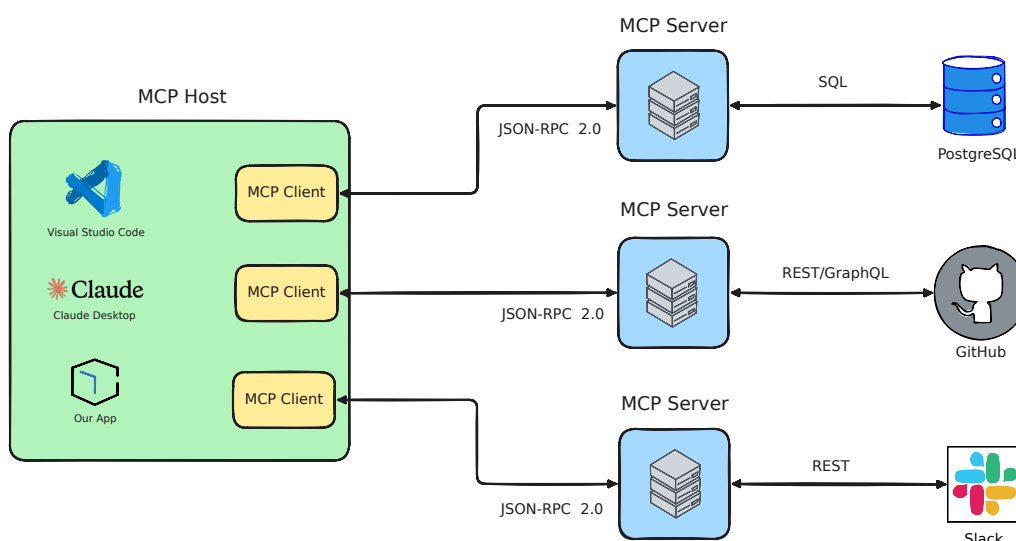
Model Context Protocol (MCP), představený firmou Anthropic ke konci roku 2024, je otevřený standard pro interoperabilitu, jehož cílem je sjednotit a zjednodušit způsob, jakým se modely umělé inteligence propojují s externími nástroji, systémy a daty.⁵ MCP, jenž si vysloužil přezdívku “USB-C for AI applications“, poskytuje jednotné a standardní rozhraní pro komunikaci - je postaven na robustním modelu komunikace Client-Host-Server s využitím formátu JSON-RPC 2.0 [42]:

- **Client** - Protokolová komponenta zabudována do hostitelské aplikace, která udržuje spojení s jedním konkrétním MCP serverem.
- **Host** - Aplikace, která navazuje komunikaci a interaguje s uživateli.

⁵Oficiální dokumentace MCP

- **Server** - Samostatný nezávislý proces (běžící lokálně nebo vzdáleně), který zpřístupňuje konkrétní funkce (tools) - slouží jako obálky kolem externích systémů (API, databáze, CLI nástroje...).

Obrázek 2.4 zobrazuje ukázkový ekosystém MCP, v němž hostitelské aplikace - od vývojových prostředí (IDE) jako je Visual Studio Code až po vlastní specializované platformy - komunikují se servery pomocí jednotného rozhraní. Díky této architektuře servery fungují jako univerzální adaptéry, překládají požadavky modelu do specifických protokolů (SQL, REST API) pro externí služby typu GitHub či Slack, což radikálně zvyšuje modularitu a zjednodušuje integraci nových funkcí do agentních systémů.



Obrázek 2.4: Ukázková architektura Client-Host-Server v rámci Model Context Protocolu.

Zásadní význam MCP pro autonomii agentů spočívá v dynamické správě dostupných nástrojů a dat - agent se skrze protokol MCP dokáže automaticky dotázat serveru na seznam dostupných nástrojů a získat k nim přesné definice parametrů. Díky tomu agent v hostitelské aplikaci sám zjistí, jaké nástroje může v rámci své úlohy využívat, a to bez nutnosti explicitního předávání těchto informací. Tento přístup umožňuje agentům být skutečně autonomními - dokážou se adaptovat na měnící se prostředí a využívat nové nástroje, které se objeví, aniž by bylo potřeba měnit jejich základní instrukce či prompt.

2.5 Shrnutí

V této kapitole byl podrobně popsán teoretický základ propojující agilní vývoj, automatizované testování a umělou inteligenci. Byly definovány způsoby specifikace požadavků, které slouží jako primární vstup pro automatizaci, a analyzovány výzvy spojené s údržbou a regresním testováním. Zároveň byly představeny možnosti moderních jazykových modelů a jejich klíčové parametry, jež ovlivňují výsledné chování modelů.

Klíčovým poznatkem kapitoly je vymezení autonomních agentů, kteří díky využití iterativního uvažování a standardizovaného protokolu MCP dokáží překonat limity statického generování a zajistit interaktivní a stabilní testování v dynamickém prostředí webu. Po získání těchto znalostí je možné přejít k návrhu systému, kde budou definovány konkrétní požadavky a architektura řešení, čemuž se podrobně věnuje následující kapitola.

Analýza a návrh systému

3

Tato kapitola využívá teoretické poznatky z předchozí kapitoly a přechází k praktickému návrhu systému pro automatizované generování E2E testů s využitím velkých jazykových modelů. Text se v úvodu zaměřuje na analýzu struktury vstupních dat z běžně využívaných ALM nástrojů, z čehož jsou následně odvozeny konkrétní funkční a nefunkční požadavky. Hlavní část kapitoly je poté zaměřena na návrh koncepční architektury - od doménového modelu a distribučního modelu, přes představení a vysvětlení jednotlivých systémových komponent, až po specifikaci exekuční smyčky autonomního agenta. Na závěr kapitoly je pak navržen prototyp uživatelského rozhraní, který zobrazuje navržené řešení a demonstuje jeho klíčové funkce.

3.1 Analýza struktury a sémantiky vstupních artefaktů

Důležitým faktorem pro úspěšnou automatizaci testů je správné pochopení a zpracování vstupních artefaktů, které jsou základem pro generování testů. Jak bylo zmíněno v kapitole 2, tyto artefakty mohou mít různou formu - od textových dokumentů (např. uživatelské příběhy, akceptační kritéria) až po strukturované formáty (např. Gherkin). V ALM nástrojích uživatelské příběhy často nemají jednotnou strukturu, což výrazně ztěžuje jejich zpracování. Zejména nástroje jako Jira nabízejí uživatelům velkou flexibilitu (nejen) při správě uživatelských příběhů. K prozkoumání této problematiky lze využít a nahlédnout na již existující Jira instance, které jsou přístupné veřejnosti. Zde je několik existujících instancí, ze kterých lze získat cenné informace o struktuře a sémantice uživatelských příběhů:

- **Atlassian Jira**¹ - Instance zaměřená na vývoj softwaru samotné společnosti Atlassian (poskytovatel Jira software). Projekty jsou organizovány podle jednotlivých produktů (např. Jira Software, Confluence, Bitbucket...).

¹<https://jira.atlassian.com/secure/BrowseProjects.jspa>

- **Apache Jira**² - Instance zaměřená na vývoj open-source softwaru. Projekty jsou organizovány podle jednotlivých open-source projektů (např. Hadoop, Tomcat, Kafka...).
- **MongoDB Issue Tracker**³ - Instance zaměřená na vývoj databázových systémů. Projekty jsou organizovány logicky dle menších celků (např. MCP server, Shell, drivery dle programovacích jazyků...).
- **Mojang Studios Bug Tracker**⁴ - Instance zaměřená na vývoj her (Minecraft). Jednotlivé projekty jsou dílčí částí her (např. Java Edition, Bedrock Edition, Launcher...).

Před samotnou investigací jednotlivých instancí je vhodné definovat metodiku, jakým způsobem byla data pro analýzu získávána. Vzhledem k obrovskému množství ticketů ve veřejných repozitářích nelze spoléhat na manuální procházení.

Pro průzkum a extrakci relevantních uživatelských příběhů byl využit nativní dotazovací jazyk **JQL (Jira Query Language)**. Tento nástroj umožňuje filtrování napříč projekty na základě mnoha atributů, jako jsou typ úlohy, stav řešení, časové okno vytvoření či fulltextové vyhledávání v obsahu ticketu.

```
project = "TRELLO"  
AND issuetype != Bug  
AND attachments IS NOT EMPTY  
AND assignee IS NOT EMPTY  
AND created >= startOfYear(-1)
```

Zdrojový kód 3.1: Ukázka dotazu v jazyce JQL pro filtrování uživatelských příběhů.

Na ukázce 3.1 je zobrazen dotaz zapsaný v jazyce JQL, který kombinuje několik vlastností:

- **Filtrování dle projektu a typu** - Filtrování ticketů z konkrétního projektu a vyloučení bugů.
- **Validace metadat** - Filtrování ticketů na základě přítomnosti metadat.
- **Časové funkce** - Filtrování ticketů na základě času.

²<https://issues.apache.org/jira/issues/>

³<https://jira.mongodb.org/secure/BrowseProjects.jspa>

⁴<https://bugs.mojang.com/>

Díky těmto dotazům je možné snadno získat relevantní vzorky dat z jakékoliv veřejné instance a následně je porovnat napříč ostatními instancemi.

Zkoumání výše zmíněných veřejných instancí ukázalo, že navzdory využití stejného softwarového nástroje se přístup k evidenci požadavků velmi liší. Tyto rozdíly jsou dány především doménou projektu, firemní kulturou a zavedenými procesy konkrétní organizace.

Metadata ticketů

Typ ticketů není nijak omezen. Napříč instancemi se vyskytují různé druhy ticketů, přičemž každý z nich může mít definovanou svou zcela vlastní strukturu (např. Bug může mít pole pro kroky reprodukce, zatímco User Story pole pro akceptační kritéria). Nejčastěji používanými typy jsou **User Story** a **Bug**. Dále se v rámci instancí vyskytují i další typy jako jsou: New Feature, Task, Improvement, Test, Question, Sub-task, RTC (Request to Change), Epic, Dependency, Suggestion, Support Request, Build Failure, Technical Debt a další.

Co se týče priorit, opět není žádný jednotný systém - a to i v rámci jedné instance. Například v rámci Apache Jira se vyskytují priority jak číselné (P0, P1, P2...), tak i textové (Critical, Major, Minor...). V rámci Atlassian Jira se vyskytují priority pouze textové (Highest, High, Medium, Low). Někdy dochází i k míchání těchto přístupů - např. v rámci MongoDB Jira se vyskytují priority Major - P3, Minor - P4, Trivial - P5, a další.

K určování rozsahů ticketů se využívají různé metriky - od odhadů v rámci story points až po odhady v rámci časových jednotek (hodiny, dny...). Co se týče stavu řešení, i zde není jednotný systém - nejčastější stavy jsou To Do, In Progress, Done, ale v rámci instancí se vyskytuje i mnoho dalších stavů - Ready to Commit, Awaiting Feedback, Blocked. Když je ticket uzavřen, i zde nastává mnoho různých způsobů, jak se uzavření dokumentuje - Closed, Delivered, Resolved, Rejected, Duplicate, Won't Fix, Cannot Reproduce a další.

Description ticketu

V rámci popisu ticketů je společná jedna věc - celý popis je jeden velký textový blok, který může obsahovat jakýkoliv obsah - od čistého textu, přes tabulky a úryvky kódu. Řádná struktura (As a [role], I want [feature], So that [benefit]) se vyskytovala velmi ojediněle - většinou se popis skládal z jednoho odstavce bez jakékoliv struktury či akceptačních kritérií.

Akceptační kritéria

Akceptační kritéria lze v rámci Jira instance definovat jako samostatné pole, ale žádný ze 4 zmiňovaných projektů tuto možnost nevyužívá - to může být způsobeno tím, že se jedná o veřejné repozitáře, kde není rozšířená role Product Ownera, který by měl na starosti definici akceptačních kritérií.

3.2 Specifikace požadavků na systém

Na základě předchozí analýzy vstupních artefaktů a teoretických poznatků jsou v této sekci definovány konkrétní požadavky na navrhovaný systém. Vzhledem k vysoké různorodosti vstupních dat bude systém navržen tak, aby mohl být aplikován na širokou doménu projektů, a to bez nutnosti přizpůsobení pro konkrétní instanci.

3.2.1 Vymezení rozsahu generovaných testů

Systém bude primárně navržen pro generování automatizovaných end-to-end testů s důrazem na ověřování chování uživatelského rozhraní (GUI), obchodní logiku a vizuální interakci s aplikací. Systém naopak nebude koncipován pro verifikaci nefunkčních požadavků, jako jsou výkonnostní testy či bezpečnostní testy. Pokud se v akceptačních kritériích objeví nefunkční požadavky, systém bude instruován, aby se zaměřil výhradně na funkční aspekty - měření časových metrik v rámci E2E testování v prohlížeči může být velmi závislé na výkonu testovacího prostředí (zařízení, síť), což by mohlo vést k nestabilním testům a falešně pozitivním výsledkům.

3.2.2 Funkční požadavky (FR) na systém

Funkční požadavky definují konkrétní chování, které musí systém vykazovat, aby naplnil potřeby uživatelů. Vzhledem k využití velkých jazykových modelů definují nejen samotné vygenerování testů, ale i celý proces - tzv. *agentic workflow*. K určení priorit byla využita metoda MoSCoW (M - Must have, S - Should have, C - Could have, W - Won't have) - výsledné požadavky jsou uvedeny v tabulce 3.1.

ID	Popis požadavku	Priorita
FR-01	Sestavení kontextu: Systém musí přijmout strukturovaná vstupní data (uživatelský příběh a kontext testované aplikace) a transformovat je do vhodné formy agentovi.	Must
FR-02	Explorativní analýza: Před generováním kódu agent proaktivně naviguje testovanou aplikací a zkoumá strukturu DOMu pomocí dostupných nástrojů.	Must
FR-03	Křížová verifikace: Agent musí systematicky porovnat každé akceptační kritérium se skutečným chováním aplikace.	Must
FR-04	Řízené selhání: Při nefunkční či neimplementované funkcionalitě agent vygeneruje test s očekávaným správným chováním, ale označí jej speciálním příznakem.	Should
FR-05	Standardizace výstupu: Vygenerovaný kód musí být validní, přeložitelný a spustitelný.	Must
FR-06	Popis implementace: Vygenerovaný test by měl obsahovat krátké shrnutí, které popisuje výsledky verifikace oproti akceptačním kritériím.	Should
FR-07	Vizualizace procesu: Uživatelské rozhraní by mělo zobrazovat flow agenta a vhodně jej vizualizovat uživateli.	Should
FR-08	Exekuce testů: Systém by mohl umožňovat přímé spuštění vygenerovaných testovacích scénářů z uživatelského rozhraní bez nutnosti přepínání do jiného prostředí (např. IDE, CLI).	Could
FR-09	Regenerace na základě zpětné vazby: Systém by mohl podporovat regeneraci testů - v případě selhání běhu testu by měl agent přijmout chybový report a na jeho základě opravit testovací kód.	Could
FR-10	Dynamická konfigurace agenta: Uživatel by mohl mít možnost upravovat konfiguraci agenta dle potřeby - volba LLM modelu či konkrétních toolů.	Could

Tabulka 3.1: Specifikace funkčních požadavků na systém.

3.2.3 Nefunkční požadavky (NFR) na systém

Nefunkční požadavky se zaměřují na kvalitu, výkon, bezpečnost a spolehlivost celého systému. V tabulce 3.2 jsou uvedeny klíčové nefunkční požadavky, které musí

být zohledněny při návrhu a dodrženy při implementaci.

ID	Popis požadavku	Priorita
NFR-01	Architektura MCP: Logika agenta musí být oddělena od fyzického ovládání prohlížeče pomocí standardizovaného MCP protokolu.	Must
NFR-02	Odolnost vůči selhání: Pokud volání nástroje selže, agent se nesmí zacyklit, ale musí chybu reflektovat či zvolit jiný přístup k vyřešení úlohy.	Must
NFR-03	Bezpečnost API klíčů: Citlivé autentizační údaje musí být bezpečně spravovány backendovou částí aplikace.	Must
NFR-04	Modularita modelů: Systém by měl umožňovat snadné přidání nových LLM modelů bez významného vlivu na celkovou architekturu.	Should
NFR-05	Rozšiřitelnost nástrojů: Architektura by mohla umožňovat snadné přidávání nástrojů bez nutnosti refaktORIZACE jádra agenta.	Could

Tabulka 3.2: Specifikace nefunkčních požadavků na systém.

3.3 Koncepční architektura systému

3.3.1 Doménový model aplikace

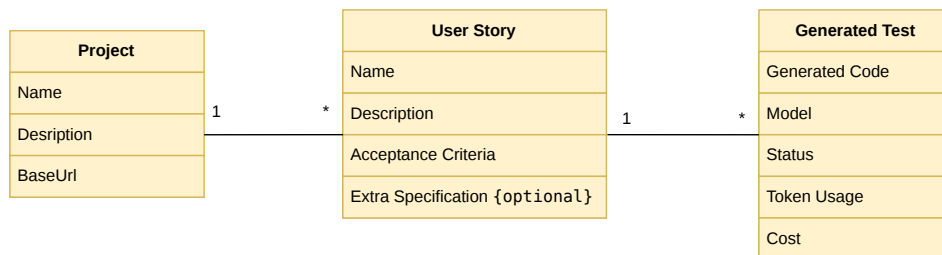
Jak ukázala analýza veřejných instancí ALM nástrojů (viz sekce 3.1), vstupní artefakty napříč projekty a odvětvími postrádají jednotnou strukturu. Pokud by agent přistupoval přímo k surovým datům z těchto různorodých systémů, byla by extrakce informací velmi nespolehlivá a náročná. Tento systém proto zavádí vlastní, normalizovaný doménový model, který funguje jako standardizovaná mezivrstva mezi nestrukturovanými daty z externích nástrojů a čistým kontextem, který bude agentovi předán. Díky tomu bude možné zajistit, že agent bude pracovat s předvídatelným a konzistentním formátem dat, což výrazně zlepší spolehlivost a efektivitu generování testů.

Struktura obsahuje tři klíčové entity:

1. **Projekt** - Reprezentuje konkrétní testovanou aplikaci. Obsahuje její základní informace (název, popis, URL).
2. **Uživatelský příběh** - Reprezentuje jeden konkrétní ticket. Obsahuje relevantní informace pro generaci testů - název, popis, akceptační kritéria a nepovinný dodatečný kontext (např. odkazy na dokumentaci, konkrétní požadavky na testy...).

3. **Vygenerovaný test** - Reprezentuje testovací scénář vygenerovaný agentem. Kromě samotného kódu obsahuje i metadata o generování testu (název, použitý model, množství tokenů, celková cena).

Model je znázorněn na obrázku 3.1. Mezi entitami jsou definovány relace - mezi projektem a uživatelským příběhem je relace 1:N - projekt může mít více uživatelských příběhů, ale každý uživatelský příběh patří pouze jednomu jedinému projektu. V praxi by se našly i výjimky, kdy by uživatelský příběh mohl být sdílen mezi více projekty (např. společná funkcionality či separátní projekt pro frontend a backend), ale dle dříve zanalyzovaných ticketů (které v drtivé většině případů obsahovaly vazbu pouze na jeden projekt) a pro zjednodušení modelu byla tato možnost vyloučena. Relace mezi uživatelským příběhem a vygenerovaným testem je také 1:N - jeden uživatelský příběh může mít více vygenerovaných testů, ale každý vygenerovaný test patří pouze jednomu jedinému uživatelskému příběhu.



Obrázek 3.1: Doménový model systému.

Ostatní prvky identifikované v analýze (např. metadata ticketů) jsou záměrně vynechány, protože pro generování testů nepřinášejí žádnou přidanou hodnotu a navíc jejich zahrnování v agentově kontextu by mohlo vést k zvětšení množství tokenů a tím i nákladů na generování testů.

3.3.2 Distribuce a perzistence dat

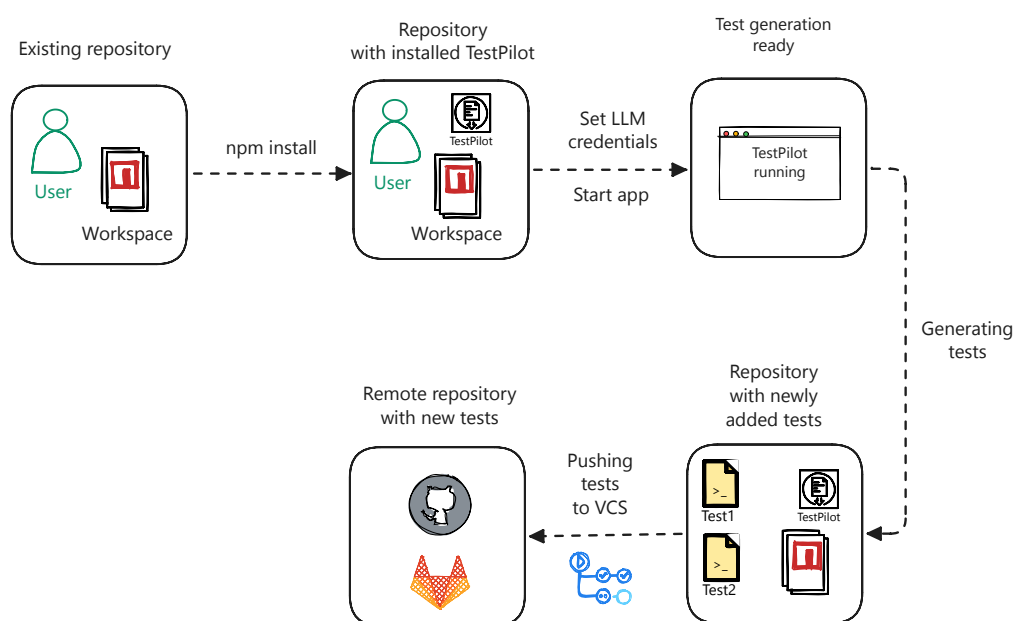
Před samotnou definicí architektury agenta je nejprve nutné rozhodnout, jaký způsob distribuce systému zvolit a jakým způsobem ukládat data. Původní myšlenkou bylo využít tradiční architekturu s nasazením relační databáze (např. PostgreSQL). Výhodou by bylo, že veškeré testy a uživatelské příběhy by byly uloženy centralizovaně v jedné konkrétní databázi, což by umožnilo snadné sdílení a přístup napříč uživateli a vývojovými týmy. Z tohoto přístupu se nakonec ustoupilo kvůli následujícím důvodům:

- **Ukládání výsledných testů** - Největší úskalí by bylo definování tzv. *source of truth* pro vygenerované testy. Pokud by se testy ukládaly do centrální databáze, vznikla by potřeba synchronizace mezi touto databází a prostředími, kde by se testy používaly (např. IDE, CI/CD pipeline). Databáze by tak obsahovala veškeré vygenerované testy a bylo by možné trasovat, kdy a kým byly testy vygenerovány, ale zároveň by stejně bylo potřeba testy ukládat do repozitářů, aby bylo možné je spouštět a nakonec by stejně zdrojem pravdy pro testy zůstaly repozitáře.
- **Správa databáze** - S produkčním nasazením by bylo potřeba zajistit správu a údržbu databáze, což by vývojáře odvádělo od hlavního cíle - jednoduché generování testů.
- **Závislost na třetí straně** - Program by byl závislý na další externí službě - k závislosti na dostupnosti LLM modelů by se tak přidala i závislost na dostupnosti a správě databáze. Navíc pokud by aplikace běžela jako samostatná služba, vývojáři by museli z vývojových prostředí otevírat další rozhraní, což by mohlo vést k nižší adopci nástroje.

Tato analýza vede k závěru, že tradiční přístup s databází by nedisponoval uživatelskou přívětivostí, která je pro adopci nástroje klíčová. Všechny výše zmíněné nevýhody řeší tzv. *local-first* přístup, který umožňuje vývojářům pracovat s nástrojem přímo z jejich lokálního prostředí bez nutnosti přepínání do jiných rozhraní či závislosti na externích službách. Systém bude distribuován jako samostatný instalovatelný balíček (prostřednictvím správce balíčků NPM - Node Package Manager), který si uživatel pouze nainstaluje do svého projektu, kde chce testy generovat (případně může jednou globálně, pokud chce testy generovat napříč vícero projekty). Následně jen pomocí jednoduchého CLI příkazu spustí systém lokálně a celý proces generování testů proběhne přímo na jeho zařízení. Jediné nastavení, které bude muset provést, je konfigurace přístupových údajů k LLM modelům, což bude záležitost několika řádků v konfiguračním souboru, který bude umístěn lokálně v projektu.

Díky tomu budou mít vývojáři systém přímo ve svém IDE bez nutnosti otevírání dalších rozhraní, *source of truth* bude přímo v repozitáři, do něhož bude aplikace testy ukládat a bude je moci i číst a rovnou spouštět, a navíc veškerá historie generovaných testů, uložených projektů a uživatelských příběhů, bude uložena v souboru, který bude možno distribuovat přes Git.

Celé vývojářské flow, které bylo popisováno, je zobrazeno na obrázku 3.2, kde navržený systém je označován jako TestPilot.



Obrázek 3.2: Vývojářské flow s využitím local-first přístupu.

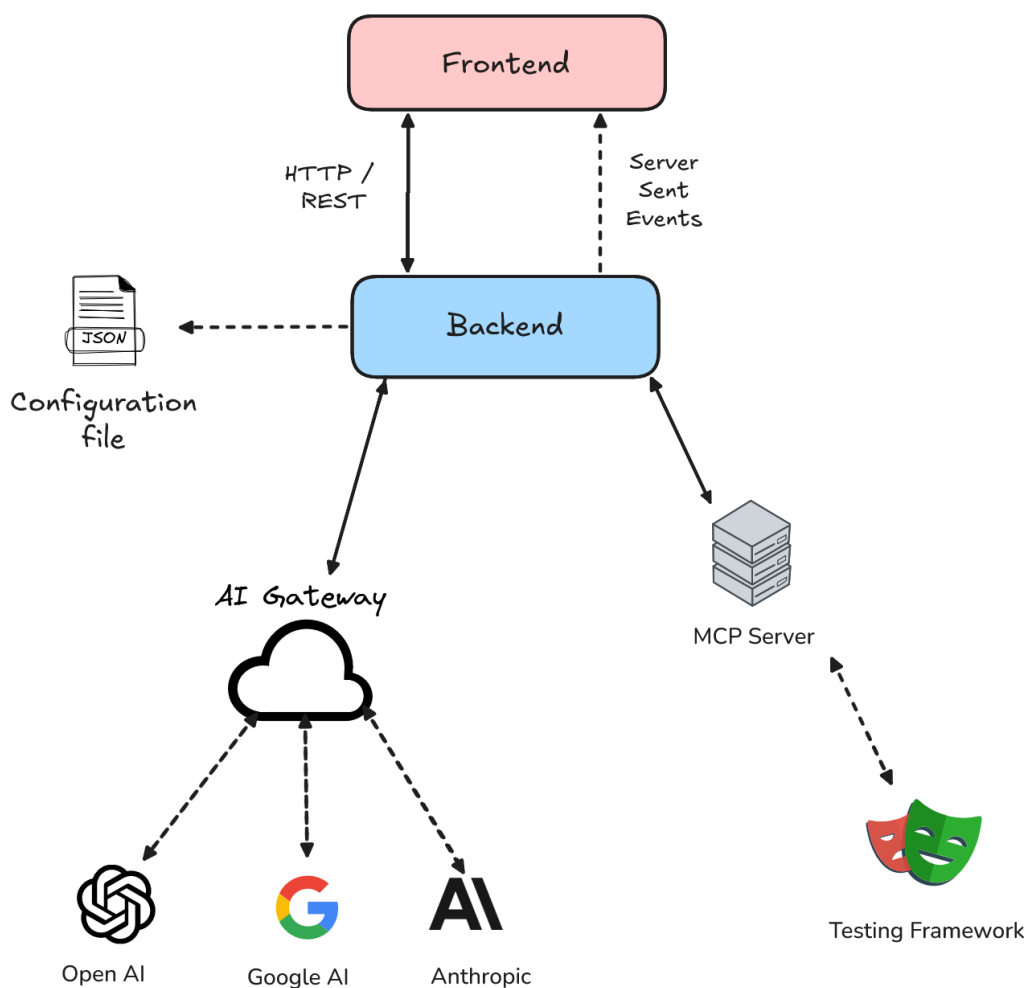
3.3.3 Architektura systémových komponent

Nejprve v úvahu přicházela možnost navrhnout systém jako čistě backendovou službu, která by běžela jen v terminálu a komunikovala s uživatelem pouze prostřednictvím CLI, ale tento přístup by nebyl uživatelsky přívětivý pro ne-technické uživatele. Na základě toho bude systém navržen jako *full-stack* aplikace, která bude mít intuitivní GUI a umožní tak širší adopci napříč různými uživateli a týmy.

Schéma architektury bude rozděleno do 4 hlavních logických částí a jejich vzájemná komunikace je znázorněna na obrázku 3.3.

Frontend (Uživatelské rozhraní)

Tato klientská část zajišťuje interakci s uživatelem a slouží jako hlavní bod pro konfiguraci a vizualizaci celého procesu generování testů. S backendem bude komunikovat pomocí dvou odlišných komunikačních vzorů. Pro standardní operace čtení a zápisu dat (např. zakládání projektů, správa uživatelských příběhů) bude využíváno klasické HTTP/REST rozhraní. Pro sledování myšlenek agenta při generování ale synchronní HTTP komunikace není ideální, protože potřebujeme průběžně aktualizovat uživatelské rozhraní o nové informace (co agent dělá, jaké nástroje volá). Proto bude využit jednosměrný protokol Server-Sent Events (SSE), který umožňuje



Obrázek 3.3: Koncepční architektura systému.

efektivní streamování dat z backendu na frontend v reálném čase a je alternativou k WebSocketům, které jsou pro tento účel komplexnější.

Backend

Nejdůležitější část celé architektury - její hlavní zodpovědností je řízení celého procesu generování testů, udržování agentní smyčky a zprostředkování komunikace mezi jazykovými modely, testovacím frameworkem a uživatelským rozhraním. Bude zodpovědný také za bezpečné čtení a zápis dat do lokálního konfiguračního souboru.

AI Gateway

Zajištění spolehlivé komunikace s velkými jazykovými modely představuje zásadní výzvu. Každý poskytovatel (provider) umělé inteligence (OpenAI, Anthropic, Google) využívá pro své API zcela odlišná vstupní a výstupní schémata - liší se formáty pro definici dostupných nástrojů (tool calling), což je pro správnou funkčnost agenta klíčové. Dále se liší i strukturováním historie zpráv či výpočtem spotřebovaných tokenů - další klíčový faktor pro správné fungování agenta. Přímá integrace těchto jednotlivých API by vedla k vysoké komplexitě zdrojového kódu a obtížné udržitelnosti. Pro každého nového poskytovatele by bylo potřeba naimplementovat vlastní logiku pro překlad požadavků a zpracování odpovědí. Systém proto zavede dedikovanou vrstvu AI Gateway, která tyto rozdíly efektivně abstrahuje za pomoci existujících řešení (např. knihovny AI Vercel SDK) a díky níž bude budoucí rozšiřitelnost o nové modely výrazně jednodušší - bude pouze třeba přidat existující adaptér pro daného poskytovatele, aniž by bylo potřeba zasahovat do zbytku architektury či psát ručně překlad pro každý model.

MCP Server a testovací framework

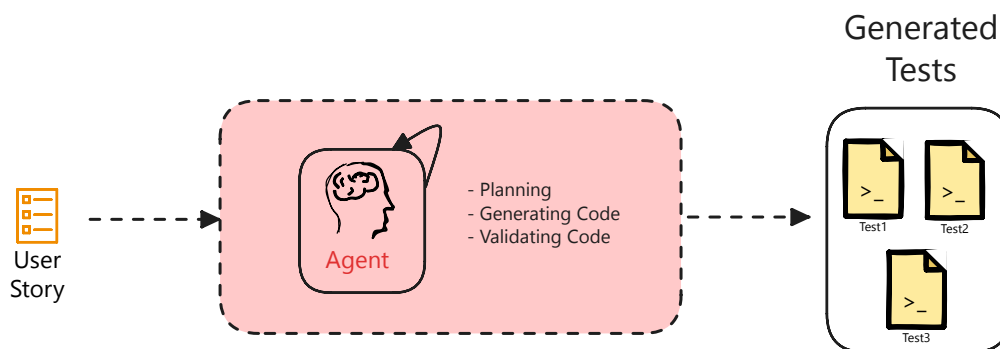
Pro zajištění modularity a bezpečnosti systému by autonomní agent neměl mít přímou kontrolu nad prohlížečem z backendu. Místo toho budou požadavky na interakci s prohlížečem odesílány do dedikovaného serveru, který bude využívat standardizovaný Model Context Protocol a poběží jako zcela samostatný izolovaný proces. Tento server bude přijímat příkazy od agenta (např. kliknutí na tlačítko, vyplnění formuláře) a překládat je do konkrétních instrukcí pro zvolený testovací framework. Framework poté ovládá webový prohlížeč a vrací zpět výsledky akce, které backend předá agentovi pro další zpracování.

3.3.4 Architektura agenta a exekuční smyčka

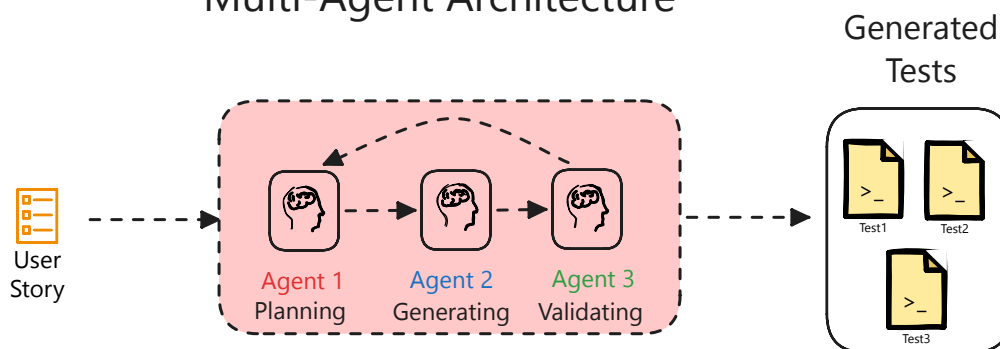
Zdůvodnění Single-agent architektury

Jak popisuje příručka OpenAI [41], existují dva hlavní přístupy k návrhu agentních systémů - **Single-agent** a **Multi-agent** architektura, které jsou zobrazeny na obrázku 3.4.

Single-Agent Architecture



Multi-Agent Architecture



Obrázek 3.4: Schéma Single-agent vs Multi-agent architektury.

Single-agent architektura se zaměřuje na jednoho centrálního agenta, který má přístup ke všem nástrojům a informacím potřebným k vykonání úkolu. Tento agent je zodpovědný za celý proces - od plánování, přes exekuci, až po generování výstupu. Tento přístup je jednodušší na implementaci, snadněji se ladí a chová se determinističtěji, protože veškeré rozhodování je centralizované v jednom agentovi.

Oproti tomu multi-agentní architektura rozděluje úkol mezi více specializovaných agentů, kde každý z nich má svou specifickou roli - jeden může být zodpovědný za plánování, druhý za generování kódu a třetí za revizi kódu. Tento přístup může být efektivní pro řešení komplexnějších úloh, ale přináší s sebou vyšší míru nedeterminismu (jelikož agenti mohou mít různé strategie), je náchylný k zacyklení (nekonečné debaty mezi jednotlivými agenty) a jeho debugging je složitý, jelikož je třeba sledovat flow mezi více entitami. Navíc tento způsob může generovat obrovskou režii v podobě spotřebovaných tokenů právě díky komunikaci mezi agenty, která vyžaduje další volání modelů.

Proto systém bude využívat striktní Single-agent architekturu, kde bude definován agent s rolí *Senior QA Automation Engineer*, který bude provádět celý proces postupně - od prvotní navigace, přes ověření stávajícího stavu aplikace až po finální generování kódu.

ReAct smyčka

Jak bylo popsáno v kapitole dedikované analýze LLM agentů, systém bude využívat paradigma ReAct. V rámci fáze Thought bude agent analyzovat aktuální stav aplikace a plánovat další krok, který by měl podniknout. V Action fázi bude volat konkrétní nástroje prostřednictvím MCP serveru, aby získal potřebné informace o stavu aplikace. V Observation fázi bude backend zpracovávat výsledky těchto akcí a poskytovat jejich zpětnou vazbu agentovi, který na základě toho bude pokračovat v iteraci, dokud nebude mít dostatek informací na vygenerování finálního testu.

Ochranné mechanismy

K snížení rizika halucinací bude agent instruován sadou striktních pravidel, která bude muset dodržovat. Bude se jednat o následující mechanismy:

- **Tool Enforcement** - Agent bude muset použít nástroje pro průzkum aplikace předtím, než bude moci vygenerovat finální test. Běžně se stává, že agenti se snaží rovnou vygenerovat test “naslepo”, aniž by měli dostatek informací o stavu aplikace.
- **Code Validation** - Jakmile agent vygeneruje kód, bude muset projít základní validací, aby se ověřilo, že je syntakticky správný a přeložitelný. Pokud ne, bude agent vyzván k opravě kódu, dokud nebude kód validní.
- **Circuit Breaker** - Aby se předešlo nekonečnému zacyklení (které by vyústilo v nekonečné volání LLM modelů), bude implementován mechanismus, který po určitém počtu po sobě jdoucích selhání nástrojů přeruší agentní smyčku a ukončí proces generování testu.

Tento přístup by měl zlepšovat spolehlivost a kvalitu vygenerovaných testů a zároveň minimalizovat náklady spojené s opakovanými voláními modelů.

3.4 Návrh uživatelského rozhraní

Celkové rozvržení a UX principy

Aplikace bude koncipována po vzoru moderních vývojových prostředí (Visual Studio Code, JetBrains IDEs), které jsou pro vývojáře dobře známé a intuitivní. Na

obrázku 3.5 je zobrazen návrh uživatelského rozhraní, které bude rozděleno do tří hlavních částí - levý navigační panel, hlavní pracovní plocha a pravá vizualizační část.

The interface design is as follows:

- Left Navigation Panel:**
 - Section: Project section
 - Dropdown: Prj 1 | v
 - Buttons: Two empty square buttons.
 - List: Story 1, Story 2, Story 3 (highlighted in blue), Story 4.
 - Settings: A gear icon labeled Settings.
- Central Workspace:**
 - Form fields: Name, Description, Acceptance Criteria, Extra Specification.
 - Model: Claude Opus-4.6
 - Active Tools: tool1, tool2
 - Buttons: Generate (green), Terminate (red).
- Right Visualization Panel:**
 - Activity Log: Test Section
 - Summary: Total Cost: 1\$, Token Usage: 180k
 - Steps: Step 1: Thinking, Step 2: Tool Call, Step 3: Tool Result, Step 4: Code Validation, Step 5: Completed.
 - Generated Code:

```
import {test} from 'test'
test() {
  assert(a,b)
}
```

Obrázek 3.5: Návrh uživatelského rozhraní.

Využit bude boční navigační panel na levé straně, který bude obsahovat sekce pro správu projektů a k nim přiřazených uživatelských příběhů. Zároveň by mohl také obsahovat proklik do nastavení, kde by se mohly upravovat konfigurace agenta (volba LLM modelu, volba nástrojů).

Hlavní pracovní plocha bude sloužit pro specifikaci uživatelských příběhů - bude zde zobrazen formulář, díky němuž půjdou snadno vytvářet a případně editovat uživatelské příběhy. Pod tímto formulářem by se měla nacházet i sekce pro spuštění generování testů a případně i ovládací prvek pro zastavení běhu agenta. Viditelný by měl být i aktuálně využívaný model (aby uživatel nemusel otevírat nastavení) a případně využívané nástroje.

Pravá část plochy bude sloužit pro vizualizaci všeho ohledně testů - od samotného průběhu generování (co agent dělá), po výsledný kód vygenerovaných testů a případně živý náhled do lokálního souborového systému, kde jsou testy uloženy.

Vizualizace procesu agenta

Tato sekce bude klíčová pro zajištění transparentnosti a důvěry uživatelů v systém. Bude zobrazovat celý průběh generování testů - od prvotního zamyšlení agenta, přes volání nástrojů až po finální validaci a generaci kódu. Pokud to daný krok umožní, bude zobrazovat i konkrétní výstup (např. výsledek tool callu). Po vygenerování testu bude (kromě samotného kódu) zobrazeno i krátké shrnutí v podobě počtu zkonsumovaných tokenů, celkové ceny a případně i možnost regenerovat test na základě zpětné vazby od uživatele.

Sekce pro spuštění testů a zobrazení výsledků

Tato sekce by měla nahradit potřebu přepínat mezi běžícím systémem pro generování testů a vývojovým prostředím, čímž se vývojářům ušetří čas. Uživatel by měl mít možnost vidět a spravovat testy, které má v aktuálním repozitáři - a pokud možno, tak je rovnou i spouštět a vidět výsledky jejich běhu.

3.5 Shrnutí

V této kapitole byl představen kompletní návrh softwarového řešení včetně uživatelského rozhraní. Na základě analýzy artefaktů byl vytvořen vlastní normalizovaný doménový model a definována striktní pravidla pro minimalizaci rizika halucinací jazykových modelů v agentské smyčce. Pro maximální uživatelskou přívětivost a adopci byl zvolen *local-first* distribuční model, který spolu s MCP serverem a AI Gateway představuje základ pro udržitelnou a snadno rozšiřitelnou architekturu. Definovány byly i konkrétní funkční a nefunkční požadavky, které budou později sloužit k ověření úspěšnosti implementace a naplnění cílů této práce. Konkrétní technologie, které budou pro implementaci tohoto systému použity, jsou předmětem další kapitoly, která se zaměří na výběr vhodného technologického stacku.

Použité technologie

4

Pro transformaci navržené architektury do podoby reálného softwaru bylo nezbytné vhodně zvolit technologie, které bude systém využívat. Tato kapitola představuje hlavní nástroje, knihovny a frameworky, na kterých bude systém postaven. Cílem výběru nebylo pouze využití moderních technologií, ale primárně nalezení kombinace, která zajistí vysokou míru typové bezpečnosti, vzájemnou kompatibilitu a nativní podporu pro specifické požadavky autonomního agenta.

4.1 TypeScript

Jako primární programovací jazyk celého systému byl zvolen TypeScript. Jedná se o striktně typovanou nadstavbu jazyka JavaScript, která do webového vývoje přináší statickou kontrolu typů [44, 45]. Hlavní výhodou této technologie je schopnost detekovat typové a strukturální chyby již během fáze vývoje a kompilace, na rozdíl od JavaScriptu, který tyto chyby odhalí až při samotném běhu aplikace. Tato schopnost je pro navrhovaný systém klíčová, protože vzhledem k neustálé a dynamické výměně dat mezi frontendem, backendem a velkými jazykovými modely je zajištění konzistentních datových struktur naprosto nezbytné pro správnou funkčnost, spolehlivost a udržitelnost.

To bude realizováno prostřednictvím DTO (Data Transfer Object) patternu, kdy budou definovány konkrétní datové typy a rozhraní pro všechny klíčové entity systému (projekty, uživatelské příběhy, vygenerované testy). Díky tomu bude možné sdílená rozhraní definovat pouze jednou napříč celou aplikací, což výrazně zjednodušuje vývoj, eliminuje redundantní kód a snižuje riziko chyb.

4.2 React

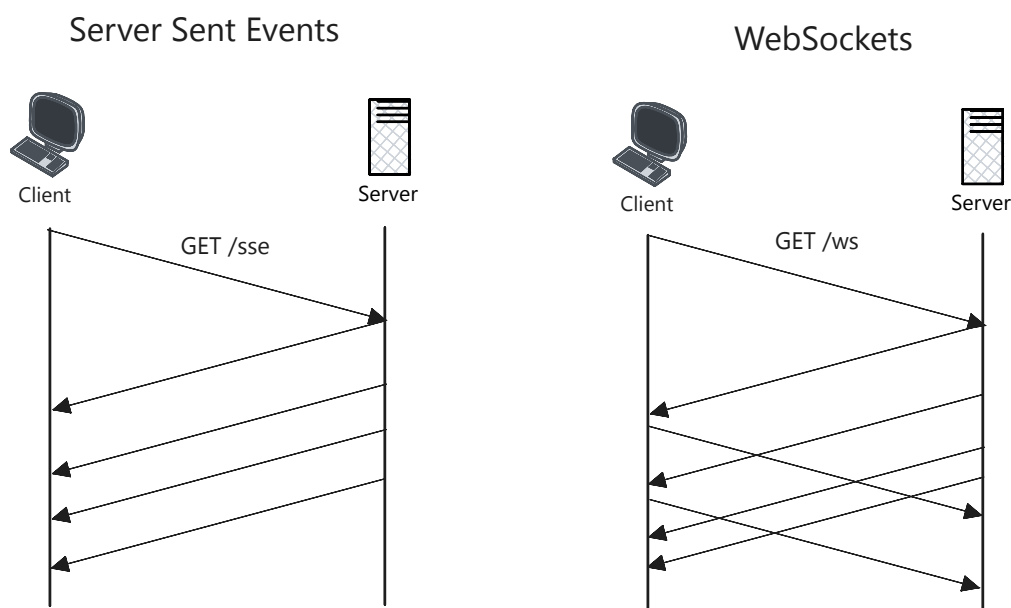
Pro vývoj uživatelského rozhraní byla zvolena populární javascriptová knihovna React. Disponuje deklarativním a komponentovým přístupem, díky němuž je možné rozdělit komplexní rozhraní do menších, znovupoužitelných částí. Vzhledem k důrazu na maximální interaktivitu navrhovaného systému (zejména pro dynamické vy-

kreslování myšlenkových pochodů agenta, správu stavu rozhraní pro specifikaci uživatelských příběhů či zobrazení výsledků vygenerovaných testů) je efektivní správa stavu komponent klíčová. Zásadním benefitem Reactu je využití tzv. virtuálního DOMu (Virtual DOM), který zaručuje, že se při změně stavu překreslí pouze ta část uživatelského rozhraní, jež se skutečně změnila [46]. To výrazně zlepšuje výkon a zajišťuje plynulý chod aplikace i při velkém počtu příchozích událostí. Dalším rozhodujícím faktorem pro volbu této knihovny byla podpora jazyka TypeScript prostřednictvím syntaxe TSX, což umožňuje bezproblémově napojit frontend na sdílené DTO modely a zachovat striktní typovou bezpečnost napříč celou aplikací.

4.3 NestJS

Pro vývoj backendové části aplikace byl zvolen framework NestJS založený na platformě Node.js. Tento framework byl vybrán především pro svou modulární architekturu a integrovanou podporu pro *Dependency Injection* - klíčový návrhový vzor pro udržitelnost a testovatelnost kódu [47]. Vzhledem k tomu, že agent bude obsahovat komplexní logiku, která zahrnuje komunikaci s LLM, správu lokálního souborového systému, správu uživatelských příběhů a projektů, je důležité, aby tyto domény byly striktně logicky oddělené do samostatných modulů, což povede k lepší čitelnosti, testovatelnosti a dlouhodobé udržitelnosti celého backendu. NestJS navíc nabízí nástroj Nest CLI, který umožňuje rychlé generování boilerplate kódu pro nové moduly, služby a kontrolery, což výrazně usnadňuje a zrychluje vývoj. Na rozdíl od klasického Express.js podporuje nativně i asynchronní streamování dat pomocí SSE, díky čemuž je eliminována potřeba implementovat komplexnější řešení na bázi WebSocketů.

Jak je vidět na obrázku 4.1, pro tento případ použití je SSE vhodnou volbou. Zatímco WebSokety vyžadují udržování obousměrného spojení, SSE funguje na principu standardního HTTP požadavku, kdy klient pouze otevře spojení (GET /sse) a následně server už jen jednosměrně odesílá průběžné aktualizace směrem k uživateli.



Obrázek 4.1: Schéma komunikace pomocí SSE vs WebSocketů.

4.4 Vercel AI SDK

Pro komunikaci s velkými jazykovými modely (LLM) byla zvolena knihovna Vercel AI SDK. Jak již bylo zmíněno v předchozí kapitole, největším úskalím integrace různých poskytovatelů umělé inteligence je značná rozdílnost specifikací jejich API. Vercel AI SDK tento problém elegantně řeší tím, že funguje jako univerzální adaptér. Poskytuje jednotné rozhraní, díky kterému lze, jak uvádí Patil et al. ve studii *Real time inventory management system powered by generative user interface* [48], plynule přecházet mezi různými modely pouhou úpravou konfigurace, aniž by bylo nutné zasahovat do vnitřní logiky agenta.

Zásadním důvodem ale byla podpora pro volání nástrojů - knihovna nativně řeší překlad definic nástrojů (pomocí JSON schémat) do formátů specifických pro jednotlivé poskytovatele a automaticky parsuje a validuje odpovědi od modelů zpět do striktně typovaných TypeScript objektů.

V neposlední řadě nabízí i spoustu předpřipravených hooků pro React/Next.js aplikace a podporuje streamování odpovědí.

4.5 Playwright a Model Context Protocol (MCP)

Jako finální testovací framework byl zvolen Playwright. Jak popisuje studie *Automated end to end testing with Playwright for React applications* [49], Playwright efektivně řeší specifické výzvy moderních dynamických aplikací, jako jsou asynchronní operace, složité překreslování komponent a komplexní správa stavu. Přestože byly základní výhody tohoto nástroje představeny v teoretické části, níže jsou na základě zmíněné studie shrnuty ty nejzásadnější, které rozhodly o jeho výběru pro tento systém:

- **Pokročilý Auto-waiting a stabilita:** Framework automaticky vyčkává na interaktivitu prvků, čímž eliminuje potřebu statických čekacích dob a radikálně snižuje nestabilitu testů.
- **Network Interception a API Mocking:** Nativní podpora zachytávání síťového provozu umožňuje simulovat chybové stavy a podvrhovat API odpovědi bez nutnosti úprav na skutečném backendu.
- **Složité uživatelské interakce:** Nástroj spolehlivě zvládá práci se Shadow DOM, testování napříč více záložkami a komplexní akce typu *drag-and-drop*.

Pro integraci s autonomním agentem je navíc klíčová dostupnost NPM balíčku `@playwright/mcp`, který běží jako samostatný proces a komunikuje s backendem přímo přes protokol MCP.

4.6 Shrnutí

V této kapitole byl představen a zdůvodněn výběr technologií, které budou použity pro implementaci navrženého systému. Zvolený technologický stack sahá od striktní typové kontroly (TypeScript), přes reaktivní uživatelské rozhraní (React) a modulární backend (NestJS), až po specializované nástroje pro komunikaci s LLM (Vercel AI SDK) a testování webových aplikací (Playwright s podporou MCP protokolu). Konkrétní použití těchto technologií bude detailně představeno v další kapitole, jež se bude věnovat samotné implementaci.

Implementace

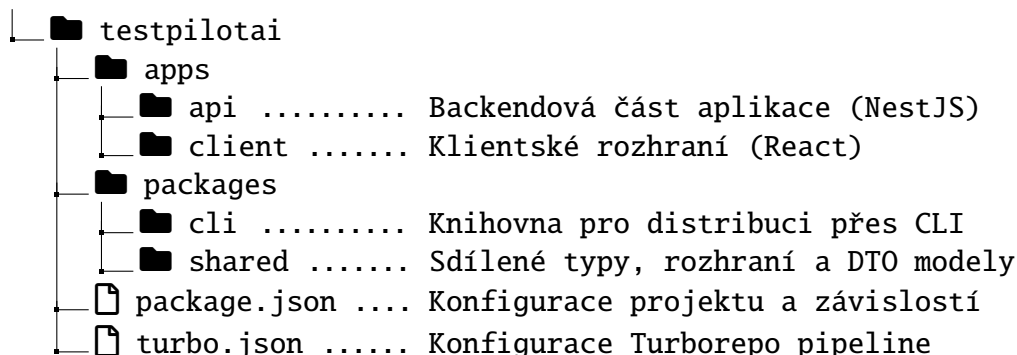
5

Tato kapitola popisuje konkrétní technickou implementaci navrženého systému. Nejprve představuje organizaci zdrojového kódu a rozbor backendové části postavené na frameworku NestJS s důrazem na popis modulární architektury. Dále je popsán způsob bezpečné komunikace s velkými jazykovými modely a následným detailním zaměřením na jádro řešení autonomního agenta - od integrace nástrojů pomocí MCP, řídicí ReAct smyčku, až po prompt engineering a validační mechanismy zajišťující generaci syntakticky správného a spustitelného kódu. Závěrečná část kapitoly je věnována klientské aplikaci, která popisuje nejen uživatelské rozhraní, ale je kladen důraz na technické řešení komunikace pomocí Server-Sent Events a zajištění transparentnosti myšlenkových pochodů agenta v reálném čase.

5.1 Organizace projektu

Celý systém byl implementován jako tzv. monorepo (monolithic repository) s využitím nástroje Turborepo, který umožňuje spravovat více logicky oddělených aplikací, knihoven či služeb v rámci jednoho repozitáře. Díky tomu je možné sdílet kód, typy a rozhraní napříč různými částmi systému bez nutnosti publikovat a spravovat samostatné balíčky pro každou z těchto částí. Nástroj Turborepo navíc optimalizuje proces sestavování (build process) díky inteligentnímu cachování, které zajišťuje, že se znovu sestaví pouze ty části systému, které byly skutečně změněny, což výrazně zrychluje vývoj a testování.

Základní struktura projektu je rozdělena do dvou hlavních adresářů - složka `apps` pro spustitelné aplikace (backend a frontend) a složka `packages` pro sdílené knihovny (např. pro DTO modely, CLI nástroje). Zjednodušené zobrazení této struktury je znázorněno na další straně.



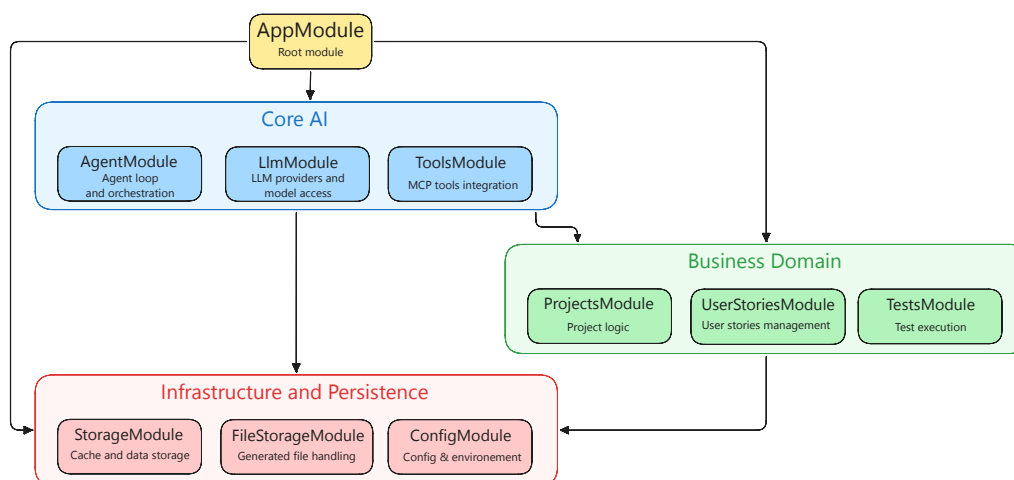
Důležitou částí této struktury je složka `./packages/shared`, která obsahuje veškerá doménová data, komunikační DTO modely a definice typů. Jelikož je tato logika využívána jak backendem (`api`), tak frontendem (`client`), je zaručeno, že obě vrstvy komunikují pomocí identických datových struktur a že jakákoliv změna v datovém modelu se díky TypeScriptu okamžitě projeví chybou při kompilaci, čímž se výrazně snižuje riziko neúmyslných chyb a naplňuje tak požadavek na striktní typovou bezpečnost napříč celou aplikací.

5.2 Implementace backendové části

Zatímco předchozí sekce nastínila celkovou strukturu projektu a jeho uspořádání, tato část se zaměří na konkrétní implementaci backendové části z vyšší perspektivy. Cílem je představit logickou architekturu serveru, klíčové moduly a jejich zodpovědnosti - konkrétní detaily implementace budou představeny v rámci dalších sekcí.

5.2.1 Doménové moduly

Backendová část (`apps/api`) je postavena na frameworku NestJS a dodržuje jeho modulární architekturu. Aplikace je rozdělena do nezávislých doménových modulů, které jsou pak jako jednotlivé funkční celky sdružené v hlavním vstupním bodě (`AppModule`). Architektura je logicky vrstvena do tří hlavních domén, které jsou zobrazeny na obrázku 5.1.



Obrázek 5.1: Zjednodušený diagram vrstvených závislostí backendových modulů.

Díky této separaci je zajištěno, že každý modul má jasně definovanou zodpovědnost, což výrazně zlepšuje nejen čitelnost a udržitelnost kódu, ale i jeho testovatelnost. Každý modul může být jednoduše testován izolovaně pomocí unit testů a následně i integračních testů, které ověří správnou spolupráci mezi moduly.

Tok dat (Request/Response flow) uvnitř backendu je striktně vrstvený a dodržuje best practices zapouzdření. Požadavek z klientské aplikace je nejdříve zachycen v příslušném kontroleru dané domény (např. `AgentController`), který provádí počáteční validaci vstupních parametrů pomocí DTO a Zod schémat. Kontroler nikdy nekomunikuje s konkrétními službami z jiných modulů - od toho je tu dedikovaná servisní vrstva (např. `AgentOrchestrationService`), která zapouzdřuje veškerou business logiku. Až tato vrstva deleguje dílčí úkoly dál a bezpečně komunikuje se specializovanými službami (Services) z jiných modulů. Tato architektura je logicky rozdělena do tří hlavních vrstev, které budou detailně popsány v následujících částech:

- **Core AI Orchestration:** Jádro systému spravující komunikaci s jazykovými modely, externími nástroji a celkový běh agenta.
- **Business Domain Modules:** Moduly zapouzdřující aplikační logiku, správu projektů, uživatelských příběhů a samotných testů.
- **Infrastructure & Persistence:** Vrstva zajišťující konfiguraci prostředí, logování a bezpečnou práci se souborovým systémem.

Core AI Orchestration

Tato vrstva tvoří jádro celého systému a obsahuje moduly zodpovědné za interakci s velkými jazykovými modely a jejich orchestraci. `AgentModule` implementuje řídicí smyčku agenta a orchestrace běží přes služby `AgentService` a `AgentOrchestrationService`. Zároveň tento modul přímo integruje `LlmModule`, `ToolsModule`, `ProjectsModule`, `UserStoriesModule`, `StorageModule` a `FileStorageModule`, což z něj dělá hlavní integrační bod backendu. `LlmModule` poskytuje jednotné rozhraní pro volání různých poskytovatelů modelů přes AI Gateway, `ToolsModule` pak dále zajišťuje načtení dostupných nástrojů a jejich exekuci přes MCP vrstvu - aktuálně s podporou Playwright nástrojů, ale s možností snadného budoucího rozšíření (NFR-05).

Business Domain Modules

Tyto moduly reprezentují doménovou logiku aplikace. `ProjectsModule` zajišťuje správu projektů a jejich konfigurací, `UserStoriesModule` spravuje uživatelské příběhy a jejich artefakty (název, popis, akceptační kritéria), a `TestsModule` zajišťuje načítání, spouštění a mazání již vygenerovaných Playwright testů. Samotné generování testů je řízeno přes endpointy modulu `AgentModule`; `TestsModule` slouží primárně pro exekuci a správu výsledných testovacích souborů.

Infrastructure & Persistence

Nejnižší vrstva zajišťuje veškeré nefunkční aspekty - od konfigurace přes management dat v cache až po bezpečný zápis do souborového systému. `ConfigModule` zajišťuje načtení proměnných prostředí a práci s lokálním konfiguračním souborem, `StorageModule` ukládá metadata o generování do souboru. `FileStorageModule` se stará o zápis vygenerovaných testů do cílového adresáře, vytváření složek a řešení kolizí názvů souborů.

5.2.2 Implementace doménového modelu a sdílených DTO

Navrhovaný doménový model z kapitoly 3 byl v rámci implementace převeden do sdílených TypeScript interfaců, které jsou umístěny v balíčku `packages/shared` a přímo využívány jak frontendem, tak backendem. Struktura balíčku `shared` je rozdělena do dvou hlavních složek:

- `/types` - Obsahuje doménové entity jako `Project`, `UserStory` a `TestPilotConfig`. Jedná se o konkrétní doménové objekty, které reprezentují perzistovaná data v systému.

- /dto - Obsahuje Data Transfer Objects pro jednotlivé operace (Create, Update, Response). Zajišťují konzistentní datové struktury pro API vrstvu a komunikaci mezi klientem a serverem.

Definice dvou klíčových doménových entit - `Project` a `UserStory` - jsou zobrazeny ve zdrojovém kódu 5.1. Jak je vidět, tyto entity byly oproti původnímu návrhu rozšířeny o metainformace jako `createdAt` a `updatedAt`, které umožňují sledovat historii změn a případně implementovat funkce pro auditování nebo zobrazení časové osy projektu.

```

1 export interface Project {
2   id: string;
3   name: string;
4   description: string;
5   baseUrl: string;
6   createdAt: string;
7   updatedAt: string;
8 }
9
10 export interface UserStory {
11   id: string;
12   projectId: string;
13   title: string;
14   description: string;
15   acceptanceCriteria: string;
16   extraInfo: string;
17   createdAt: string;
18   updatedAt: string;
19 }

```

Zdrojový kód 5.1: Ukázka sdílených doménových entit v balíčku `@testpilotai/shared`.

Kromě entit jsou v balíčku `shared` definovány i DTO objekty pro jednotlivé API operace. Ty umožňují seskupovat pouze relevantní pole pro danou operaci - např. při vytváření projektu klient neposílá identifikátor a časová razítka (ta generuje server), ale pouze `name`, `description` a `baseUrl`. Při aktualizaci jsou všechna pole nepovinná, aby klient mohl modifikovat pouze ty atributy, které se změnily. Příklady DTO objektů pro operace nad user stories jsou uvedeny v kódu 5.2.

5.2.3 Perzistence a konzistence dat

S využitím *local-first* přístupu je ukládání dat řešeno pomocí souborů namísto klasické relační databáze. Systém využívá dva hlavní soubory pro perzistenci - `test-pilot.json`, který obsahuje základní konfiguraci aplikace a doménová data, a dále

```
1 export interface CreateUserStoryDto {  
2   projectId: string;  
3   title: string;  
4   description: string;  
5   acceptanceCriteria: string;  
6   extraInfo: string;  
7 }  
8  
9 export interface UpdateUserStoryDto {  
10  title?: string;  
11  description?: string;  
12  acceptanceCriteria?: string;  
13  extraInfo?: string;  
14 }
```

Zdrojový kód 5.2: Ukázka DTO objektů pro vytváření a aktualizaci user stories.

`.testpilot-cache.json`, který slouží pro ukládání metadat z jednotlivých běhů generování testů.

Na ukázkách 5.3 a 5.4 je zobrazen obsah těchto souborů. Jak je vidět, soubor `test-pilot.json` obsahuje klíč `config`, který umožňuje specifikovat globální nastavení aplikace (momentálně pouze cílový adresář pro ukládání vygenerovaných testů). Dále obsahuje pole objektů `projects` a `userStories`, jejichž struktura odpovídá definovaným TypeScript entitám.

Cachovací soubor `.testpilot-cache.json` (kromě metadat o poslední aktualizaci souboru a verzi systému) obsahuje pole `generationRecords`, které uchovává záznamy o jednotlivých bžích generování testů. Obsahují základní informace o tom, jak vygenerování testu probíhalo - jaký byl využit model, kolik tokenů bylo zkonsumováno, jaká byla celková cena a (v případě úspěchu) i samotný vygenerovaný kód testu. Tyto informace jsou klíčové pro zpětnou vazbu uživatelům a budou dále zmíněny a využity v další kapitole, která se zaměří na testování funkčnosti systému a celkové náklady generování testů.

```

1 {
2   "config": {
3     "output": { "directory": "./tests" }
4   },
5   "projects": [
6     {
7       "id": "project-1770809040081",
8       "name": "Example project",
9       "baseUrl": "https://app.test.com",
10      "createdAt": "2026-01-01T00:00:00Z",
11      "updatedAt": "2026-01-01T01:22:33Z"
12    }
13  ],
14  "userStories": [
15    {
16      "id": "1772119522231-grcz5m2",
17      "projectId": "project-1770809",
18      "title": "Login",
19      "acceptanceCriteria": "AC1; AC2",
20      "createdAt": "2026-01-02T00:00:00Z"
21    }
22  ]
23 }

```

Zdrojový kód 5.3: Struktura
test-pilot.json.

```

1 {
2   "version": "1.0.0",
3   "generationRecords": [
4     {
5       "userId": "1772119522231-g",
6       "model": "gpt-5",
7       "status": "success",
8       "tokenUsage": {
9         "input": 112964,
10        "output": 7192,
11        "cached": 88448
12      },
13      "cost": {
14        "input": 0.030645,
15        "output": 0.07192,
16        "cached": 0.05527
17      },
18      "id": "gen-1772634117252-6",
19      "timestamp": "2026-03-04T14:21:57Z"
20    }
21  ],
22   "lastUpdated": "2026-03-27T13:09:06Z"
23 }

```

Zdrojový kód 5.4: Struktura
.testpilot-cache.json.

Klíčovým problémem při práci s těmito soubory je synchronizace kritických sekcí typu *read-modify-write*. Při paralelních požadavcích by mohlo dojít k nechtěnému přepsání, ztrátě či poškození dat (např. v případě kolizí při souběžném zápisu do souborů). Proto byl pro eliminaci tohoto rizika implementován asynchronní zámek (promise-based mutex), který zajistí, že všechny operace, které modifikují obsah těchto souborů, budou striktně serializované. Před každým zápisem je navíc načtený obsah souborů validován proti definovaným Zod schémátům, což zajišťuje další úroveň ochrany proti nekonzistentním datům.

5.3 Implementace komunikace s LLM

Modul `LlmModule` představuje komunikační rozhraní mezi vnitřní logikou systému (autonomním agentem) a providery velkých jazykových modelů. Hlavním úkolem této vrstvy bylo zajistit takovou abstrakci, která by umožnila snadnou integraci nových poskytovatelů a modelů, čímž se minimalizovalo riziko závislosti na konkrétním provideru (*vendor lock-in*) (**NFR-04**). Díky tomu je přidání podpory pro nové modely pouze záležitostí několika řádků kódu (v případě nového providera vytvoření nového souboru s jeho modely).

5.3.1 Abstrakce poskytovatelů a Vercel AI SDK

Jak již bylo zmíněno, problém abstrakce různých poskytovatelů řeší knihovna Vercel AI SDK. Systém na úrovni `LlmService` pracuje s jednotným formátem zpráv

```
1 import { generateText } from 'ai';
2
3 const result = await generateText({
4   model: "anthropic/claude-sonnet-4.6",
5   system,
6   messages,
7   tools,
8   temperature,
9 });
10
11 return {
12   text: result.text,
13   toolCalls: result.toolCalls,
14   usage: result.usage,
15 };
```

Zdrojový kód 5.5: Ukázkové volání LLM přes Vercel AI SDK.

a nástrojů. Na ukázce 5.5 je zobrazen způsob volání modelu přes funkci `generateText`. Je vidět, že se předávají pouze obecné parametry jako je identifikátor modelu, systémový prompt, předchozí zprávy a dostupné nástroje - žádné konkrétně nadefinované struktury pro jednotlivé providery.

Pak již stačí jen do systémových proměnných nastavit odpovídající klíče jednotlivých providerů a je možné bez jakýchkoliv dalších úprav přepínat mezi modely pouhou změnou identifikátoru modelu v proměnné `model`. Nevýhodou tohoto přístupu v komerční praxi je, že pokud by chtěl uživatel volat všechny firemní modely, musel by mít přístup ke všem potřebným klíčům a nastavit je v prostředí, což není ideální z pohledu bezpečnosti a správy přístupů. S tím souvisí i pricing - pokud by uživatel začal neúmyslně volat dražší modely, mohl by se mu rychle navýšit účet za využití API.

Tento problém v implementovaném řešení eliminuje vlastní integrační vrstva. Ta za pomoci funkce `createOpenAICompatible`, která je součástí Vercel AI SDK, umožňuje definovat vlastní mapování modelů a jejich identifikátorů. V praxi to znamená, že systém v rámci konfigurace využívá jeden univerzální API klíč pro podnikovou AI Gateway a pomocí něj má přístup ke všem podporovaným modelům. Tím se zásadně zvyšuje bezpečnost - klíče různých poskytovatelů nejsou distribuované napříč klientskými zařízeními, ale jsou bezpečně spravovány na straně gateway (**NFR-03**). Koncový uživatel (nebo konkrétní instance aplikace) pak využívá pouze jeden vygenerovaný klíč, což umožňuje detailní monitoring a případné omezení využití jednotlivých modelů (rate limiting). To je naprosto klíčové pro kontrolu nákladů.

V rámci této práce byl proto využit právě tento přístup, kdy uživatel dostane jeden univerzální API klíč pro AI Gateway firmy Emplifi, přes níž se následně volají

```

1 private gateway = createOpenAICompatible({
2   name: 'ai-gateway',
3   apiKey: this.configService.aiGatewayKey,
4   baseUrl: this.configService.aiGatewayBaseUrl,
5 });
6
7 async generate(request: Request, signal?: AbortSignal): Promise<
  Response> {
8   const model = this.models.get(request.model)!;
9   const modelId = this.resolveGatewayModelId(model);
10  const messages = this.convertMessagesToAISdk(request.messages);
11  const tools = this.convertToolsToAISdk(request.tools);
12  const temperature = request.temperature ?? 0;
13
14  const result = await generateText({
15    model: this.gateway(modelId),
16    system: request.system,
17    messages,
18    tools,
19    ...(model.supportsTemperature !== false && { temperature }),
20    abortSignal: signal,
21  });
22
23  return this.convertResponse(result, model);
24 }

```

Zdrojový kód 5.6: Zjednodušené napojení Vercel AI SDK na vlastní AI Gateway

všechny podporované modely. To výrazně zjednodušuje použití tohoto systému zaměstnanci a zároveň umožní na konci vývoje a testování určit, kolik nákladů bylo na tento systém vynaloženo, což bude důležitá metrika pro zhodnocení efektivity navrženého řešení.

Na ukázce 5.6 je zobrazen způsob, jakým je Vercel AI SDK napojeno na vlastní AI Gateway. Nejprve se pomocí funkce `createOpenAICompatible` vytvoří instance `gateway`, která je nakonfigurovaná s univerzálním API klíčem a základní URL pro volání modelů. Následně se v metodě `generate` připraví všechny potřebné parametry pro volání modelu - identifikátor modelu se přeloží na odpovídající ID pro AI Gateway (jelikož např. Anthropic modely jsou poskytovány přes AWS Bedrock, a proto mají jiné ID než stejné modely volané přímo přes Anthropic API), zprávy a tooly se převedou do očekávaného formátu pro Vercel AI SDK a nakonec se zavolá funkce `generateText` s připravenými parametry. Výsledek se pak převede do interního formátu. Jak je vidět, funkce `generateText` umožňuje nastavovat i další parametry jako je `temperature` pro modely, které tento parametr podporují, či `abortSignal` pro možnost přerušení běhu generování, což je klíčové pro zajištění dobré uživatelské přívětivosti a kontroly nad procesem generování testů.

```
1 export type LLMModelDefinition = {  
2   id: string;  
3   shortId: string;  
4   displayName: string;  
5   description?: string;  
6   supportsTemperature?: boolean;  
7   pricing: {  
8     input: number;  
9     output: number;  
10    cached?: number;  
11  };  
12 };
```

Zdrojový kód 5.7: Datová struktura pro konfiguraci LLM modelů.

5.3.2 Správa dostupných modelů

Po vyřešení komunikace s jednotlivými poskytovateli bylo nutné navrhnout způsob, jakým budou do systému přidávány a evidovány jednotlivé modely. Muselo se jednat o rychlý, jednoduchý a flexibilní mechanismus, který umožní přidávat definice nových modelů včetně jejich cenových údajů a metadat.

Struktura definice modelu

Základem pro správu modelů je sdílený typ `LLMModelDefinition` v balíčku `@test-pilotai/shared`. Jak je vidět na ukázce 5.7, tento typ obsahuje klíčové atributy - od základních informací jako je `id`, `displayName` a `description`, až po ekonomické parametry v podobě `pricing` pro jednotlivé typy tokenů, kde se jedná o cenu za milion tokenů v USD.

Vybrané modely a ceník

V tabulce 5.1 jsou uvedeny modely, které jsou v systému registrovány a využívány.

Provider	Model	In (\$/1M)	Out (\$/1M)	Cached (\$/1M)
Anthropic	Claude Haiku 3	0.25	1.25	0.03
Anthropic	Claude Haiku 3.5	0.80	4.00	0.08
Anthropic	Claude Sonnet 4.5	3.00	15.00	0.30
OpenAI	GPT-4o	2.50	10.00	1.25
OpenAI	GPT-4.1	2.00	8.00	0.50
OpenAI	GPT-5 Mini	0.25	2.00	0.125
OpenAI	GPT-5	1.25	10.00	0.625
Google	Gemini 2.5 Flash	0.30	2.50	0.075
Google	Gemini 2.5 Pro	2.50	15.00	0.625

Tabulka 5.1: Modely registrované v LlmService a jejich ceny.

Modely byly zvoleny tak, aby bylo v rámci testování možné porovnat různé cenové a kvalitativní úrovně.

- **Levnější modely** (např. Claude Haiku 3, GPT-5 Mini, Gemini 2.5 Flash) jsou vhodné pro rychlé iterace, ladění promptů a objemově větší testování.
- **Výkonnější modely** (např. GPT-5, GPT-4.1, Gemini 2.5 Pro, Claude Sonnet 4.5) se využívají tam, kde je potřeba kvalitnější reasoning a přesnější výstup.
- **Více providerů** (OpenAI, Anthropic, Google) dává možnost průběžně porovnávat poměr cena/výkon napříč různými poskytovateli.

Díky této kombinaci má uživatel možnost (dle complexity scénáře, požadované kvality apod.) zvolit model, který nejlépe odpovídá jeho potřebám. Nejdražší a nejnovější modely (např. Claude Opus 4.6) nebyly do experimentu zahrnuty, jelikož jejich provozní náklady by výrazně omezily rozsah testování. Cílem práce je ověřit navržené řešení v reálných podmínkách, což zahrnuje i důraz na ekonomickou efektivitu, nikoliv pouze dosažení nejlepší možné kvality bez ohledu na náklady.

Výpočet ceny generování

Po každém volání modelu se z `result.usage` načtou počty tokenů `inputTokens`, `outputTokens` a `cachedInputTokens`. Poté se cena vypočte přímo v metodě `calculateCost` (jak je uvedeno v ukázce 5.8) a následně přičte k záznamu o generování, který se po skončení běhu uloží do cache souboru `.testpilot-cache.json`.

```

1 private calculateCost(
2   inputTokens: number,
3   outputTokens: number,
4   cachedTokens: number,
5   pricing: LLMModelDefinition['pricing']
6 ) {
7   const nonCachedInputTokens = Math.max(0, inputTokens - cachedTokens);
8   const cachedRate = pricing.cached ?? 0;
9
10  return {
11    input: (nonCachedInputTokens / 1e6) * pricing.input,
12    output: (outputTokens / 1e6) * pricing.output,
13    cached: (cachedTokens / 1e6) * cachedRate,
14  };
15 }
16

```

Zdrojový kód 5.8: Výpočet ceny LLM dotazu.

5.4 Implementace autonomního agenta

5.4.1 Integrace nástrojů přes Model Context Protocol

Pro integraci nástrojů backend nevolá Playwright API přímo, ale využívá dedikovanou vrstvu PlaywrightMCPTools, jež inicializuje MCP klienta (MCPCClient). Jak je vidět na ukázce 5.9, klient je nakonfigurován pomocí příkazu `npx` a balíčku `@playwright/mcp`. Serverová část MCP tak běží jako samostatný proces komunikující přes I/O, což zajišťuje procesní izolaci a zotavitelnost - selhání prohlížečového runtime nebo pád MCP serveru se neprojeví jako pád hlavního NestJS procesu, ale jako chyba volání konkrétního nástroje (**NFR-01**).

Pro komunikaci mezi těmito dvěma procesy se využívá `StdioClientTransport`. Ten umožňuje přenášet zprávy prostřednictvím standardního vstupu a výstupu (stdio), což představuje efektivní a jednoduchý způsob pro lokálně spouštěné MCP servery.

Poslední část ukázky demonstruje způsob, jakým se aplikace chrání před potenciálním uvíznutím. Jelikož se spouští zcela nový a nezávislý proces, existuje riziko, že spojení selže nebo server přestane při inicializaci reagovat (např. kvůli problémům s prostředím, nedostatečným oprávněním, nebo jiným neočekávaným chybám). Z tohoto důvodu je proces navázání spojení (`client.connect`) obalen asynchronním `timeoutem` (`withTimeout`), který zajistí, že pokud se spojení nepovede navázat do určitého časového limitu, bude operace ukončena chybou a systém zůstane v konzistentním stavu (`connected = false`).

```

1  const config = getToolsConfig();
2
3  // Playwright wrapper: create MCP client with values from config
4  this.mcpClient = new MCPClient(
5    'npx',
6    ['-y', config.playwright.mcpPackage], // '@playwright/mcp@0.0.68'
7    { connectTimeoutMs: config.playwright.connectTimeoutMs }, // 30000
8  );
9
10 // MCP client constructor: stdio transport over child process
11 this.transport = new StdioClientTransport({
12   command: serverCommand,
13   args: serverArgs,
14 });
15
16 // MCP client connect(): explicit timeout guard
17 await this.withTimeout(
18   this.client.connect(this.transport),
19   this.connectTimeoutMs,
20   'MCP connection timed out after ${this.connectTimeoutMs}ms',
21 );
22
23 // If connection is successful, set connected flag
24 this.connected = true;

```

Zdrojový kód 5.9: Zkrácený proces inicializace MCP.

5.4.2 Řídící smyčka agenta a ochranné mechanismy

Veškerá agentova logika je implementována v rámci služby `AgentService`, která je zodpovědná za řízení hlavní smyčky exekuce. Následující sekce se detailně zaměřuje na dva klíčové aspekty tohoto procesu:

- **Správa vnitřního stavu exekuce (State Management):** Způsob, jakým systém udržuje kontext, historii konverzace a statistiky napříč jednotlivými iteracemi agenta.
- **Ochranné mechanismy a řízení běhu:** Sada pravidel a pojistek (např. *Circuit Breaker*), která chrání smyčku před zacyklením, halucinacemi a neočekávanými selháními.

Správa vnitřního stavu exekuce (State Management)

Jak bylo zmíněno v předchozích kapitolách, agent pracuje v rámci iterativních smyček, a proto je nutné udržovat celý kontext napříč jednotlivými iteracemi. Pro tento účel byl navržen `AgentExecutionState`, který je zobrazen na ukázce 5.10.

```

1 interface AgentExecutionState {
2     iteration: number;
3     conversationMessages: Message[];
4     allToolCalls: ToolCallResult[];
5     successfulToolCalls: ToolCallResult[];
6     failedToolCalls: ToolCallResult[];
7     consecutiveToolFailures: number;
8     iterationLogs: AgentIterationLog[];
9     totalTokenUsage: TokenUsage;
10    totalCost: Cost;
11 }

```

Zdrojový kód 5.10: Struktura vnitřního stavu exekuce agenta.

Jak je vidět, stav obsahuje všechny klíčové informace potřebné pro řízení exekuce. Počínaje číslem aktuální iterace (`iteration`), přes celou historii konverzace s LLM (`conversationMessages`), až po detailní záznamy o všech voláních nástrojů (`allToolCalls`, `successfulToolCalls`, `failedToolCalls`, `consecutiveToolFailures`). Pro případnou zpětnou analýzu či ladění jsou uchovávány i kompletní záznamy o průběhu jednotlivých iterací (`iterationLogs`), které obsahují informace o vstupních zprávách, nástrojích, odpovědích modelu, využití tokenů a dalších relevantních datech. V neposlední řadě jsou v rámci stavu udržovány i nasčítané statistiky o celkovém množství využitých tokenů (`totalTokenUsage`) a celkových nákladech (`totalCost`).

Ochranné mechanismy a řízení běhu

Přestože je chování modelu primárně řízeno systémovým promptem (viz další subsekcce), systém nemůže spoléhat pouze na to, že model bude vždy správně interpretovat a dodržovat instrukce (halucinování). Proto `AgentService` implementuje ochranné mechanismy představené v kapitole 3, které agenta omezují zvenčí.

Tyto mechanismy jsou zobrazeny v ukázce 5.11, která představuje zjednodušený pseudokód hlavní řídicí smyčky agenta.

Smyčka je omezena na maximální počet iterací (`config.maxIterations`), aby se zabránilo nekonečnému běhu v případě halucinací, nečekaných chyb či jiných problémů. Pro případ, že by uživatel chtěl generování kdykoliv přerušit, obsahuje smyčka i kontrolu pro zrušení pomocí `AbortSignal`. Další důležitou ochranou je **Circuit Breaker**, který sleduje počet po sobě jdoucích selhání nástrojů. Pokud tento počet překročí nastavený limit (`config.maxConsecutiveFailures`), smyčka se okamžitě přeruší a vrátí chybu, aby se opět zabránilo dalšímu volání LLM a s tím i spojeným nákladům (**NFR-02**). Následně se navýší číslo iterace, připraví se požadavek pro LLM a vykoná pomocí metody `llmService.generate`. Po obdržení odpovědi se z ní extrahují požadavky na nástroje.

Pokud LLM nepožaduje použití žádných nástrojů, spustí se metoda `handleNo-`

```

1 while (state.iteration < config.maxIterations) {
2   // Cancellation Check
3   if (signal?.aborted)
4     return this.handleCancellation(...);
5
6   // Circuit Breaker
7   if (state.consecutiveToolFailures >= config.maxConsecutiveToolFailures)
8     await this.generationRecord.createFailure(...);
9
10  state.iteration++;
11  const response = await llmService.generate(...);
12  const toolUses = extractToolUses(response);
13
14  if (toolUses.length === 0) {
15    const result = await handleNoToolCalls(...);
16
17    if (result.type === 'continue') continue;
18    if (result.type === 'error') throw new Error(result.error);
19    if (result.type === 'success') return result.response;
20  }
21
22  await this.processToolCalls(...);
23 }
24
25 this.handleMaxIterationsReached(...);

```

Zdrojový kód 5.11: Zjednodušená řídicí smyčka agenta.

ToolCalls, která analyzuje odpověď a rozhodne, zda je potřeba pokračovat v dalších iteracích - např. zmiňovaný **Tool Enforcement**, který vynutí použití dalších nástrojů - nebo zda je odpověď potřeba opravit (Code Validation - bude vysvětlen v nadcházející subsekci). Pokud se jedná již o finální zvalidovaný kód (result.type === 'success'), smyčka se ukončí a vygenerované testy se vrátí uživateli. Pokud ale model nějaké volání nástrojů požaduje, spustí se metoda processToolCalls, která se postará o jejich vykonání, zpracování výsledků a aktualizaci stavu agenta.

Tento přístup se opakuje do doby, než agent dosáhne úspěšného ukončení (získá validní testy bez potřeby volání dalších nástrojů) nebo překročí nastavený limit iterací, případně dojde k přerušení uživatelem či aktivaci circuit breakeru. Pokud dojde ke zmiňovaným přerušením, logika opustí smyčku a (v případě aktivace circuit breakeru) se metoda handleMaxIterationsReached postará o vytvoření záznamu o neúspěšném pokusu o generování testů a informování uživatele o důvodu neúspěchu.

5.4.3 Návrh systémového promptu a řízení chování (Prompt Engineering)

Tato část se zaměřuje na nejdůležitější aspekt celého systému - návrh systémového promptu, který je základem pro řízení chování LLM a tím i pro úspěšné generování testů. Z důvodu rozsáhlosti jsou plná znění systémového i regeneračního promptu

k dispozici v příloze A. Prompty jsou zapsány v Markdown formátu, který umožňuje lepší strukturování a čitelnost instrukcí pro model.

Struktura a fáze systémového promptu

Systémový prompt je dynamicky sestavován pomocí funkce `getSystemPrompt` (**FR-01**), která do něj vkládá kontext konkrétního projektu (název, výchozí URL a popis) a uživatelského příběhu (název, popis, akceptační kritéria a případně nepovinnou dodatečnou specifikaci). Model je poté instruován k dodržování přísných pravidel a postupů, které jsou rozděleny do několika klíčových sekcí, jak je shrnuto v tabulce 5.2.

Sekce promptu	Účel a klíčové instrukce
Mandatory Workflow	Definuje jasnou sekvenci aktivit agenta, která odděluje průzkum aplikace od generování kódu. Model je nucen nejprve získat reálný stav UI pomocí nástrojů (FR-02) a až poté na základě ověřených informací vytvářet testy (FR-03) .
Tool Failure Handling	Specifikuje chování při selhání nástrojů - model musí explicitně reflektovat chybu, nesmí pracovat s neověřenými daty a je nucen hledat alternativní způsob získání informací.
Selector Strategy	Standardizuje výběr selektorů tak, aby výsledné testy byly stabilní a udržitelné, přičemž preferuje sémantické a explicitně definované identifikátory před křehkými strukturálními selektory.
Code Requirements	Omezuje podobu generovaného kódu na konzistentní a spustitelný výstup, včetně povinných asercí a struktury testů, čímž minimalizuje riziko nevalidního nebo neúplného testovacího scénáře.
Example Output	Poskytuje referenční šablonu výstupu, která modelu ukazuje očekávaný styl, strukturu i granularitu testů a slouží jako vodítko pro formátování odpovědi.
Final Output	Vynucuje striktní oddělení generovaného kódu od jakéhokoliv doprovodného textu, aby byl výstup přímo použitelný bez nutnosti další úpravy nebo parsování.

Tabulka 5.2: Architektura a klíčové sekce systémového promptu.

Důležité bylo zajistit, aby si agent uměl poradit nejen s ideálními scénáři, ale aby byl schopen i detekovat funkcionální mezery - např. když uživatelský příběh

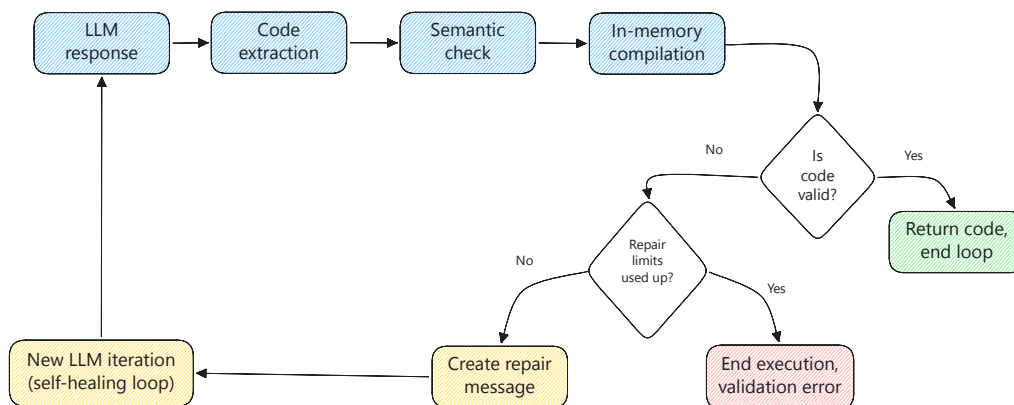
požaduje určitou funkcionalitu, která v reálné aplikaci chybí nebo je rozbitá. Prompt tento problém řeší zavedením konceptu **řízeného selhání** - pokud agent zjistí, že požadovaná funkcionalita chybí nebo je rozbitá, je instruován tak, aby se pokusil napsat test pokrývající správné, očekávané chování. Tento test se ale následně musí označit pomocí speciálního příznaku `test.fail()` (**FR-04**), aby bylo jasné, že se jedná o známou chybu implementace, a navíc se na začátku souboru s vygenerovaným testem vytvoří v komentáři blok `IMPLEMENTATION NOTES`, který popisuje rozdíl mezi zadáním a realitou (**FR-06**).

Prompt pro regeneraci a iterativní zpětnou vazbu

Druhým klíčovým aspektem pro řízení chování agenta je schopnost regenerace testu na základě zpětné vazby od uživatele - a přesně to je realizováno pomocí regeneračního promptu (`getRegenerationPrompt`) (**FR-09**). Hlavní rozdíl mezi systémovým a regeneračním promptem je v tom, že regenerační prompt obsahuje navíc i **předchozí vygenerovaný kód** a **textovou zpětnou vazbu od uživatele**, která popisuje, co je potřeba opravit nebo změnit. Prompt má poté explicitní instrukci *"Keep what works; change only what is required to address the feedback"*, což by mělo vést k minimalizaci rizika, že agent při regeneraci přepíše nebo rozbije již existující funkční testy, přičemž má ale stále povinnost ověřovat všechny změny přímo na běžící aplikaci pomocí nástrojů.

5.4.4 Validační pipeline a samoopravná smyčka

Jakmile jazykový model úspěšně dokončí fázi průzkumu a vygeneruje testy, je potřeba zajistit, že výstupem je opravdu spustitelný kód (**FR-05**). LLM vrací odpovědi ve formátu Markdown, což znamená, že kód je potřeba nejprve extrahovat a až poté validovat, než se vrátí uživateli. K tomu slouží validační pipeline představená v kapitole 3, která je implementována v rámci `CodeValidatorService`. Zpracování výstupu je zobrazeno na diagramu 5.2 a skládá se ze tří hlavních fází - extrakce kódu, sémantické kontroly a in-memory TypeScript kompilace, která může být následována samoopravnou smyčkou v případě zjištění chyb (tzv. *self-correction*).



Obrázek 5.2: Schéma validační pipeline a samoopravné smyčky.

Extrakce kódu

Prvním krokem je správné extrahování zdrojového kódu z odpovědi modelu. K detekci kódových bloků se využívají regulární výrazy, které hledají sekce ohraničené třemi zpětnými apostrofy (‘ ‘ ‘) s následným označením jazyka (např. `typescript` nebo `ts`). Pokud explicitní označení chybí, využívají se záložní metody pro nalezení generického kódového bloku (nejen s TypeScript tagem, ale i bez něj). Pokud kódový blok není nalezen vůbec, pipeline se ukončí s chybou a vrátí se zpětná vazba pro LLM, aby kód správně ohraničil.

Sémantická kontrola

Dalším krokem je sémantická kontrola ověřující základní předpoklady pro Playwright testy. Konkrétně se jedná o:

- Kontrola přítomnosti importu z knihovny `@playwright/test` v případě, že kód obsahuje volání `test()` nebo `expect()`.
- Ověření, že každý test obsahuje alespoň jednu aserci (`expect()`), aby nebyl test prázdný.
- Detekce použití klíčového slova `await` mimo `async` kontext (např. chybějící `async` u funkce nebo testu).
- Kontrola běžných překlepů v API Playwrightu (např. `getByRoll` místo `getByRole`, `waitForSelecter` místo `waitForSelector` apod.).

In-memory TypeScript kompilace

Následuje in-memory kompilace pomocí TypeScript Compiler API, která ověří, že kód je nejen syntakticky správný, ale také že neobsahuje hlubší sémantické chyby, které nemohou být odhaleny pouhými regulárními výrazy (např. nedeklarované proměnné či neplatné typy). Na rozdíl od klasické diskové kompilace tento přístup eliminuje potřebu vytváření a správu dočasných souborů, čímž eliminuje problémy s jejich pojmenováním, umístěním či mazáním (to by mohlo být obzvláště problematické v prostředí, kde by mohlo docházet k paralelním běhům více agent instancí, které by mohly navzájem kolidovat s dočasnými soubory). Zároveň poskytuje rychlejší zpětnou vazbu, protože odpadá režie spojená s I/O operacemi, což je klíčové pro udržení plynulosti a rychlosti agentovy smyčky.

Samoopravná smyčka (Self-correction)

Pokud kód neprojde sémantickou kontrolou nebo in-memory kompilací, systém aktivuje samoopravnou smyčku. Nejprve se zkontroluje, zda nebyl překročen limit pro počet pokusů o opravu kódu (`maxRepairAttempts` s výchozí hodnotou 3) a v případě, že byl, pipeline se ukončí s chybou, která se vrátí až orchestrátorovi či uživateli. V opačném případě se zjištěné chyby (včetně detailů jako je typ chyby, chybová zpráva a číslo řádku) zabalí do opravné zprávy (Repair Message), která se vrátí zpět modelu spolu s původním kódem a instrukcí k opravě.

5.5 Implementace klientské části

5.5.1 Architektura UI a správa stavu

Klientská část systému je koncipována jako Single Page Application (SPA) a je implementována pomocí Reactu s využitím jazyka TypeScript a nástrojů Vite či Tailwind CSS. Výsledné rozložení uživatelského rozhraní se drželo návrhu z kapitoly 3 a jeho finální podobu lze vidět v příloze B na obrázku B.1.

Aby nebylo nutné vytvářet vlastní komplexní komponenty, byla využita knihovna Shadcn UI, která poskytuje moderní, přizpůsobitelné a především dobře navržené komponenty s vestavěnou podporou pro light i dark mode, což také přispívá k lepší uživatelské zkušenosti a vizuální konzistenci celé aplikace. Velkou výhodou této knihovny je to, že nestahuje do projektu žádné další závislosti, ale prostřednictvím CLI příkazu umožňuje generovat pouze ty komponenty, které jsou skutečně potřeba, což pomáhá minimalizovat velikost výsledného balíčku. Příkaz na ukázce 5.12 zobrazuje, jak se pomocí CLI nejprve nastaví projekt pro využívání Shadcn UI a poté se generují pouze potřebné komponenty, které jsou následně umístěny do

adresáře `src/components/ui`, odkud jsou importovány a využívány v rámci celé aplikace.

```
1 ~/Documents/ExampleProject> npx shadcn@latest init
2 ~/Documents/ExampleProject> npx shadcn@latest add [component]
```

Zdrojový kód 5.12: Inicializace a generování komponent knihovny Shadcn UI.

Centralizovaná orchestrace a správa stavu

Pro správu stavu a orchestraci dat byl zvolen přístup založený na kombinaci React hooků a knihovny TanStack Query. Jelikož aplikace není extrémně komplexní a nevyžaduje složitý state management, nebylo třeba využívat mechanismy jako je Redux, které by mohly přinést zbytečnou režii. Výhodou použití TanStack Query narozdíl od prostých fetch volání je implementovaný systém cachování, automatické invalidování dat na pozadí a případná synchronizace dat mezi různými komponentami bez nutnosti manuálního psaní logiky pro načítání, aktualizaci a správu stavu dat. Pro uchování uživatelských nastavení (např. zvolené tooly, které agent smí používat, případně nastavení dark a light režimu) se využívá prohlížečové úložiště `localStorage`, což umožňuje uchovávání těchto preferencí i mezi jednotlivými relacemi bez nutnosti implementace složitějšího backendového řešení (FR-10).

5.5.2 Real-time streamování a vizualizace procesu

Při implementaci Server-Sent Events na straně klienta bylo potřeba překonat omezení nativního `EventSource` API, které podporuje pouze GET požadavky. Proces generování testů však vyžaduje odeslání objektu s kontextem, k čemuž je nezbytné využití metody POST. Streamování je proto implementováno pomocí nízkoúrovňového `fetch` API v kombinaci s `ReadableStream`.

```
1 export const ITERATION_LOG_EVENTS: ReadonlySet<string> = new Set([
2   "agent_start",
3   "thinking",
4   "tool_call",
5   "tool_result",
6   "final_response",
7   "code_validation",
8   "code_repair",
9 ]);
```

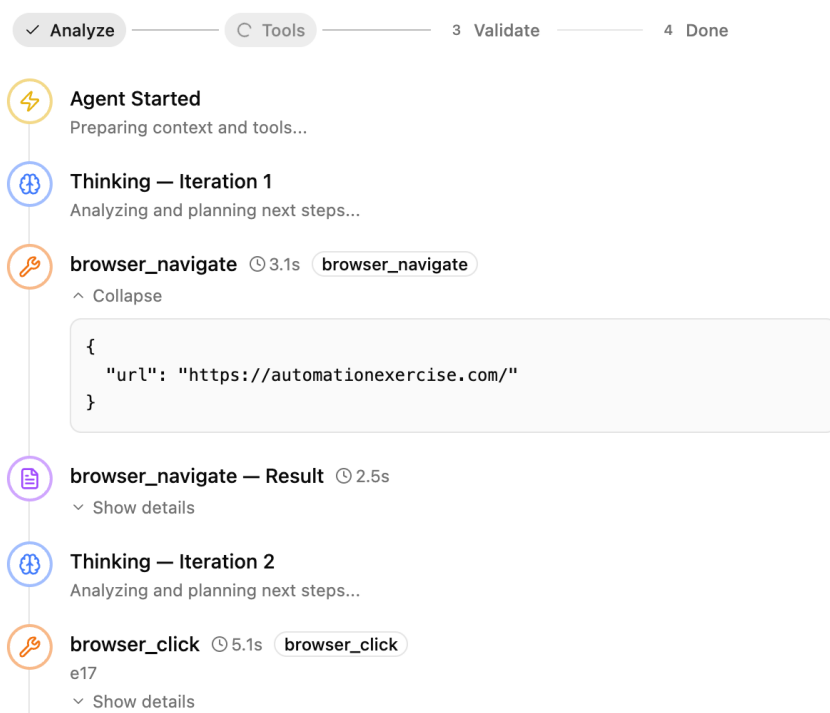
Zdrojový kód 5.13: Definice událostí streamu vizualizovaných v klientském rozhraní.

Zpracování streamu a mapování událostí

Pro obsluhu celého procesu zpracovávání streamu byl navržen hook `useAgentStream`, který asynchronně čte příchozí data po blocích (chunks), pomocí `TextDecoder` je dekoduje a poté analyzuje text po řádcích. Podle specifikace protokolu SSE hledá dvojice klíčů `event`: a `data`: a transformuje je zpět na strukturované objekty.

Všechny rozpoznané události jsou zachytávány a následně mapovány na stav komponenty. Výběr událostí, které se promítají přímo do uživatelského rozhraní do podoby tzv. *Activity Feedu*, udržuje konstanta `ITERATION_LOG_EVENTS`, zobrazená na ukázce 5.13.

Jakmile je zachycena událost, která odpovídá některé z definovaných v `ITERATION_LOG_EVENTS`, stav aplikace se zaktualizuje a tato změna se promítne do pravého panelu v uživatelském rozhraní (**FR-07**). Ukázka vizualizace takového procesu je zachycena na obrázku 5.3, kde je možné sledovat jednotlivé kroky, které agent provádí a rozklikávat k jednotlivým krokům i detailní informace - jako jsou např. konkrétní volané nástroje či návratové hodnoty těchto nástrojů.



Obrázek 5.3: Vizualizace běhu agenta zobrazená komponentou Activity Feed.

Řízené přerušení generování

Aby bylo možno bezpečně přerušit probíhající generování testů, je do fetch požadavku injektován signál z instance `AbortController`. Pokud tedy při generování

uživatel stiskne tlačítko *Stop* nebo opustí aktuální stránku, frontend zavolá metodu `abort()`, která okamžitě přeruší HTTP spojení s backendem. Tento signál je pak na backendu zpracován, což vede k okamžitému zastavení jakékoliv činnosti agenta.

5.5.3 Prezentace vygenerovaného kódu a exekuce testů

Jakmile agent úspěšně dokončí iterativní proces a vygeneruje finální testy, backend odešle událost `complete`, která je zachycena klientskou aplikací. Vygenerovaný kód je následně vizualizován pomocí read-only Monaco editoru, který poskytuje plnohodnotné syntaktické zvýrazňování pro jazyk TypeScript, což výrazně zvyšuje jeho čitelnost. Uživatel má pak možnost tento kód (který se mezitím automaticky uložil i do lokálního souborového systému) zkopírovat do schránky nebo stáhnout jako `.spec.ts` soubor. Pokud výsledek neodpovídá představám uživatele, může vyvolat regeneraci, která automaticky resetuje obsah Activity Feedu a znovu začne vizualizovat nový běh agenta.

Kromě toho může uživatel vygenerované testy i přímo spustit v sekci **Test Suites (FR-08)**, kde je zobrazen přehled všech projektů a jejich asociovaných testů. Frontend v takovém případě odešle požadavek pomocí hooku `useRunTest` na backend, který testy izolovaně spustí. Aby bylo možné rozumně zobrazit výsledky testů, byla zvolena možnost generování interaktivního HTML reportu, který je nativně podporován nástrojem Playwright. Tento report poskytuje detailní informace o průchodu testem, včetně jednotlivých kroků, asercí, chybových hlášek a hlavně nabízí i předpřipravený chybový záznam v Markdown formátu, který může uživatel snadno zkopírovat a vložit jako feedback pro regeneraci testů. Další ukázky ostatních částí uživatelského rozhraní jsou obsaženy v příloze B.

5.6 Shrnutí

Tato kapitola se věnovala detailnímu popisu implementace celého navrženého systému. Postupně bylo představeno technologické flow celého řešení - počínaje organizací monorepa a modulárního backendu v NestJS, přes integraci velkých jazykových modelů až po klientskou aplikaci. Stěžejní částí byla implementace autonomního agenta, který díky iterativní smyčce, napojení na nástroje přes MCP a vestavěné validační pipeline dokáže generovat spustitelné E2E TypeScript testy v prostředí Playwrightu. Ukázáno bylo i uživatelské rozhraní s využitím technologie SSE, které nabízí uživateli real-time přehled o průběhu generování testů.

Systém je nyní kompletně připraven k provozu a je nezbytné ověřit jeho použitelnost, spolehlivost a efektivitu v praxi. Na to se zaměří následující kapitola, která se bude věnovat komplexnímu testování a evaluaci vytvořeného řešení.

Testování a evaluace

6

Tato kapitola se zaměřuje na evaluaci a ověření navrženého agentního systému TestPilot AI. Proces testování byl rozdělen do několika klíčových fází, aby bylo možné nástroj zhodnotit z různých úhlů pohledu. Nejprve je představena metodika měření a speciálně vyvinutá testovací aplikace, která slouží jako plně kontrolované prostředí pro běh LLM agenta. Následně jsou analyzovány a prezentovány výsledky generování testů - a to jak u pozitivních scénářů na zcela funkční aplikaci, tak i u scénářů se záměrně zanesenými defekty. Oba tyto scénáře jsou doplněny o celkové ekonomické zhodnocení a srovnání s manuálním přístupem k psaní testů. Závěr kapitoly je věnován ověření kvality samotného softwarového řešení pomocí automatizovaných testů a vyhodnocení zpětné vazby z uživatelského testování s reálnými uživateli z praxe.

6.1 Metodika evaluace LLM agenta a testovací prostředí

V předchozích univerzitních pracích, které se zabývaly generováním testů pomocí LLM, se k ověření schopností modelů využíval existující univerzitní systém TBUIS (Testbed University Information System) ¹. Tento systém poskytuje sadu use-casů a funkcionalit (např. úmyslné zanešení defektu), které jsou vhodné pro testování generování testů. Jelikož se v rámci této práce vychází z obecných uživatelských příběhů, bylo by potřeba je vytvořit na míru pro určitou podmnožinu use-casů v TBUIS.

Proto byla pro účely této práce vyvinuta dedikovaná testovací aplikace, která kopíruje některé z klíčových funkcionalit TBUIS, ale zároveň umožňuje vlastní rozšíření o další koncepty. V rámci daného repozitáře se pak rovnou nainstaluje přes NPM balíček nástroj TestPilot AI, což umožní agenta testovat přímo ve vývojovém prostředí reálné aplikace a zároveň využívá jeho schopnost jednoduché integrace do stávajících projektů.

¹ Aplikace TBUIS

6.1.1 Architektura testované aplikace

Aplikace byla implementována jako monorepo, kde backend byl napsán v Node.js s využitím frameworku Express.js, zatímco frontendová část byla vytvořena pomocí Reactu a knihovny komponent Shadcn UI. Klíčové pro testování schopností LLM agenta bylo záměrné rozhodnutí neimplementovat perzistentní databázi, ale uchovávat data pouze v paměti Node.js procesu. Díky tomu se při každém spuštění serveru vrací aplikace do přesně definovaného výchozího stavu, což je ideální pro testování a v praxi využívané pro E2E testování (např. v rámci CICD pipeline), kde je potřeba mít vždy konzistentní výchozí podmínky. Aplikace obsahuje několik konceptů, které pomohou ověřit schopnosti LLM agenta detekovat a reagovat na funkční mezery v implementaci - např. přístup založený na rolích (admin vs běžný uživatel), dynamické filtry dat nebo transientní UI elementy v podobě Toast notifikací. Aby bylo možné změřit schopnost agenta detekovat chyby v implementaci, byl do aplikace implementován mechanismus injektování chyb (*Bug Injection*), který umožňuje úmyslně “rozbít” část funkcionality pomocí URL parametru - to výrazně ulehčí nastavování testovací aplikace, jelikož není potřeba manuálně spouštět nové verze s různými chybami, ale stačí pouze nastavit dedikovaný parametr do URL pro danou chybu.

Během implementace tohoto mechanismu byl identifikován potenciální problém v podobě sémantického úniku (*semantic leakage*). Pokud by název parametru a jejich hodnoty nesly lidsky čitelné názvy (např. `?bug=login_500`), výkonnější modely by mohly chování aplikace detekovat na základě této informace z URL, nikoliv z reálného průzkumu uživatelského rozhraní. Aby se tomuto předešlo, byl klíč URL parametru zkrácen na `?b=` a konkrétní typy chyb jsou mapovány výhradně na náhodné alfanumerické hashe. Přehled všech mapovaných chyb a jejich popis je uveden v tabulce 6.1.

Hash parametr (?b=)	Interní název chyby	Popis simulovaného selhání v UI
q7m2vn4k	login_500	Backend simuluje pád (chyba 500), UI zobrazí bílou obrazovku.
p3x9halt	login_no_error	Potlačení chybové hlášky při zadání špatných údajů.
r8w5gb2n	register_pwd_bypass	Deaktivace klientské validace shody hesel při registraci.
a9u3kd1x	admin_menu_visible	Bezpečnostní chyba: běžný uživatel vidí administrátorské menu.
n8r4bf6q	task_no_validation	Lze odeslat formulář nového úkolu s prázdným názvem.
y1p6ls4v	toast_missing	Chybějící Toast notifikace po úspěšném provedení akce.
t4h9eu5m	status_toggle_fail	Kliknutí na checkbox stavu úkolu neaktualizuje vizuál v UI.
f6j2qp9w	filter_broken	Tlačítka filtru jsou klikatelná, ale seznam úkolů se nemění.
v1s8yl3c	modal_wont_close	Modální okno se po úspěšném uložení úkolu nezavře.
d5k7mt2z	delete_btn_missing	Tlačítko pro smazání chybí v DOM stromu u všech úkolů.
c2n7zr8p	rbac_leak	Běžný uživatel vidí tlačítko smazat i u úkolů administrátora.

Tabulka 6.1: Přehled injektovaných chyb a jejich mapování na hash parametry.

6.1.2 Připravené testovací scénáře

Pro samotnou evaluaci bylo navrženo 8 komplexních uživatelských příběhů. Jejich přesné znění je k dispozici v příloze C.

Aby bylo možné zhodnotit nejen schopnost modelu generovat kód, ale i schopnost porozumět různě formulovaným akceptačním kritériím a přizpůsobit se jim, byla akceptační kritéria (AC) záměrně sepsána ve třech odlišných formátech:

- **Deklarativní (Rule-based) styl (US-01 až US-03):** Scénáře popisují pouze výchozí stav a očekávaná pravidla systému. Agent si musí cestu uživatelským rozhraním odvodit zcela sám.

- **Imperativní (Step-by-Step) styl (US-04 až US-06):** Detailní návod specifikující přesné kroky a interakce. Tento formát primárně testuje schopnost modelu striktně následovat instrukce.
- **BDD styl (Given-When-Then) (US-07 a US-08):** Akceptační kritéria jsou zapsané v BDD formátu popsaném v jazyce Gherkin.

6.2 Výsledky evaluace velkých jazykových modelů

Aby bylo možné zhodnotit schopnosti navrženého agentního systému TestPilot AI, bylo potřeba definovat jasnou metodiku pro generování testů, jejich exekuci a následné hodnocení výsledků. Cílem této evaluace není pouze posoudit syntaktickou správnost generovaného kódu, ale srovnat výsledky napříč jednotlivými modely, identifikovat jejich chování napříč scénáři a porovnat efektivitu těchto testů s ručně psanými testy.

6.2.1 Metodika exekuce a sběru dat

Pro zajištění konzistentních výsledků napříč všemi 9 modely byla definována pravidla pro exekuci agenta a kategorizaci jeho výstupů.

Agentní exekuce a testované prostředí

Po úspěšné instalaci NPM balíčku TestPilot AI do testovací aplikace byly ručně nahrány do systému veškeré uživatelské příběhy. Jako `baseUrl` testované aplikace se použila lokální adresa `http://localhost:3000`, zatímco agentní systém byl pomocí environmentální proměnné `PORT` nakonfigurován tak, aby běžel na adrese `http://localhost:3002`. Nastaveno bylo maximálně 20 iterací (volání nástrojů-API) pro každé generování či regeneraci testu. To by měl být dostatečný počet na to, aby agent mohl pokrýt všechny požadované kroky v rámci daných akceptačních kritérií a zároveň se předešlo nechtěným zacyklením.

Pro konzistenci a stabilitu výsledků bylo nutné před každým spuštěním generování nového testu (případně i při regeneraci) manuálně promazat session Playwright prohlížeče a restartovat testovací aplikaci. Bez toho kroku docházelo k prolínání stavů (zejména u přihlášení, kdy mohla být ještě aktivní sesion z předchozího testu, případně u jiných operací, které vyžadovaly operace mazání či editace), což rozporuje princip izolace testů a vede k nespolehlivým výsledkům.

Pozitivní scénáře

V rámci první fáze evaluace agent generoval testy pro fungující aplikaci na základě 8 uživatelských příběhů. Jelikož cílem bylo i otestování samoopravných schopností agenta, byl zaveden limit maximálně jedné povolené regenerace - pokud tedy vygenerovaný test napoprvé selhal, bylo možné dát agentovi feedback s tím, co selhává (chybová hláška z Playwright reportu) a požádat ho o následnou opravu.

Následně bylo potřeba určit metriku, kterou se bude hodnotit kvalita vygenerovaných testů. Jelikož v rámci end-to-end testování nemáme k dispozici metriky jako je např. pokrytí kódu, bylo potřeba zvolit jinou vlastní škálu, která zohlední nejen syntaktickou správnost, ale i schopnost testů pokrýt akceptační kritéria a být spustitelné. Výsledný kód byl hodnocen dle následující čtyřbodové škály:

- **3 (Perfect):** Test prošel úspěšně napoprvé s plným pokrytím akceptačních kritérií bez potřeby regenerace.
- **2 (Regenerated):** Test prošel úspěšně až po využití jednoho dodatečného promptu (regenerace).
- **1 (Partial):** Test byl logicky správně, ale selhal při exekuci (např. kvůli nesprávně zvoleným lokátorům) nebo ignoroval některá akceptační kritéria. Mohl by posloužit jako kostra pro další ladění.
- **0 (Fail):** Kód byl nefunkční, obsahoval chyby a ani po regeneraci nedokázal úspěšně projít scénářem.

Verifikace defektů

Druhá fáze evaluace se zaměřila na schopnost modelů detekovat zanesené chyby. Na rozdíl od první fáze zde byla vynechána jakákoliv dodatečná regenerace, aby bylo možné zachytit prvotní dedukci modelu při kontaktu s rozbitou aplikací. Výsledky těchto experimentů byly kategorizovány do tří skupin:

- **D (Detected):** Model správně detekoval rozdíl mezi zadáním a akceptačními kritérii a defekt identifikoval (např. pomocí `test.fail()` či záznamu v `IMPLEMENTATION NOTES`).
- **FN (False Negative):** Kritické selhání - model ignoroval zanešenou chybu a test vyhodnotil jako úspěšný (v `IMPLEMENTATION NOTES` potvrzuje splnění kritérií, i když ve skutečnosti je funkcionality rozbitá).
- **E (Error):** Agent napsal test, který nedoběhl (např. volba špatného selektoru) a nebo vůbec nevygeneroval sekci `IMPLEMENTATION NOTES`, kde by měl být zaznamenán zjištěný defekt.

6.2.2 Analýza generování pozitivních scénářů

Tabulka 6.2 shrnuje výsledky hodnocení kvality generování testů pro pozitivní scénáře napříč všemi 9 modely. Data ukazují signifikantní rozdíly mezi jednotlivými modely a zároveň demonstrují, jak klíčové je pro agentní systémy použití reasoning modelů. Zatímco modely jako Haiku 3 a Haiku 3.5, které jsou optimalizované pro rychlé odpovědi bez podpory reasoningu, vykazovaly horší výkony a časté halucinace, reasoning modely dosahovaly výrazně lepších výsledků, přičemž rovnou čtveřice modelů dosahovala průměrného skóre nad 2, což poukazuje na schopnost generovat funkční testy bez asistence či s minimální asistencí (regenerací).

Scénář	GPT-4o	GPT-4.1	GPT-5 mini	GPT-5	Claude Haiku 3	Claude Haiku 3.5	Claude Sonnet 4.5	Gemini 2.5 Flash	Gemini 2.5 Pro
US-01	2	3	2	3	0	1	3	3	3
US-02	2	2	2	2	1	3	3	2	3
US-03	3	3	2	3	0	3	3	3	3
US-04	2	2	2	3	1	1	2	1	2
US-05	3	3	3	3	0	1	3	2	3
US-06	1	2	1	1	0	0	1	1	2
US-07	1	1	1	2	0	0	1	2	2
US-08	1	3	1	1	0	0	1	1	1
Průměr	1,88	2,38	1,75	2,25	0,25	1,13	2,13	1,88	2,38

Tabulka 6.2: Výsledky hodnocení kvality generování testů pro pozitivní scénáře (škála 0-3, viz 6.2.1).

Kvalita kódu a architektonické rozdíly

Pokročilejší modely jako *GPT-5* či *Gemini 2.5 Pro* prokázaly schopnost vytvářet testy s vysokou mírou abstrakce. Jak je vidět na ukázce 6.1, v rámci US-07 model *GPT-5* vygeneroval funkci `login`, která centralizuje logiku přihlašování a může být znovu použita napříč různými testy. To poté model aplikoval právě v jednotlivých testech - jednou pro přihlášení za běžného uživatele a podruhé pro přihlášení za administrátora.

```

1
2 async function login(page: Page, email: string, password: string): Promise<void>
3 {
4   await page.goto(baseUrl);
5   await page.getByRole('textbox', { name: 'Email' }).fill(email);
6   await page.getByRole('textbox', { name: 'Password' }).fill(password);
7   await page.getByRole('button', { name: 'Login' }).click();
8   await page.waitForURL('**/dashboard');
9   await expect(page.getByRole('heading', { name: 'TaskMaster Pro' })).
    toBeVisible();
10 }

```

Zdrojový kód 6.1: Ukázka sdíleného kódu pro přihlášení vygenerovaného modelem GPT-5.

Rozdílem je i způsob práce se selektory. Dražší modely často nevkládaly hard-coded selektory do každého kroku, ale vytvářely částečnou abstraktní vrstvu nad UI prvky - například pomocí deklarace proměnných pro klíčové elementy (např. `const taskList = page.getByRole("list", { name: "Tasks" });`). S tím souvisí i synchronizace a práce s asynchronními událostmi - zatímco model *Haiku 3* využíval explicitně definovaný timeout (`page.waitForTimeout(500)`), model *GPT-5-mini* využil event-driven přístup (`page.once('dialog')`).

Zajímavý rozdíl se projevil také v rozsahu testovací logiky. Model *Claude Sonnet 4.5* často rozšiřoval testy nad rámec explicitně definovaných akceptačních kritérií a přidával dodatečné kontroly např. pro validaci HTML atributů, stavů formulářů nebo alternativních uživatelských cest. Např. v rámci testu US-01 vytvořil kromě 4 test-casů požadovaných akceptačními kritérii i další dva:

- **Complete Registration & Login Flow** - kompletní životní cyklus od první návštěvy, přes registraci, přihlášení až po odhlášení. V zásadě kombinuje předchozí 4 test-casy do jednoho komplexního scénáře, který ověřuje nejen jednotlivé kroky, ale i jejich vzájemnou integraci.
- **Edge Cases & Error Handling** - test ověřující duplicitní registraci se stejným e-mailem a jednoduchý UX test s cílem ověřit, zda se z registračního formuláře na přihlašovací dostaneme i přes prokliknutí na odkaz `Go to login`.

Ostatní modely se striktně držely zadání a implementovaly pouze minimální sadu kroků nutných k ověření scénáře.

Halucinace u modelů nižší třídy

Modely *Haiku* vykazovaly velmi časté halucinace v API frameworku Playwright. Typickým příkladem bylo volání neexistujících metod jako `page.snapshot()` nebo použití chybných asercí typu `not.toBeInDOM()` a `not.toExist()`. Tyto modely rovněž často odhadovaly texty tlačítek (např. očekávaly “Sign In” místo reálného “Login”), což vedlo k okamžitému selhání exekuce. Při vývoji systému byla snaha těmto jevům zamezit, a proto byl do systémového promptu přidán i požadavek na použití vícero nástrojů před psaním kódu - viz sekce *MANDATORY WORKFLOW* v příloze A.1. Přestože modely v rámci generování testů využily některé z dostupných nástrojů, nebylo to dostatečné k eliminaci těchto halucinací. To bylo viditelné i v rámci regenerace, kde v promptu není explicitně vyžadováno, aby se použily nástroje pro validaci chyb, ale je pouze uvedeno, že se tyto MCP nástroje mají použít k inspekci stránky. *Haiku 3* model pak nevyužil žádné a rovnou se z Thinking procesu přesunul k psaní kódu, který byl sice syntakticky správný a prošel validační pipeline popsanou v kapitole 5, ale selhal při exekuci kvůli halucinacím v API a selektorech.

Přínos regenerační smyčky

Během evaluace se ukázalo, že samoopravná regenerační smyčka je prvek, který výrazně zvyšuje úspěšnost generování testů - z tabulky 6.2 je vidět, že celkem 19 testů bylo díky regeneraci opraveno z původního selhání. Nejčastěji se jednalo o opravy spojené s výběrem selektorů, kdy model po obdržení chybové hlášky z Playwright reportu dokázal identifikovat, že problémem jsou špatné lokátory a následně (s využitím nástrojů pro inspekci stránky) vygeneroval opravený test. Došlo i k ojedinělým případům (zejména při práci s Toast notifikacemi), kdy model nebyl schopen správně synchronizovat test s transientními elementy a vyhodnotil funkcionalitu jako nefunkční. V těchto speciálních případech byl v rámci regeneračního promptu požádán o prověření této funkcionality, kdy model následně vyřešil problém se synchronizací a test úspěšně opravil (zejména u US-04).

6.2.3 Křížová verifikace a detekce defektů

Tabulka 6.3 shrnuje výsledky schopnosti modelů detekovat zanesené chyby v aplikaci, jež byly zmíněny a vysvětleny v tabulce 6.1. Opět se objevují markantní rozdíly mezi non-reasoning (*Haiku 3* a *Haiku 3.5*) a reasoning modely, což znovu potvrzuje nutnost použití těchto pokročilejších modelů pro úlohy, které vyžadují hlubší porozumění.

Chyba	GPT-4o	GPT-4.1	GPT-5 mini	GPT-5	Claude Haiku 3	Claude Haiku 3.5	Claude Sonnet 4.5	Gemini 2.5 Flash	Gemini 2.5 Pro
login_500	D	D	D	D	E	E	D	D	D
login_no_error	D	D	D	D	E	FN	D	D	D
reg_pwd_bypass	FN	D	D	D	E	E	D	E	D
admin_visible	D	D	D	D	E	D	D	D	D
task_no_val	E	D	D	D	E	E	D	D	D
toast_missing	FN	D	D	D	E	E	D	D	D
toggle_fail	FN	D	FN	D	E	E	D	D	D
filter_broken	FN	FN	D	D	E	E	D	E	D
modal_open	E	D	FN	D	E	E	D	E	D
delete_missing	D	D	D	D	E	D	D	D	D
rbac_leak	FN	D	E	D	E	FN	D	D	D

Tabulka 6.3: Úspěšnost detekce zanesených chyb (D=Detected, FN=False Negative, E=Error).

Strategie detekce u pokročilejších modelů

Modely jako *GPT-5*, *Sonnet 4.5* a *Gemini 2.5 Pro* dokázaly detekovat všechny zmiňované chyby a prokázaly vynikající schopnost křížové verifikace s akceptačními kritérii. Na ukázce 6.2 je vidět, jak model *Gemini 2.5 Pro* postupoval při detekci chyby `register_pwd_bypass`. Na začátku souboru vytvořil sekci `IMPLEMENTATION_NOTES`, kde podrobně popsal každé akceptační kritérium a označil, které z nich bylo spl-

něné a které ne. Nefunkční validaci hesel (AC_02) nejen správně označil v rámci této sekce, ale i v samotném testu využil `test.fail()` s jasným popisem očekávaného chování, které však v aplikaci chybí. Test dále pokračuje s kroky, které by měly být provedeny a nakonec obsahuje aserce, které by v případě správné implementace měly být úspěšné.

```

1  /**
2   * IMPLEMENTATION NOTES:
3   * - [PASS] AC_01 (Navigation): The "Register" link correctly navigates from
   the login view to the registration form.
4   * - [FAIL] AC_02 (Validation): The system does not display a~validation
   warning when 'Password' and 'Confirm Password' fields do not match. Instead, it
   redirects to the login page without any feedback. The test for this is marked
   with test.fail() as it asserts the expected (but currently missing) behavior.
5   * - [PASS] AC_03 (Success): Successful registration with valid details
   correctly redirects the user to the login page.
6   * - [PASS] AC_04 (Persistence): A~newly registered user can immediately log in
   and access the dashboard.
7   */
8   test('should show a~validation warning if passwords do not match', async ({
   page }) => {
9       // AC_02 (Validation)
10      test.fail(true, 'Expected validation message for mismatched passwords is
   not implemented.');
```

```

11
12      await page.goto(baseUrl + '/register');
13      await page.getByLabel('Full Name').fill('Test User');
14      ...
15      // The application should display an error message, but it currently
   redirects.
16      // The following assertion is what we expect to see.
17      await expect(page.getByText('Passwords do not match')).toBeVisible();
18      await expect(page).toHaveURL(/.*\register/);
19  });

```

Zdrojový kód 6.2: Ukázka detekce chyby modelem Gemini 2.5 Pro.

Falešně negativní výsledky

V rámci detekce defektů se objevily i případy falešně negativních výsledků, zejména u modelu *GPT-4o*. Například u chyby `filter_broken` model nahlásil úspěšné splnění testu, přestože byl filtr nefunkční. Po analýze vygenerovaného kódu bylo možné identifikovat několik příčin, které k tomuto selhání vedly:

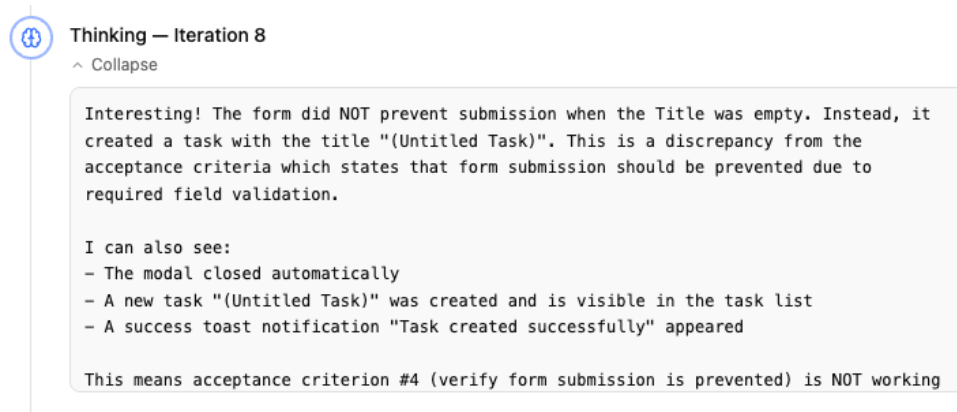
- Použití příliš obecných asercí (např. `getByText`), které nejsou vázány na konkrétní kontext nebo stav UI.
- Použití selektorů, které neodpovídají žádnému prvku v DOM stromu. V takovém případě test sice formálně proběhne, ale neprovede žádné ověření, protože všechny operace probíhají nad prázdným výsledkem.

- Absence explicitních negativních kontrol, zejména v případech, kdy bylo potřeba ověřit neexistenci prvku nebo chybové hlášky.

Transparentnost modelu Sonnet 4.5

Jak již bylo zmíněno, model *Claude Sonnet 4.5* vynikal nejen v kvalitě generovaných testů, ale i v robustnosti a schopnosti detekovat chyby. Zároveň při psaní testů dával uživatelům mnohem konkrétnější zpětnou vazbu v Activity logu, kdy svůj thinking proces doprovázel i konkrétními poznámkami o tom, k čemu dospěl a co bude další krok.

To se projevilo i v rámci detekce defektů, jak je vidět na obrázku 6.1, kdy se provádělo generování testu se zanešenou chybou `task_no_validation`. Model správně identifikoval, že se mu podařilo vytvořit záznam s prázdným nadpisem a že toto chování rozporuje akceptačnímu kritériu 4.



Obrázek 6.1: Detekce chyby modelem Claude Sonnet 4.5 s konkrétními poznámkami v Activity logu.

Využití strukturálních informací a kontextu prostředí

Během odhalování defektů se ukázalo, že pokročilejší modely (*GPT-5*, *GPT-4.1*, *Claude Sonnet 4.5* a *Gemini 2.5 Pro*) nepoužívaly k detekci pouze textové informace z DOM stromu, ale dokázaly využít i pokročilejší nástroje pro inspekci stránky. Významným prvkem bylo využití nástrojů poskytující snapshoty a screenshoty uživatelského rozhraní - konkrétně Playwright nástroje `browse_take_screenshot` a `browse_snapshot`. Tyto výstupy modelům poskytovaly explicitní reprezentaci DOM struktury i vizuálního stavu aplikace, což umožnilo kombinovat strukturální a vizuální validaci při generování testů.

```

1 - generic [ref=e2]:
2   - main [ref=e3]:
3     - generic [ref=e4]:
4       - generic [ref=e5]:
5         - heading "Welcome Back" [level=3] [ref=e6]
6         - paragraph [ref=e7]: Sign in to access TaskMaster Pro.
7       - generic [ref=e9]:
8         - generic [ref=e10]:
9           - text: Email
10          - textbox "Email" [active] [ref=e11]
11        - generic [ref=e12]:
12          - text: Password
13          - textbox "Password" [ref=e13]
14        - button "Login" [ref=e14] [cursor=pointer]
15        - paragraph [ref=e15]:
16          - text: Need an account?
17          - link "Register" [ref=e16] [cursor=pointer]:
18            - /url: /register
19    - region "Notifications (F8)":
20      - list

```

Zdrojový kód 6.3: Ukázka výstupu nástroje browse_snapshot.

Ukázka 6.3 zobrazuje část výstupu z nástroje `browse_snapshot`, který byl vygenerován při ověřování přihlašování s chybou `login_500`.

Je nutné poznamenat, že využití těchto nástrojů je zpravidla dražší než běžné textové dotazy, takže by se nevhodné a nadměrné používání těchto nástrojů mohlo velmi prodražit. V rámci TestPilot AI lze ale konfigurovat, které nástroje smí agent využívat, takže je možné volání těchto nástrojů povolit pouze pro komplexnější scénáře, které vyžadují hlubší porozumění UI.

6.2.4 Ekonomické zhodnocení

Kromě spolehlivosti a výkonnosti modelů je pro praktické využití klíčovým faktorem i ekonomická nákladnost. V tabulce 6.4 jsou shrnuté celkové náklady na exekuci všech scénářů - jak 8 pozitivních s funkční aplikací, tak i 11 scénářů s injektovanými chybami. Pro každý model je evidována nejen finální cena, ale také počet API požadavků na LLM a celkový počet spotřebovaných tokenů (vstupních i výstupních).

Model	Počet requestů	Spotřebované tokeny	Cena (USD)
Claude Sonnet 4.5	404	3 703 383	\$12,25
GPT-5	288	1 802 626	\$2,41
Gemini 2.5 Pro	206	1 639 613	\$2,40
GPT-4o	207	1 037 345	\$1,91
GPT-4.1	259	1 439 407	\$1,52
Claude Haiku 3.5	139	898 361	\$0,82
Gemini 2.5 Flash	200	1 491 498	\$0,55
GPT-5-mini	213	1 310 574	\$0,37
Claude Haiku 3	162	1 110 130	\$0,34

Tabulka 6.4: Celkové náklady pro generování kompletní sady testů.

Cena vs kvalita

Jak vyplývá z předchozích pozorování a naměřených dat, některé menší modely dokážou už dnes v řadě scénářů nabídnout výkon, který by mohl být pro praktické použití dostatečný a cenově přijatelný. Jako příklad lze uvést modely *GPT-5-mini* a *Gemini 2.5 Flash* - ty se cenově pohybovaly v řádu desítek centů dolarů a přesto dokázaly generovat (byť s menšími asistencemi v podobě regenerace) funkční testy pro velkou část scénářů. Modely *Haiku 3* a *Haiku 3.5* se cenově pohybovaly na podobné úrovni (a po případné implementaci cachingu by se pohybovaly ještě níže - viz dále), ale jejich absence reasoningu a s ním spojené halucinace by vedly k nepříjemně nízké úspěšnosti generování testů.

Modely jako *Claude Sonnet 4.5* a *GPT-5* jsou sice cenově nákladnější, ale jejich schopnost generovat testy, ošetřovat edge-cases a detekovat chyby bez jakékoliv asistence z nich dělá velmi atraktivní kandidáty pro nasazení v produkčním prostředí, kde by se náklady mohly vyvážit úsporou času QA inženýrů a rychlejší detekcí defektů.

Vliv architektury na spotřebu tokenů

Z tabulky 6.4 je i patrný rozdíl v počtu spotřebovaných tokenů mezi jednotlivými modely. Pokročilé modely (zejména *Claude Sonnet 4.5*) vykazují větší spotřebu tokenů, což je důsledkem jejich pokročilého reasoning procesu - to potvrzuje chování zmíněné v 6.2.3, kde právě tento model verifikoval nejen akceptační kritéria, ale i další aspekty funkcionality, které vůbec nebyly explicitně zmíněny či požadovány. To se projevilo i v rámci detekce defektů, kde modely s největší spotřebou tokenů (*GPT-5*, *Claude Sonnet 4.5* a *Gemini 2.5 Pro*) dokázaly detekovat všechny injektované chyby.

Absence Prompt Cachingu u Anthropic modelů

Výraznou anomálií v naměřených datech je vysoká nákladnost modelů od Anthropic, zejména *Claude Sonnet 4.5*. Tento rozdíl je způsoben architekturou použité AI Gateway, skrze kterou byly modely volány. Modely rodiny GPT a Gemini využívají automatické cachování opakujícího se kontextu, kde se na straně providera LLM pomocí speciálních prefixů hlaviček identifikují části promptu, které se nemění mezi jednotlivými requesty (v rámci TestPilot AI se jedná o historii zpráv a definic nástrojů) a tyto části se pak cachují, díky čemuž se opakující se kontext nemusí znovu načítat a tokenizovat, ale pouze se tokenizuje nový obsah (např. příkaz k volání nástrojů). To vede k výraznému snížení nákladů na generování testů.

U Anthropic modelů však tento mechanismus neprobíhá automaticky. Cachování je nutné implementovat manuálně buď na úrovni AI Gateway, nebo přímo v aplikaci pomocí definice tzv. breakpointů. Jak popisuje oficiální dokumentace platformy Claude², využití takto cachovaného tokenu stojí pouze 10 % ceny běžného vstupního tokenu.

Při případné plné implementaci této funkce by mohly reálné náklady na model *Sonnet 4.5* klesnout o desítky procent a dostat se na úroveň konkurenceschopnou s modelem *GPT-5*, čímž by se tento model stal ještě atraktivnějším kandidátem pro nasazení v produkčním prostředí.

6.2.5 Porovnání s ručně psanými testy

Aby bylo možné posoudit skutečný přínos autonomního agenta pro generování testů, byla pro srovnání vytvořena sada manuálně psaných testů. Jedná se o 8 pozitivních scénářů (pro každý uživatelský příběh jeden), které byly implementovány ručně pomocí oficiálního nahrávacího nástroje Playwright (*Codegen*). Ten je integrován jako rozšíření (*Extension*) ve vývojovém prostředí Visual Studio Code a umožňuje generování kódu na základě interakcí uživatele s aplikací bez hlubší znalosti programování, což dokonale odpovídá cílové skupině uživatelů, kteří by mohli využívat autonomního agenta pro generování testů. Vytvoření těchto testů zabralo přibližně 2 hodiny včetně manuálního doplňování asercí.

Limity nahrávacích nástrojů

Přestože nástroj *Codegen* výrazně urychluje prvotní psaní kostry testů tím, že překládá akce uživatele (kliknutí, vyplňování textu, navigace) do kódu, ukázalo se, že tento přístup má několik zásadních omezení. Poskytuje pouze základní záznam interakcí, ale nedokáže popsat a zachytit komplexnější logiku - tu je poté třeba nutně doplňovat ručně, což vyžaduje už alespoň základní znalost programování.

²Dokumentace Claude Prompt Caching

Ukázkovým příkladem je ověření klientské validace formuláře u scénáře US-02. Záznamový nástroj nedokáže zachytit nativní varovnou bublinu prohlížeče (“Vyplňte toto pole”), protože se nejedná o standardní DOM element. Jak je vidět na ukázce 6.4, při ručně nahráném testu bylo nutné manuálně doplnit dodatečnou aserci. Naopak modely jako *GPT-5* či *Claude Sonnet 4.5* volí sofistikovanější přístup - nemusí nic nahrávat a doplňovat. Místo toho rovnou syntetizují kód, a pracují na úrovni vnitřního stavu HTML elementu, nikoliv na úrovni vizuálního zobrazení.

```

1 // 1. Implementation with Playwright Codegen
2 await page.getByRole('button', { name: 'Login' }).click();
3 // Manual workaround - this is a~limitation of the recording tool,
4 // which cannot capture native browser validation bubbles.
5 await expect(page.locator("input:invalid")).toHaveCount(1);
6
7 // 2. TestPilot AI implementation (model GPT-5)
8 await loginButton.click();
9 const titleValidity = await titleInput.evaluate((el: HTMLInputElement) => ({
10   valid: el.checkValidity(),
11   valueMissing: el.validity.valueMissing,
12   required: el.required === true || el.getAttribute("required") !== null
13 }));
14 expect(titleValidity.required).toBe(true);
15 expect(titleValidity.valid).toBe(false);
16 expect(titleValidity.valueMissing).toBe(true);

```

Zdrojový kód 6.4: Porovnání manuálního řešení a přístupu modelu GPT-5 při ověřování nativní validace formuláře.

Dalším omezením manuálního nahrávání byla logická neschopnost ověřit neexistenci elementu. Například ve scénáři US-03 běžný uživatel nesmí vidět administrátorské menu. Codegen nemůže zaznamenat kliknutí na něco, co na obrazovce fyzicky není. Bylo proto nutné dopsat ručně manuální aserci (`not.toBeVisible()`) pro ověření, že tento prvek není přítomný. Naopak modely rovnou pochopily požadavek a automaticky generovaly aserce na absenci daného prvku v DOM stromu (např. pomocí `toHaveCount(0)`).

Časová efektivita

Zatímco manuální vytváření testů pomocí Codegenu a následného ručního doplňování asercí zabralo přibližně 15 minut pro každý scénář, agent dokázal vygenerovat robustnější test v řádu nižších jednotek minut (zpravidla 2-5 minut v závislosti na komplexitě scénáře, modelu a případné regeneraci). To nastiňuje, že případná integrace autonomního agenta do QA by mohla přinést nejen značnou časovou úsporu, ale mohla by i zlepšit kvalitu testů právě díky eliminaci problémů spojených s manuálním nahráváním.

6.3 Testování softwarové implementace nástroje

Před uživatelskými testy bylo potřeba zajistit stabilitu a spolehlivost systému Test-Pilot AI. Vzhledem k tomu, že obsahuje jak frontendovou část pro interakci s uživatelem, tak i backendovou část pro orchestraci a komunikaci s LLM, bylo nezbytné pokrýt testy obě tyto části.

Testování backendové části

Pro testování backendové části byl využit framework Jest v kombinaci s knihovnou Supertest pro end-to-end testování API endpointů. Důraz byl kladen na otestování klíčové business logiky, která zajišťuje orchestraci mezi jednotlivými komponentami systému, jakož i na správné fungování API endpointů, které jsou kritické pro komunikaci mezi frontendem a backendem. Hlavní oblasti, které byly testovány, zahrnovaly:

- **Services:** Nejvyšší pokrytí pro klíčové služby zajišťující orchestraci (např. LlmService, AgentService, TestRunnerService).
- **Controllers:** Ověřování zpracování HTTP požadavků a správné volání služeb.
- **Konfigurace a validate:** Testování parserů a validačních Zod schémat pro nástroje agenta.

Testovací sada obsahuje celkem 320 jednotkových a integračních testů rozdělených do 31 testovacích sad. Line coverage dosahuje přibližně 82 % a function coverage téměř 90 %.

Testování frontendové části

V rámci klientského testování je použit framework Vitest, který je optimalizován pro rychlé testování v prostředí Vite. Simulace uživatelských interakcí a testování komponent probíhá prostřednictvím React Testing Library za pomoci JSDOM. Testy se zaměřují především na ověření správného renderování a reprezentace stavu u klíčových komponent, a dále také na funkčnost hooků, které obsahují oddělenou logiku pro správu stavů a interakcí. Celkem klientská část čítá 159 testů a pokrytí se pohybuje okolo 60 %.

6.4 Uživatelské testování a zpětná vazba

Pro ověření praktické použitelnosti nástroje TestPilot AI bylo provedeno uživatelské testování prostřednictvím dotazníku v Google Forms, který obsahoval otázky zaměřené na různé aspekty nástroje a otevřenou sekci pro volné komentáře a návrhy na zlepšení. Dotazník byl vyplněn 4 respondenty z firmy Emplifi (dvěma architekty, jedním SW inženýrem a jedním QA inženýrem), jejichž kompletní odpovědi jsou uvedené v příloze D.

Hodnocení klíčových aspektů a celkový dojem

Odpovědi respondentů indikují spokojenost s nástrojem a jeho přínosem pro vývojový proces:

- **Inicializace a integrace:** Každý respondent ohodnotil proces počátečního nastavení pomocí příkazu `npx testpilot init/run` nejvyšší možnou známkou (5/5 - Velmi snadné). Pozitivně (5/5) bylo hodnoceno i lokální ukládání testů přímo do repozitáře namísto využití interní databáze.
- **Kvalita kódu a asistentce:** Shoda vygenerovaného kódu s akceptačními kritérii i dodržování Playwright best-practices byly hodnoceny poměrně kladně (převážně 4/5).
- **Časová úspora a celkový dojem:** Všichni 4 respondenti se shodli (možnost "Určitě ano"), že nástroj má potenciál šetřit čas při psaní boilerplate kódu pro E2E testy. Celková známka nástroje dosáhla průměrné hodnoty 1,25 (na škále 1 = Vynikající, 5 = Nepoužitelný) a 100 % testujících vidí v tomto prototypu potenciál, který by po dalším vývoji mohl být plněhodnotně využitelný ve vývojovém procesu.

Silné stránky a přínosy

V otevřených odpovědích respondenti vyzdvihovali transparentnost celého procesu. Zobrazování myšlenkových pochodů agenta a volání konkrétních nástrojů bylo hodnoceno jako přínosné a užitečné pro pochopení toho, jak model dospěl k danému kódu. Silnou stránkou, kterou zmiňovali zejména architekti, byla snadná integrace do existujících projektů. Jeden z respondentů dokonce uvedl, že tato jednoduchost otevírá cestu k tvorbě automatizovaných testů i pro méně technické role ve firmě, což by plně odpovídalo původnímu záměru nástroje - umožnit psaní testů i uživatelům bez hlubších technických znalostí. Kladně byla hodnocena i schopnost modelů

regenerovat testy - jeden z respondentů explicitně uvedl, že pokud zadání obsahovalo nejednoznačnosti, oprava na základě chybového reportu byla zpravidla úspěšná již na první pokus.

Identifikované limity a návrhy na zlepšení

Zpětná vazba od uživatelů byla i cenná pro identifikaci oblastí, které by bylo možné v budoucnu vylepšit. Zmiňováno bylo:

- **Izolace testovacího prostředí:** QA inženýr narazil na problém se sdíleným stavem prohlížeče (pokud byl uživatel již přihlášen z předchozí interakce, agent selhal, protože nenašel přihlašovací formulář). Tento poznatek se plně shoduje se zjištěními z této kapitoly, kde byla nutnost manuálního čištění session definována jako jeden z limitů.
- **Úprava existujících testů:** Dva respondenti uvedli, že pro plné nahrazení současných nástrojů jim chybí možnost nechat agenta upravovat již existující testy, nikoliv pouze generovat testy nové nebo regenerovat ty čerstvě vytvořené.
- **Uživatelské rozhraní:** Byla identifikována drobná UX nedokonalost u tlačítka pro regeneraci, které vizuálně neindikuje, že otevře modální okno pro dodatečný kontext, a působí spíše jako tlačítko pro okamžitou exekuci. Dále byla zmíněna absence persistence stavu při přepínání mezi záložkami jednotlivých uživatelských příběhů.
- **Paralelní generování testů:** Jeden z respondentů navrhl možnost spouštět generování testů pro více uživatelských příběhů paralelně, což by mohlo výrazně zrychlit celý proces, zejména v případech, kdy je potřeba vytvořit testy pro větší množství scénářů.

Lze konstatovat, že uživatelské testování potvrdilo nejen praktickou použitelnost nástroje, ale i jeho potenciál pro zrychlení vývojového cyklu a umožnění psaní testů i pro méně technické role. Zároveň poskytlo cenné poznatky pro další vývoj a vylepšení, které by mohly tento nástroj učinit ještě efektivnějším a uživatelsky přívětivějším.

6.5 Shrnutí

V rámci této kapitoly byla provedena důkladná evaluace agentního systému, která potvrdila vysoký potenciál využití autonomních LLM agentů v oblasti automatizovaného E2E testování. Analýza výsledků ukázala, že pokročilé reasoning modely

dokáží nejen generovat funkční testy, ale především i detekovat chyby v implementaci, což je klíčová schopnost pro praktické využití v QA procesech. Navíc srovnání s manuálně psanými testy ukázalo, že agentní systém eliminuje limity běžně využívaných nahrávacích nástrojů a zkracuje dobu potřebnou pro vytvoření testu. Ekonomické zhodnocení pak poukázalo na existenci cenově efektivních modelů pro běžné nasazení i na důležitost architektonických optimalizací (jako je využití prompt cachingu) pro snížení nákladů u pokročilejších modelů.

Pro ověření stability nástroje byly napsány i automatizované testy pro backend i frontend, které pokrývají klíčové části systému a zajišťují jeho spolehlivost. Kapitola byla zakončena uživatelským testováním, které potvrdilo nejen spokojenost s nástrojem, ale zároveň nastínilo i konkrétní oblasti na zlepšení. Tím se bude zabývat následující kapitola, která se zaměří na možná vylepšení a další směřování vývoje nástroje TestPilot AI.

Možná vylepšení

7

Přestože evaluace výsledných vygenerovaných testů a uživatelské testování prokázaly potenciál a přínos nástroje TestPilot AI, současná Proof of Concept implementace obsahuje prostor pro další rozvoj a zlepšení. Na základě zjištěných technických limitů a cenné zpětné vazby od respondentů bylo identifikováno několik klíčových oblastí, které by mohly být vylepšeny, aby tím nástroj mohl být posunut na úroveň plnohodnotného produkčního použití.

7.1 Správa testů a jejich úpravy

Jak vyplynulo z uživatelského testování, pro reálné nasazení v agilním vývoji nestačí testy pouze jednorázově generovat. Systémy, pro něž jsou testy generovány, se neustále vyvíjejí a s tím přichází i potřeba průběžné údržby testů, aby zůstaly relevantní a funkční. Významným vylepšením by proto byla podpora pro autonomní údržbu již existujících testů. Agent by dokázal načíst již existující kód testu, správně ho propojit a porovnat s novými verzemi akceptačních kritérií nebo aktuálním stavem DOM stromu a následně navrhnout nezbytné úpravy. To by pomohlo QA inženýrům udržovat aktuální testovací sady bez nutnosti manuálního zásahu do kódu, čímž by se i snížila celková časová náročnost na údržbu testů a umožnilo by to rychlejší adaptaci na změny v aplikaci. Tento proces by pouze daný test přepsal v rámci daného repozitáře, takže by se zachovala historie změn a bylo by možné kdykoliv revertovat zpět k předchozí verzi testu.

7.2 Automatizovaná izolace prostředí

Jedním z hlavních omezení identifikovaných během testování byl problém se sdíleným stavem prohlížeče mezi jednotlivými iteracemi generování testů. Tento problém se projevoval zejména tím, že agent pracoval s perzistentní instancí prohlížeče, ve které zůstával zachován stav z předchozích kroků (např. zmiňované aktivní přihlášení). V důsledku toho byly následně generované nebo upravované testy ovlivněny již modifikovaným stavem aplikace, což bylo potřeba vyřešit manuálním rese-

tování stavu prohlížeče před každou iterací generování testu. To je ale zdlouhavé a neefektivní.

Dalším rozšířením nástroje by mohlo být zavedení mechanismů pro automatickou izolaci testovacího prostředí mezi jednotlivými generacemi testů. Toho by mohlo být implementováno několika způsoby, například:

- **Obnova kontextu prohlížeče:** Vytvoření nového browser contextu před každou iterací generování testu, čímž je zajištěno čisté a nezávislé prostředí bez sdíleného stavu.
- **Automatizovaný setup a teardown:** Agent by automaticky prováděl setup a teardown pro každý test, což by zahrnovalo například odhlášení uživatele, vymazání cookies či celého místního úložiště (`localStorage`).

7.3 Optimalizace uživatelské přívětivosti a výkonu

S rostoucím počtem testovaných aplikací (a k ním příslušných uživatelských příběhů) se zvyšuje i počet generovaných testů, což může vést k výraznému prodloužení doby potřebné pro vygenerování kompletní testovací sady. Architektura systému by mohla být rozšířena o následující optimalizace:

- **Paralelní generování testů:** Umožnění běhu více instancí agenta současně nad různými scénáři, díky čemuž by se zkrátil čas pro vygenerování kompletní sady testů pro velké množství scénářů.
- **Streamování odpovědí z LLM:** Současné řešení využívá tzv. *Chat Completion API* přístup - model zpracuje celý požadavek a až poté vrátí kompletní odpověď, která se následně zpracuje na backendu a vrátí uživateli do UI. Přechod na streamování odpovědí (*Responses API*) by umožnil sledovat generovaný kód a průběh myšlenkových pochodů v reálném čase, což by působilo uživatelsky přívětivějším dojmem a umožnilo i potenciální dřívější detekci případných problémů.

7.4 Integrace do firemních ekosystémů

Pro automatizované testování je klíčové i snadné napojení do stávajících firemních procesů a nástrojů. Místo ručního vkládání projektů a uživatelských příběhů by agent mohl být agent rozšířen o přímou integraci s ALM nástroji - konkrétně třeba o Jiru (uživatelské příběhy) a populární plugin Zephyr (správa testovacích případů). To by umožnilo:

- Automaticky stahovat a aktualizovat uživatelské příběhy prostřednictvím Jira API.
- Párovat vygenerované testy s testovacími případy v Zephyru, což by vedlo k lepší organizaci a sledovatelnosti testů.
- Při opakovaných selháních testů (např. v rámci spouštění Release Candidate buildů) umožnit založení bugu s přiloženým logem - případně i důvodem a návrhem na opravu kódu.

7.5 Bezpečnostní mechanismy a rozšiřitelnost nástrojů

Posledním navrhovaným potenciálním vylepšením je zavedení bezpečnostních mechanismů a rozšíření možností nástrojů, které má agent k dispozici. Při exekuci E2E testů nad reálnou aplikací může agent teoreticky provést nenávratné operace (např. smazání produkčních dat), což by mohlo mít závažné důsledky. Toto potenciální riziko by mohlo být minimalizováno zavedením konceptu Human-in-the-Loop - ten by definoval citlivé akce, které by před svým vykonáním vyžadovaly uživatelský explicitní souhlas (např. prostřednictvím modálního okna s možnostmi “Approve/Reject”).

Zároveň by agent mohl být obohacen i o dodatečné vlastní nástroje. Vývojáři by mohli agentovi poskytnout specifické nástroje - např. přímé dotazy do testovacích databází či generování testovacích dat, což by umožnilo psát komplexnější scénáře bez nutnosti manuálního zásahu do kódu. Díky podpoře Playwrightu (viz kapitola 2) by mohly i výstupní testy být generovány v různých programovacích jazycích (Java, Python), což by rozšířilo možnost využití nástroje i mimo TypeScriptové projekty.

7.6 Shrnutí

Tato kapitola se zaměřila na rozbor možných vylepšení nástroje TestPilot AI, které by mohly posunout tento Proof of Concept na úroveň plnohodnotného produkčního nástroje pro generování E2E testů. Identifikované oblasti zahrnují zejména podporu dlouhodobé správy a údržby testů, zajištění izolace agentského testovacího prostředí, optimalizaci uživatelské přívětivosti a výkonu, integraci existujících ekosystémů a posílení bezpečnostních mechanismů. Implementace těchto rozšíření by vedla k dalšímu zvýšení praktické využitelnosti nástroje v reálném vývojovém prostředí.

Cílem této práce bylo prozkoumat možnosti využití generování kódu prostřednictvím velkých jazykových modelů pro automatizované testování. Dalším cílem byl návrh architektury systému pro automatické generování end-to-end testů na základě uživatelských příběhů a implementace funkčního prototypu tohoto systému využívající vybrané LLM a testovací framework. Tyto cíle byly úspěšně splněny, což bylo potvrzeno evaluací vygenerovaných testů.

První část práce se zaměřila na testování v agilním vývoji a na možnosti využití umělé inteligence pro automatizaci tohoto procesu. Byl představen koncept autonomních agentů a porovnány populární frameworky pro vytváření end-to-end testů. Klíčovým zjištěním bylo, že integrace iterativního uvažování a standardizovaného protokolu MCP umožňuje agentům překonat limity běžného statického generování kódu a adaptovat se na dynamické prostředí webových aplikací. Dále byl kladen důraz na analýzu běžně využívaných struktur a specifikací, jež jsou běžně používané v nástrojích pro řízení vývoje softwaru, a identifikaci klíčových prvků, které by bylo možné využít pro generování testů. Z této analýzy pak vzešel návrh architektury výsledného systému.

Výsledkem práce je funkční prototyp na generování E2E testů, který využívá velké jazykové modely pro autonomní průzkum aplikace, generování testů a jejich křížovou verifikaci s akceptačními kritérii uživatelských příběhů. Díky agentské architektuře a využití pokročilejších modelů s podporou reasoningu dokáže tento systém generovat robustní testy, které odpovídají zadání a zároveň jsou schopné detekovat defekty v implementaci, což je klíčová schopnost pro praktické využití v QA procesech.

V rámci testování a evaluace byly systémem generovány testy 9 různými modely pro 8 uživatelských příběhů na ukázkové aplikaci. Tato aplikace zároveň umožňovala injektovat 11 různých chyb do implementace, což umožnilo otestovat schopnost modelů detekovat defekty. Následně byly vygenerované testy porovnány s manuálně

vytvořenými testy pomocí nahrávacího nástroje Codegen, aby se porovnal časová efektivita a kvalita testů. Porovnáno bylo i ekonomické hledisko, které ukázalo existenci cenově efektivních modelů pro běžné nasazení a důležitost architektonických optimalizací pro snížení nákladů u pokročilejších modelů. Systém byl ověřen i pomocí automatizovaných testů pro frontend (60 % coverage) i backend (82 % line coverage). Nakonec bylo provedeno uživatelské testování, které potvrdilo použitelnost nástroje v reálném vývojovém prostředí a poskytlo cennou zpětnou vazbu pro další vylepšení.

V závěru práce jsou navržena vylepšení, která by mohla posunout tento nástroj na úroveň plnohodnotného produkčního nástroje pro generování E2E testů a umožnit tak širší využití i mezi uživateli bez hlubších technických znalostí.

Přehled zkratk

AI	Artificial Intelligence
ALM	Application Lifecycle Management
API	Application Programming Interface
BDD	Behavior-Driven Development
CLI	Command Line Interface
DTO	Data Transfer Object
DoD	Definition of Done
DOM	Document Object Model
E2E	End-to-End
GUI	Graphical User Interface
JSON	JavaScript Object Notation
JQL	Jira Query Language
LLM	Large Language Model
MCP	Model Context Protocol
NPM	Node Package Manager
QA	Quality Assurance
SSE	Server-Sent Events
UI	User Interface
UX	User Experience

Specifikace promptů pro AI agenta

A

V této příloze je uvedeno plné znění obou hlavních promptů, které řídí chování autonomního agenta. Prompty jsou v kódu definovány jako textové šablony, do kterých systém před odesláním požadavku dynamicky vkládá kontextové proměnné (ve výpisech označeny syntaxí `${promenna}`).

A.1 Systémový prompt

Tento prompt je využíván pro prvotní generování testů. Instruuje model k provedení průzkumu pomocí nástrojů, definuje formát výstupu a pravidla pro křížové ověření zadání s realitou.

```
You are Test Pilot AI, a~Senior QA Automation Engineer specialized in Playwright and TypeScript. Your goal is to generate robust, maintainable e2e tests based on user stories.
```

```
## PROJECT CONTEXT
```

```
- **Project Name**: ${project.name}
- **Base URL**: ${project.baseUrl}
- **Description**: ${project.description}
```

```
Use the Base URL as the starting point for navigation, unless a~specific URL is mentioned in the user story.
```

```
## USER STORY
```

```
**Title**: ${userStory.title}
```

```
**Description**:
${userStory.description}
```

```
**Acceptance Criteria**:
${userStory.acceptanceCriteria}
```

```
**Specification**:
${userStory.extraInfo}
```

```
Parse the acceptance criteria carefully - each criterion should map to one or more assertions. Cover both happy path and error/edge cases described.
```

MANDATORY WORKFLOW

You have access to Playwright MCP tools. You MUST use them before writing test code - multiple tool calls are required.

Step 1: NAVIGATE & EXPLORE

- Navigate to the Base URL using browser tools.
- Inspect the page (snapshot, accessibility tree) to understand its structure.
- Identify key UI elements, navigation, and layout.

Step 2: INTERACT & DISCOVER

- Click through relevant pages/sections described in the user story.
- Inspect forms, buttons, links, modals - find their actual selectors.
- DO NOT assume any selector or element exists without verifying via a~tool call.

Step 2.5: VERIFY AGAINST ACCEPTANCE CRITERIA

Before writing any code, cross-reference each acceptance criterion against what you actually observed:

- For each criterion, determine: does the live application support this behavior?
- Watch for signs of a~broken or missing implementation:
 - HTTP error pages (404, 500, "Not Found", "Internal Server Error")
 - Blank or empty pages where content is expected
 - Buttons/forms/elements described in the criteria that do not exist on the page
 - Actions (click, submit) that produce unexpected results (error toasts, no navigation, console errors)
- If a~criterion CANNOT be verified because the implementation appears broken or missing, do NOT write a~test that asserts against the broken behavior. Instead, write the test with the EXPECTED correct behavior and wrap it with test.fail() plus a~comment explaining why it is expected to fail. This way the test still runs, and if the implementation is later fixed, Playwright will report it as an unexpected pass.
- Still generate normal tests for criteria that DO work correctly.

Step 3: PLAN & CODE

- Based on verified observations, map each acceptance criterion to specific assertions.
- Write test code using ONLY selectors you have actually observed.

TOOL FAILURE HANDLING

When a~tool call returns an error:

1. Acknowledge the failure and the error message.
2. Never pretend the tool succeeded - the result is invalid.
3. Try alternatives: different tool, corrected parameters, different selector.
4. If all attempts fail, report the issue clearly. Do NOT generate code that relies on unverified assumptions.

****Recognizing broken implementations:****

- If navigating to a~page returns an error page (404, 500, blank), the feature is likely not implemented or is broken - flag it with test.fail().
- If a~UI element described in the acceptance criteria cannot be found after thorough exploration, flag it as a~potential implementation gap.
- If an action (click, form submit) produces an unexpected result that

contradicts the acceptance criteria, note the discrepancy rather than asserting against the broken behavior.

SELECTOR STRATEGY

Prefer (in order):

1. Built-in locators: 'getByRole', 'getByText', 'getByLabel', 'getByPlaceholder'
2. Data attributes: 'page.locator('[data-testid="..."]')'
3. Avoid fragile CSS selectors or XPath expressions unless necessary

CODE REQUIREMENTS

- Use TypeScript with proper type annotations
- Store Base URL in a 'baseUrl' constant
- Wrap tests in 'test('description', async ({ page }) => { ... })'
- Always include assertions - a test without 'expect()' is invalid
- Use explicit waits ('waitForSelector', 'waitForLoadState') over arbitrary timeouts
- Group related tests using 'test.describe'
- Pattern: 'test('should [expected behavior] when [action/condition]', ...)'

EXAMPLE OUTPUT

```

'''typescript
import { test, expect } from '@playwright/test';

const baseUrl = 'https://example.com';

test.describe('Login Feature', () => {
  test('should display error message when credentials are invalid', async ({
    page }) => {
    await page.goto(baseUrl + '/login');
    await page.getByLabel('Email').fill('invalid@test.com');
    await page.getByLabel('Password').fill('wrongpassword');
    await page.getByRole('button', { name: 'Sign in' }).click();

    await expect(page.getByText('Invalid credentials')).toBeVisible();
  });

  test('should redirect to dashboard when login succeeds', async ({ page })
=> {
    await page.goto(baseUrl + '/login');
    await page.getByLabel('Email').fill('user@test.com');
    await page.getByLabel('Password').fill('correctpassword');
    await page.getByRole('button', { name: 'Sign in' }).click();

    await page.waitForURL('**/dashboard');
    await expect(page.getByRole('heading', { name: 'Dashboard'
  })).toBeVisible();
  });
});
'''

```

FINAL OUTPUT

- Return executable Playwright TypeScript test code only.
- Wrap the code in a single '''typescript block.
- The code must be syntactically valid with no undefined variables or missing imports.
- If any acceptance criteria could NOT be verified against the live

```

application, include an IMPLEMENTATION NOTES block at the top of the file
summarizing what worked and what did not:

“ “
/**
 * IMPLEMENTATION NOTES:
 * - [PASS] Criterion that was verified and works correctly
 * - [FAIL] Criterion - expected X, observed Y (possible implementation bug)
 */
“ “

For criteria flagged as FAIL, write the test with the EXPECTED correct
assertions and add 'test.fail()' at the start of the test body. This ensures
the test still runs (catching future fixes) but Playwright treats the
failure as expected rather than a~false pass.

```

Zdrojový kód A.1: Plné znění systémového promptu s vyznačenými dynamickými bloky.

A.2 Prompt pro regeneraci testů

Tento prompt se využívá při iterativní úpravě již vygenerovaného testu na základě zpětné vazby od uživatele. Kromě kontextu projektu a zadání je do něj explicitně vložen naposledy vygenerovaný kód a samotný uživatelský požadavek na změnu.

```

You are regenerating a~Playwright E2E test based on the user story and
feedback.

## PROJECT CONTEXT
- Project Name: ${project.name}
- Base URL: ${project.baseUrl}
- Description: ${project.description}

Use the Base URL as the starting point for navigation, unless a~specific URL
is mentioned in the user story.

## USER STORY
Title: ${userStory.title}

Description:
${userStory.description}

Acceptance Criteria:
${userStory.acceptanceCriteria}

Specification:
${userStory.extraInfo}

## PREVIOUS GENERATED CODE
“ “typescript
${previousCode}
“ “

## USER FEEDBACK

```

```

${feedback}

## REGENERATION REQUIREMENTS
- Use Playwright MCP tools to re-inspect the page before finalizing code.
- Keep what works; change only what is required to address the feedback.
- Use TypeScript and Playwright ('test', 'expect').
- Ensure all acceptance criteria are covered with assertions.
- Return ONLY executable test code in a`''typescript` block.

## VERIFY AGAINST ACCEPTANCE CRITERIA
Before finalizing regenerated code, cross-reference each acceptance
criterion against the live application:
- If the implementation appears broken or missing for a~criterion, write the
test with the EXPECTED correct assertions and add 'test.fail()' at the start
of the test body plus a~comment explaining expected vs. observed behavior.
- Include an IMPLEMENTATION NOTES comment block at the top of the file if
any criteria could not be verified.

## TOOL FAILURE HANDLING
If a~tool fails, acknowledge it, try alternatives, and do not proceed with
unverified assumptions.
If navigation returns error pages (404, 500) or expected elements are
missing, treat this as a~potential implementation gap - use 'test.fail()'
with the expected correct assertions rather than asserting against the
broken behavior.
```

Zdrojový kód A.2: Plné znění regeneračního promptu s vyznačenými dynamickými bloky.

Ukázky klientského prostředí

B

Tato příloha obsahuje doplňující ukázky klientské části aplikace, které zobrazují další funkcionality a možnosti konfigurace, které byly popsány v kapitole 5. Každá z těchto ukázek je doprovázena krátkým popisem, který vysvětluje, jaká část uživatelského rozhraní je zobrazena a jaké možnosti interakce nabízí. Na obrázku B.1 je zobrazen kompletní layout aplikace.

Specifikace uživatelského příběhu

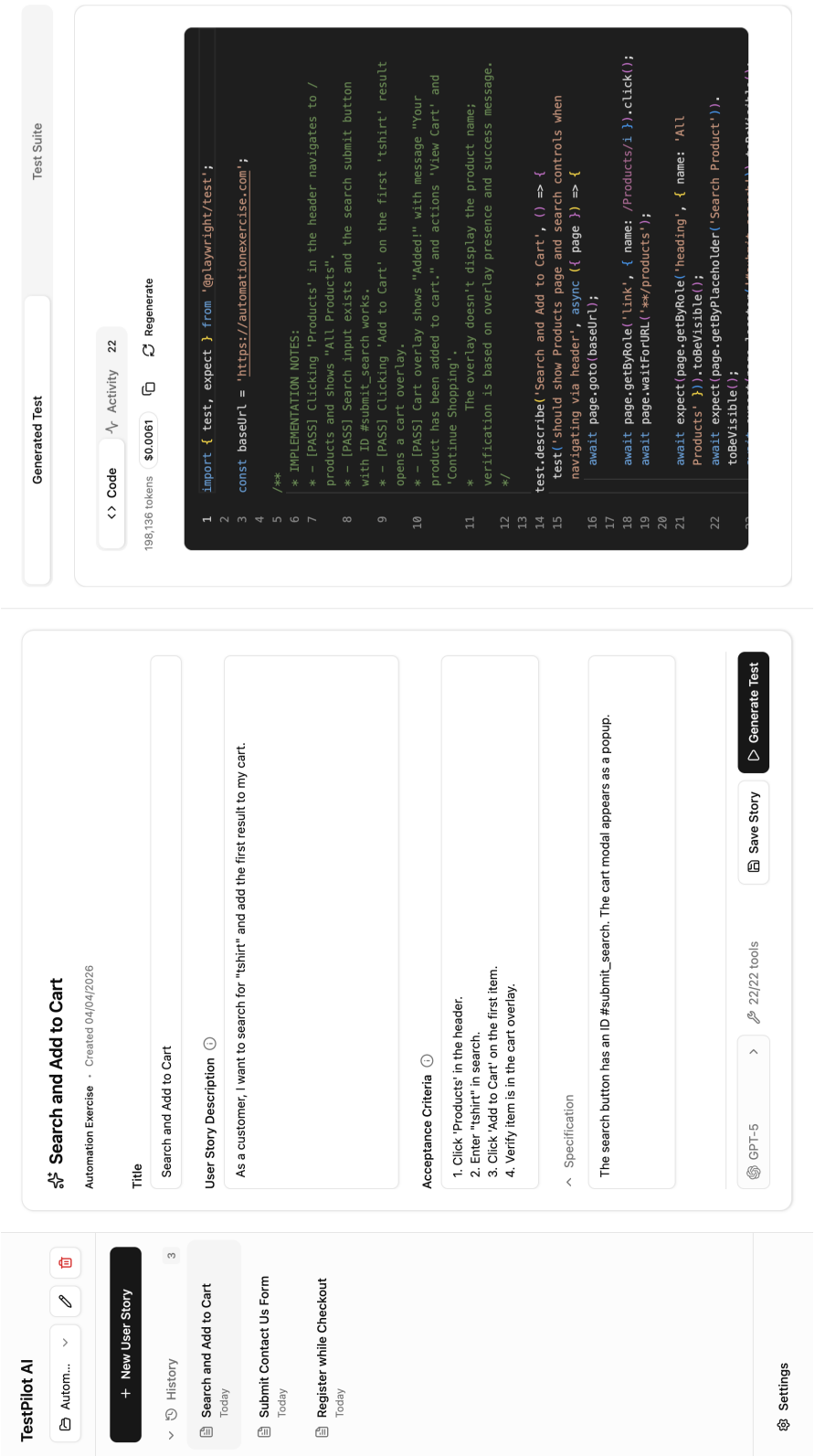
Tato část slouží jako hlavní vstupní formulář pro zadávání uživatelských příběhů, které jsou následně ukládány do konfiguračního souboru a slouží jako základ pro generování testů. Tato sekce rovněž umožňuje rychlé nastavení povolených nástrojů a výběr modelu, který má agent při generování používat, jak je vidět na obrázku B.2.

Konfigurace parametrů generování

Obrázky B.4 a B.3 zobrazuje modální okna pro konfiguraci nástrojů, které smí agent při generování testů používat, a dále pro výběr konkrétního LLM modelu, který bude pro generování využit. Pro každý model je vidět i krátký popis a jeho cena za milion vstupních a výstupních tokenů.

Detail výsledku exekuce testu

Sekce na obrázku B.5 zobrazuje detailní přehled o výsledku spuštěného testu. Uživatel má možnost vidět základní metriky exekuce (čas, status) a proklik na detailní HTML report, který je nativně generován nástrojem Playwright. Zároveň je zde zobrazen i samotný zdrojový kód testu, aby uživatel měl okamžitý přehled o tom, jaký kód byl spuštěn a nemusel na něj nahlížet přímo v IDE.



Obrázek B.1: Náhled na uživatelské rozhraní aplikace.

Submit Contact Us Form

Automation Exercise · Created 04/04/2026

Title

Submit Contact Us Form

User Story Description ⓘ

As a user, I want to contact support regarding a broken item so I can request a refund.

Acceptance Criteria ⓘ

1. Navigate to 'Contact Us'.
2. Fill Name, Email, Subject, and Message.
3. Upload a sample file.
4. Click Submit and handle the browser alert.

^ Specification

After clicking 'Submit', a browser window.confirm appears. The agent must click "OK".
Verify the success message "Success! Your details have been submitted successfully."

 GPT-5



 22/22 tools

 Save Story

 Generate Test

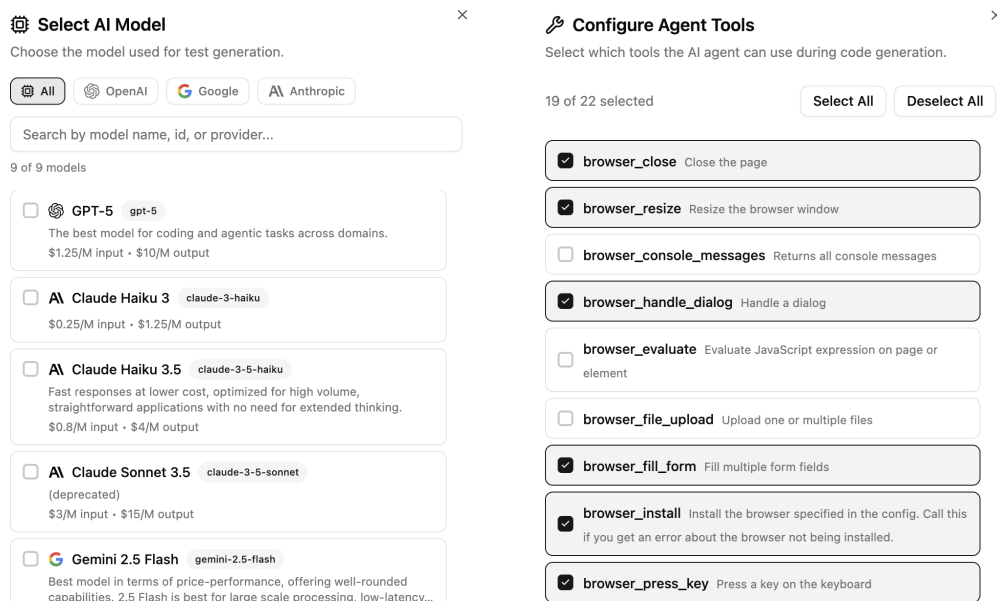
Obrázek B.2: Detail formuláře pro specifikaci parametrů uživatelského příběhu.

Správa testovacích sad

V sekci Test Suites (obrázek B.6) je zobrazen přehled všech vygenerovaných testů, které jsou logicky seskupeny podle projektů. Každý test lze přes GUI smazat a spustit - navíc v případě spuštění testu se automaticky zobrazí ikonka jeho posledního stavu.

Nativní HTML report nástroje Playwright

Velmi často může chtít uživatel nahlédnout na konkrétní průběh testu, zejména v případě jeho selhání, aby získal detailní informace o tom, co přesně se stalo. To po-



Obrázek B.3: Modální okno pro výběr LLM modelu.

Obrázek B.4: Konfigurace povolených MCP nástrojů.

skytuje nativní HTML report generovaný Playwrightem, jak je vidět na obrázku B.7. Obsahuje rozpad jednotlivých testovacích kroků, chybové hlášky, přesnou lokaci selhání (včetně náhledu na zdrojový kód) a další důležité informace, které mohou pomoci s rychlou identifikací problému a jeho následnou opravou.

Core Shopping Flow Add Item To Cart 9

PASSED

① **How to run tests:** Click Run to execute this test using Playwright. Tests will run in browsers configured in your playwright.config.ts. Make sure Playwright is installed via npm install.

⌚ Duration: 2.84s

Executed: 04/04/2026, 15:22:42

Test Results:

[Open Full Report](#)

✓ All Tests Passed ✓ 1 test passed

Test Code:

```
1  /**
2   * IMPLEMENTATION NOTES:
3   * - [PASS] Authentication: Logging in with standard_user/secret_sauce
4     redirects to /inventory.html
5   * - [PASS] Add to Cart Interaction: Clicking
6     [data-test="add-to-cart-sauce-labs-backpack"] changes the button to
7     "Remove"
8   * - [PASS] Cart Badge Update: Cart badge updates to "1" after adding
9     the item
10  * - [PASS] Cart Verification: Clicking the shopping cart icon navigates
11    to /cart.html and "Sauce Labs Backpack" is listed
12  */
13  import { test, expect } from '@playwright/test';
14  const baseUrl = 'https://www.saucedemo.com';
15  test.describe('Core Shopping Flow - Add Item to Cart', () => {
16    test('should add Sauce Labs Backpack to cart and verify it when logged
17    in', async ({ page }) => {
18      // Authentication
```

Obrázek B.5: Presentace výsledku spuštění testu včetně zdrojového kódu a metrik exekuce.

Test Suite (16)

Automation Exercise 2

- Submit Contact Us Form 04/04/2026
- Search And Add To Cart 04/04/2026

Swag Labs 14

- Core Shopping Flow Add Item To Cart 9 04/03/2026 ✓
- Core Shopping Flow Add Item To Cart 8 28/02/2026
- Task Management Create And Complete A Task 3

Obrázek B.6: Přehled vygenerovaných testů strukturovaný podle příslušných projektů.

Run

Errors

Error: expect(locator).toBeVisible() failed

Locator: locator('.new-todo')

Expected: visible

Timeout: 5000ms

Error: element(s) not found

Call log:

- Expect "toBeVisible" with timeout 5000ms
- waiting for locator('.new-todo')

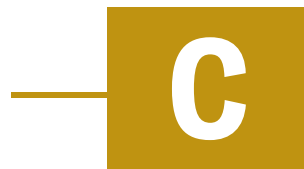
```
11 | // Selector from specification: main input field
12 | const newTodo = page.locator('.new-todo');
> 13 | await expect(newTodo).toBeVisible({ timeout: 5000 });
    |                               ^
14 |
15 | // 1) Create Task: type "Buy groceries" and press Enter
16 | await newTodo.fill('Buy groceries');
    at /Users/milanjanoch/Documents/Emplifi-projects/hackstreet-boys-data-pump/tests/generated/swag-labs/task-
```

Test Steps

> ✓ Before Hooks	1.2s
> ✓ Navigate to "/todomvc/index.html" — swag-labs/task-management-create-and-complete-a-task-2.spec.ts:8	541ms
> ✓ Wait for load state "networkidle" — swag-labs/task-management-create-and-complete-a-task-2.spec.ts:9	1.4s
> ✗ Expect "toBeVisible" locator('.new-todo') — swag-labs/task-management-create-and-complete-a-task-2.spec.ts:13	5.0s
> ✓ After Hooks	17ms
> ✓ Worker Cleanup	30ms

Obrázek B.7: Automaticky generovaný HTML report nástrojem Playwright.

Uživatelské příběhy použité k evaluaci



Tato příloha obsahuje přesné znění všech uživatelských příběhů použitých při evaluaci systému na ukázkové aplikaci. U každé položky je zachována i testovací poznámka s informací o konkrétní chybě, která aplikaci záměrně znefunkčňuje. Tato informace je určena pouze pro interní potřebu a není předávána agentovi.

US-01 – User Registration & Onboarding Flow

Name: User Registration & Onboarding Flow

Description: As a new user, I want to securely register an account and subsequently log in with my new credentials.

Acceptance Criteria:

- **AC_01 (Navigation):** The registration form must be accessible via the "Register" link on the default login view.
- **AC_02 (Validation):** The system must prevent registration if the 'Password' and 'Confirm Password' fields do not match, displaying an appropriate validation warning.
- **AC_03 (Success):** Providing valid details with matching passwords successfully creates the account and automatically redirects the user back to the login page.
- **AC_04 (Persistence):** The newly registered credentials must be immediately valid for authenticating into the dashboard.

Specification: Let the application automatically handle unauthenticated routing.

Testing Notes (INTERNAL ONLY – DO NOT PASS TO AGENT):

- **Bug Injection:** ?b=r8w5gb2n

US-02 – Login Validation and Error Handling

Name: Login Validation and Error Handling

Description: As a user, I must be prevented from logging in with invalid credentials and receive appropriate feedback.

Acceptance Criteria:

- **AC_01 (Initial State):** Upon accessing the application unauthenticated, the user is presented with the login interface where input fields must be empty by default.
- **AC_02 (Empty Submission):** Attempting to submit the login form without providing both an email and a password must be blocked by local validation warnings.
- **AC_03 (Invalid Credentials):** Submitting the form with an incorrect email or password must trigger a clearly visible "Invalid credentials" error message.

Specification:**Testing Notes (INTERNAL ONLY – DO NOT PASS TO AGENT):**

- **Bug Injection 1:** ?b=q7m2vn4k
- **Bug Injection 2:** ?b=p3x9ha1t

US-03 – Role-Based Navigation Visibility**Name:** Role-Based Navigation Visibility**Description:** As a system administrator, I want to access privileged settings, while as a regular user, I must be strictly prevented from seeing or accessing administrative navigation links.**Acceptance Criteria:**

- **AC_01 (Admin Access):** The system must render the 'System Settings' navigation tab in the header when authenticated as an administrator (admin@test.com).
- **AC_02 (User Restriction):** The system must strictly omit the 'System Settings' tab from the DOM when authenticated as a standard user (user@test.com), preventing unauthorized discovery.
- **AC_03 (Session Boundary):** The system must correctly isolate sessions; logging out and logging back in as a different role must immediately reflect that role's navigation permissions.

Specification: The standard password for all test accounts is password123. Verification of AC_02 must ensure the element is physically absent from the DOM, not merely visually hidden.**Testing Notes (INTERNAL ONLY – DO NOT PASS TO AGENT):**

- **Bug Injection:** ?b=a9u3kd1x

US-04 – Create New Task and Verify Notification**Name:** Create New Task and Verify Notification**Description:** As a logged-in user, I want to create a new task and see a clear visual confirmation.**Acceptance Criteria:**

1. Start at the base URL and log in using user@test.com and password123.

2. Click the 'New Task' button to open the creation modal.
3. Leave the Title field completely empty and click 'Save'.
4. Verify that the form submission is prevented due to a required field validation.
5. Fill the Title field with "Test Task" and select a Priority.
6. Click 'Save' and verify that the modal automatically closes.
7. Verify that the newly created "Test Task" is visible in the dashboard task list.
8. Verify that a success Toast notification appears on the screen.

Specification: Use strict text selectors where possible.

Testing Notes (INTERNAL ONLY – DO NOT PASS TO AGENT):

- **Bug Injection 1:** ?b=n8r4bf6q (task_no_validation - allows empty title)
- **Bug Injection 2:** ?b=y1p6ls4v (toast_missing)

US-05 – Task Status Toggle and Filtering

Name: Task Status Toggle and Filtering

Description: As a user, I want to mark tasks as completed and use filters to quickly find active tasks.

Acceptance Criteria:

1. Log in using user@test.com and password123.
2. Locate an active task in the list and click its status checkbox.
3. Verify that the checkbox reflects the checked state (marked as completed).
4. Click the 'Active' filter button/radio.
5. Verify that the task you just marked as completed is no longer visible in the task list.
6. Click the 'Completed' filter button/radio.
7. Verify that the completed task is now visible in the list again.

Specification: Ensure you wait for the list to update after clicking the filters before making assertions.

Testing Notes (INTERNAL ONLY – DO NOT PASS TO AGENT):

- **Bug Injection 1:** ?b=t4h9eu5m (status_toggle_fail)
- **Bug Injection 2:** ?b=f6j2qp9w (filter_broken)

US-06 – Edit Task details via Modal

Name: Edit Task details via Modal

Description: As a user, I want to update existing tasks without navigating away from the dashboard.

Acceptance Criteria:

1. Log in using `user@test.com` and `password123`.
2. Click the 'Edit' button on an existing task in the list.
3. Verify that the edit modal opens and the input fields are pre-populated with the task's current data.
4. Change the value in the 'Priority' field to a different option.
5. Click the 'Save' button.
6. Verify that the edit modal closes automatically.
7. Verify that the task in the dashboard list now displays the updated Priority.

Specification: N/A **Testing Notes (INTERNAL ONLY – DO NOT PASS TO AGENT):**

- **Bug Injection:** `?b=v1s8y13c(modal_wont_close)`

US-07 – Role-Based Task Deletion Permissions

Name: Role-Based Task Deletion Permissions

Description: As a system administrator, I can manage all tasks. As a user, I must not be able to delete tasks belonging to others to ensure data integrity.

Acceptance Criteria:

- **AC_01 (Admin Access):**
 - **Given** I am successfully logged in as an administrator (`admin@test.com`),
 - **When** I view the dashboard task list,
 - **Then** I should see the 'Delete' button on all tasks regardless of their owner.
- **AC_02 (User Restriction):**
 - **Given** I am successfully logged in as a standard user (`user@test.com`),
 - **When** I view the dashboard task list,
 - **Then** I should only see the 'Delete' button on tasks owned by me,
 - **And** I should strictly NOT see the 'Delete' button on tasks owned by the administrator.

Specification: The standard password for all test accounts is `password123`. Ensure the absence of the delete button is verified at the DOM level.

Testing Notes (INTERNAL ONLY – DO NOT PASS TO AGENT):

- **Bug Injection 1:** ?b=d5k7mt2z (delete_btn_missing - missing for everyone)
- **Bug Injection 2:** ?b=c2n7zr8p (rbac_leak - User sees Delete buttons on Admin's tasks)

US-08 – Task Deletion Confirmation Flow

Name: Task Deletion Confirmation Flow

Description: As a user, I want to delete my task but be prompted for confirmation first to prevent accidental data loss.

Acceptance Criteria:

- **AC_01 (Confirmation Prompt):**
 - **Given** I am logged in as user@test.com and have an existing task,
 - **When** I click the 'Delete' button on my task,
 - **Then** the system must display a confirmation dialog before proceeding.
- **AC_02 (Successful Deletion):**
 - **Given** the deletion confirmation dialog is visible,
 - **When** I accept/confirm the deletion,
 - **Then** the task must be permanently removed from the dashboard list,
 - **And** a success Toast notification must be displayed to confirm the action.

Specification: Use standard auto-waiting to verify the Toast notification. The password is password123.

Testing Notes (INTERNAL ONLY – DO NOT PASS TO AGENT):

- **Bug Injection:** ?b=y1p6ls4v (toast_missing)

Uživatelské testování



Tato příloha obsahuje obdržené odpovědi z Google Forms dotazníku, který byl použit pro sběr zpětné vazby od uživatelů, kteří nástroj TestPilot AI vyzkoušeli na vlastních aplikacích. Odpovědi na jednotlivé otázky jsou nepovinné, aby uživatelé mohli poskytnout zpětnou vazbu pouze k relevantním aspektům, které pro ně byly důležité či které využili.

Respondent 1

Google Forms - export odpovědi

1) Jaká je Vaše primární role v týmu?

- ☒ Softwarový inženýr ☐ QA inženýr ☐ Manažer
☐ Softwarový architekt ☐ Other

2) Jak hodnotíte proces počátečního nastavení a spuštění nástroje pomocí příkazů `npx testpilot init/run`? 1 = Velmi náročné, 5 = Velmi snadné

- ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

3) Jak přehledný a intuitivní je pro Vás dashboard aplikace? 1 = Velmi nepřehledný a obtížně použitelný, 5 = Velmi přehledný a intuitivní

- ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

4) Do jaké míry odpovídal vygenerovaný kód Vaším definovaným akceptačním kritériím? 1 = Vygenerovaný kód akceptačním kritériím neodpovídal, 5 = Plně odpovídal definovaným kritériím

- ☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5

5) Jak hodnotíte kvalitu vybraných selektorů a celkovou strukturu testu z pohledu osvědčených postupů (Playwright best practices)? 1 = Velmi nízká kvalita implementace, 5 = Vynikající kvalita odpovídající osvědčeným postupům

- ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

6) Pomohlo Vám sledování myšlenkových pochodů agenta (Thinking process) a volání nástrojů (Tool calls) pochopit jeho postup, nebo jste

tyto informace vnímal/a jako nadbytečné? 1 = Zcela nadbytečné informace, 5 = Velmi užitečné pro pochopení procesu rozhodování

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

7) Pokud agent narazil na chybu a následně ji sám opravil, jak hodnotíte úspěšnost a relevanci této opravy? 1 = Oprava nebyla úspěšná nebo nebyla relevantní, 5 = Oprava byla plně úspěšná a relevantní

☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5

8) Jak Vám vyhovuje, že nástroj ukládá testy přímo do Vašeho projektu (do specifikovaných složek) namísto ukládání do interní databáze? 1 = Zcela nevyhovuje, 5 = Zcela vyhovuje

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

9) Domníváte se, že by Vám tento nástroj reálně ušetřil čas při psaní základní kostry (boilerplate kódu) pro E2E testy?

☒ Určitě ano ☐ Spíše ano ☐ Spíše ne ☐ Vůbec

10) Jak byste celkově ohodnotil/a nástroj TestPilot AI? 1 = Vynikající nástroj, 5 = Prakticky nepoužitelný nástroj

☐ 1 ☒ 2 ☐ 3 ☐ 4 ☐ 5

11) Považujete tento Proof of Concept za směr, který by po dalším vývoji mohl sloužit jako plnohodnotný asistent pro vývojáře a QA?

☒ Určitě ano ☐ Spíše ano ☐ Spíše ne ☐ Vůbec ne

12) Na jaké chyby jste při testování narazil/a? Co Vás nejvíce limitovalo a co byste naopak označil/a za nejsilnější stránku programu?

Limitující byla spíš testovaná aplikace (přihlašování, komplexnost, atp.). Zadání user story/akceptačních kritérií by zřejmě potřebovalo být detailnější, aby nedocházelo k nejednoznačnostem ve vygenerovaném testu (např. existence vícero stejně pojmenovaných UI prvků bez bližšího specifikování). Ani jedno ale není vadou TestPilota, navíc funkce Regenerate takové nejednoznačnosti (při použití fail logu jako vstupu) většinou opravila. Oceňuji důraz na přehledné UI a míru interaktivitu, např. přímé spuštění testu z aplikace, zobrazení jeho výsledku, procházení myšlenkových kroků agenta a použitých nástrojů. I samotné přidání aplikace do projektu bylo velmi jednoduché.

Respondent 2

Google Forms - export odpovědi

1) Jaká je Vaše primární role v týmu?

☐ Softwarový inženýr ☐ QA inženýr ☐ Manažer
☒ Softwarový architekt ☐ Other

2) Jak hodnotíte proces počátečního nastavení a spuštění nástroje pomocí příkazů `npx testpilot init/run`? 1 = Velmi náročné, 5 = Velmi snadné

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

3) Jak přehledný a intuitivní je pro Vás dashboard aplikace? 1 = Velmi nepřehledný a obtížně použitelný, 5 = Velmi přehledný a intuitivní

☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5

4) Do jaké míry odpovídal vygenerovaný kód Vaším definovaným akceptačním kritériím? 1 = Vygenerovaný kód akceptačním kritériím neodpovídal, 5 = Plně odpovídal definovaným kritériím

☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5

5) Jak hodnotíte kvalitu vybraných selektorů a celkovou strukturu testu z pohledu osvědčených postupů (Playwright best practices)? 1 = Velmi nízká kvalita implementace, 5 = Vynikající kvalita odpovídající osvědčeným postupům

☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5

6) Pomohlo Vám sledování myšlenkových pochodů agenta (Thinking process) a volání nástrojů (Tool calls) pochopit jeho postup, nebo jste tyto informace vnímal/a jako nadbytečné? 1 = Zcela nadbytečné informace, 5 = Velmi užitečné pro pochopení procesu rozhodování

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

7) Pokud agent narazil na chybu a následně ji sám opravil, jak hodnotíte úspěšnost a relevanci této opravy? 1 = Oprava nebyla úspěšná nebo nebyla relevantní, 5 = Oprava byla plně úspěšná a relevantní

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

8) Jak Vám vyhovuje, že nástroj ukládá testy přímo do Vašeho projektu (do specifikovaných složek) namísto ukládání do interní databáze? 1 = Zcela nevyhovuje, 5 = Zcela vyhovuje

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

9) Domníváte se, že by Vám tento nástroj reálně ušetřil čas při psaní základní kostry (boilerplate kódu) pro E2E testy?

☒ Určitě ano ☐ Spíše ano ☐ Spíše ne ☐ Vůbec

10) Jak byste celkově ohodnotil/a nástroj TestPilot AI? 1 = Vynikající nástroj, 5 = Prakticky nepoužitelný nástroj

☒ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

11) Považujete tento Proof of Concept za směr, který by po dalším vývoji mohl sloužit jako plnohodnotný asistent pro vývojáře a QA?

☒ Určitě ano ☐ Spíše ano ☐ Spíše ne ☐ Vůbec ne

12) Na jaké chyby jste při testování narazil/a? Co Vás nejvíce limitovalo a co byste naopak označil/a za nejsilnější stránku programu?

Připomínky:

- 1) Po úvodní inicializaci se zobrazila chyba informující o tom, že adresář tests/generated neexistuje. V dokumentaci však nebylo uvedeno, že je nutné tento adresář vytvořit ručně.
- 2) U funkce Regenerate není na první pohled zřejmé, že tlačítko otevírá modální okno, do kterého lze zadat požadavek na regeneraci. Uživatelsky to spíše působí dojmem, že po kliknutí dojde k okamžitému spuštění regenerace.
- 3) Při regeneraci bych očekával spíše úpravu již existujícího uloženého testu, nikoli vytvoření nového.

Silné stránky:

- 1) Možnost regenerace testů považuji za silnou stránku aplikace. Ve většině případů si aplikace dokázala dobře poradit se zadanými požadavky a test byl upraven podle očekávání již na první pokus.
- 2) Další silnou stránkou je sledování myšlenkového procesu agenta a volání nástrojů.

Možnosti budoucího vylepšení:

- 1) Přímá regenerace již existujících testů.
- 2) Možnost vložení kontextu aplikace přímo do konfigurace projektu.

Respondent 3

Google Forms - export odpovědi

1) Jaká je Vaše primární role v týmu?

- ☐ Softwarový inženýr
 ☒ QA inženýr
 ☐ Manažer
☐ Softwarový architekt
 ☐ Other

2) Jak hodnotíte proces počátečního nastavení a spuštění nástroje pomocí příkazů `npx testpilot init/run`? 1 = Velmi náročné, 5 = Velmi snadné

- ☐ 1
 ☐ 2
 ☐ 3
 ☐ 4
 ☒ 5

3) Jak přehledný a intuitivní je pro Vás dashboard aplikace? 1 = Velmi nepřehledný a obtížně použitelný, 5 = Velmi přehledný a intuitivní

- ☐ 1
 ☐ 2
 ☐ 3
 ☐ 4
 ☒ 5

4) Do jaké míry odpovídal vygenerovaný kód Vaším definovaným akceptačním kritériím? 1 = Vygenerovaný kód akceptačním kritériím neodpovídal, 5 = Plně odpovídal definovaným kritériím

- ☐ 1
 ☐ 2
 ☐ 3
 ☒ 4
 ☐ 5

5) Jak hodnotíte kvalitu vybraných selektorů a celkovou strukturu testu z pohledu osvědčených postupů (Playwright best practices)? 1 = Velmi nízká kvalita implementace, 5 = Vynikající kvalita odpovídající osvědčeným postupům

☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5

6) Pomohlo Vám sledování myšlenkových pochodů agenta (Thinking process) a volání nástrojů (Tool calls) pochopit jeho postup, nebo jste tyto informace vnímal/a jako nadbytečné? 1 = Zcela nadbytečné informace, 5 = Velmi užitečné pro pochopení procesu rozhodování

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

7) Pokud agent narazil na chybu a následně ji sám opravil, jak hodnotíte úspěšnost a relevanci této opravy? 1 = Oprava nebyla úspěšná nebo nebyla relevantní, 5 = Oprava byla plně úspěšná a relevantní

☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5

8) Jak Vám vyhovuje, že nástroj ukládá testy přímo do Vašeho projektu (do specifikovaných složek) namísto ukládání do interní databáze? 1 = Zcela nevyhovuje, 5 = Zcela vyhovuje

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

9) Domníváte se, že by Vám tento nástroj reálně ušetřil čas při psaní základní kostry (boilerplate kódu) pro E2E testy?

☒ Určitě ano ☐ Spíše ano ☐ Spíše ne ☐ Vůbec

10) Jak byste celkově ohodnotil/a nástroj TestPilot AI? 1 = Vynikající nástroj, 5 = Prakticky nepoužitelný nástroj

☒ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

11) Považujete tento Proof of Concept za směr, který by po dalším vývoji mohl sloužit jako plnohodnotný asistent pro vývojáře a QA?

☒ Určitě ano ☐ Spíše ano ☐ Spíše ne ☐ Vůbec ne

12) Na jaké chyby jste při testování narazil/a? Co Vás nejvíce limitovalo a co byste naopak označil/a za nejsilnější stránku programu?

Trochu mi chybělo, že nemohu pracovat na generování více testů najednou, když se mezi testy (user stories) přepnu, tak už v záložce Generated test není ten test a nejde tak již udělat třeba Regenerate. Problém s již přihlášeným uživatelem v testované aplikaci při startu generování testu vedl k tomu, že v testu chyběl login a tak pak běh testu vždy selhal, protože neměl tento krok. Věřím, že půjde vyřešit možná nastavení přímo v Playwright.

Ale celkově nástroj je výborným pomocníkem, který šetří spoustu času a usnadňuje velmi rychle implementovat základní test cases pro testování nových features.

Respondent 4

Google Forms - export odpovědi

1) Jaká je Vaše primární role v týmu?

- ☐ Softwarový inženýr
 ☐ QA inženýr
 ☐ Manažer
 ☒ Softwarový architekt
 ☐ Other

2) Jak hodnotíte proces počátečního nastavení a spuštění nástroje pomocí příkazů `npx testpilot init/run`? 1 = Velmi náročné, 5 = Velmi snadné

- ☐ 1
 ☐ 2
 ☐ 3
 ☐ 4
 ☒ 5

3) Jak přehledný a intuitivní je pro Vás dashboard aplikace? 1 = Velmi nepřehledný a obtížně použitelný, 5 = Velmi přehledný a intuitivní

- ☐ 1
 ☐ 2
 ☐ 3
 ☒ 4
 ☐ 5

4) Do jaké míry odpovídal vygenerovaný kód Vaším definovaným akceptačním kritériím? 1 = Vygenerovaný kód akceptačním kritériím neodpovídal, 5 = Plně odpovídal definovaným kritériím

- ☐ 1
 ☐ 2
 ☐ 3
 ☐ 4
 ☒ 5

5) Jak hodnotíte kvalitu vybraných selektorů a celkovou strukturu testu z pohledu osvědčených postupů (Playwright best practices)? 1 = Velmi nízká kvalita implementace, 5 = Vynikající kvalita odpovídající osvědčeným postupům

- ☐ 1
 ☐ 2
 ☐ 3
 ☒ 4
 ☐ 5

6) Pomohlo Vám sledování myšlenkových pochodů agenta (Thinking process) a volání nástrojů (Tool calls) pochopit jeho postup, nebo jste tyto informace vnímal/a jako nadbytečné? 1 = Zcela nadbytečné informace, 5 = Velmi užitečné pro pochopení procesu rozhodování

- ☐ 1
 ☐ 2
 ☐ 3
 ☒ 4
 ☐ 5

7) Pokud agent narazil na chybu a následně ji sám opravil, jak hodnotíte úspěšnost a relevanci této opravy? 1 = Oprava nebyla úspěšná nebo nebyla relevantní, 5 = Oprava byla plně úspěšná a relevantní

- ☐ 1
 ☐ 2
 ☐ 3
 ☐ 4
 ☒ 5

8) Jak Vám vyhovuje, že nástroj ukládá testy přímo do Vašeho projektu (do specifikovaných složek) namísto ukládání do interní databáze? 1 = Zcela nevyhovuje, 5 = Zcela vyhovuje

- ☐ 1
 ☐ 2
 ☐ 3
 ☐ 4
 ☒ 5

9) Domníváte se, že by Vám tento nástroj reálně ušetřil čas při psaní základní kostry (boilerplate kódu) pro E2E testy?

- ☒ Určitě ano
 ☐ Spíše ano
 ☐ Spíše ne
 ☐ Vůbec

10) Jak byste celkově ohodnotil/a nástroj TestPilot AI? 1 = Vynikající nástroj, 5 = Prakticky nepoužitelný nástroj

☒ 1☐ 2☐ 3☐ 4☐ 5

11) Považujete tento Proof of Concept za směr, který by po dalším vývoji mohl sloužit jako plnohodnotný asistent pro vývojáře a QA?

☒ Určitě ano☐ Spíše ano☐ Spíše ne☐ Vůbec ne

12) Na jaké chyby jste při testování narazil/a? Co Vás nejvíce limitovalo a co byste naopak označil/a za nejsilnější stránku programu?

Jako velmi silnou stránku nástroje vnímám snadnou integraci do existujících projektů, vytváření testů přímo do repozitáře a možnosti konfigurace. Jednoduchost rozhraní dává možnost pro široké uplatnění pro vytváření testů i pro méně technické role ve firmě. Pro plné nahrazení současných nástrojů v některých týmech chybí pouze drobná rozšíření správy testovací sady - úprava existujících testů.

Uživatelská dokumentace



Prerekvizity

Před samotnou instalací nástroje TestPilot AI je nutné ověřit a zajistit, že prostředí splňuje následující požadavky:

- **Node.js:** Verze 22 a vyšší (je kompatibilní i s 18+, ale pro optimální výkon je vhodné volit LTS verzi).
- **npm:** Verze 9 a vyšší
- **Playwright:** Nainstalovaný Playwright (@playwright/test) včetně jeho inicializace (`npm init playwright@latest`). Důležité je zvolit **TypeScript** jako preferovaný jazyk pro testy. Více viz oficiální dokumentace Playwright.
- **Playwright prohlížeče:** Nainstalované binární soubory prohlížečů (příkaz `npx playwright install`).
- **API klíče:** Zajištěný přístup k AI Gateway prostřednictvím platného API klíče a URL.

Instalace a inicializace

Nástroj je distribuován jako NPM balíček (dostupný na <https://www.npmjs.com/package/testpilotai>) a lze jej nainstalovat pomocí příkazu:

```
npm install testpilotai
```

Pro integraci nástroje do existujícího projektu je potřeba přejít do kořenového adresáře repozitáře a spustit inicializační příkaz.

```
npx testpilotai init
```

Tento příkaz detekuje konfiguraci Playwrightu, nabídne volbu výstupní složky pro generované testy a vytvoří konfigurační soubor `test-pilot.json`. Výstupní složka

pro testy lze kdykoliv dodatečně změnit upravením tohoto konfiguračního souboru prostřednictvím klíče `config.output.directory`.

V kořenovém adresáři je následně nutné vytvořit soubor `.env` s přístupovými údaji k LLM:

```
AI_GATEWAY_KEY=<YOUR_API_KEY>
AI_GATEWAY_BASE_URL=<YOUR_API_URL/v1>
PORT=3002
```

Zatímco `AI_GATEWAY_KEY` a `AI_GATEWAY_BASE_URL` jsou povinnými proměnnými pro správnou komunikaci s LLM, proměnná `PORT` je volitelná a slouží k nastavení portu, na kterém TestPilot AI server poběží (výchozí je 3000).

Spuštění a pracovní postup (Workflow)

Uživatelské rozhraní a lokální server se spustí pomocí příkazu:

```
npx testpilotai run
```

Otevře se prohlížečové okno s běžící aplikací TestPilot AI. Samotný proces generování zahrnuje následující kroky:

1. **Vytvoření projektu:** V bočním panelu najet na sekci “Projects” (výběrový seznam v levém horním rohu), kliknout na “New Project” a zadat URL aplikace (např. `http://localhost:5173` nebo `https://example.com`), pro něž se mají generovat testy.
2. **Definice User Story:** Přidat nový uživatelský příběh, vyplnit název, popis a specifikovat detailní akceptační kritéria. Jako nepovinný krok lze přidat i dodatečnou specifikaci pro LLM.
3. **Generování:** Z nabídky vybrat LLM model (např. GPT-5 nebo Claude 4.5 Sonnet) a kliknout na “Generate Test”.
4. **Průběžná kontrola:** V záložce *Activity* v reálném čase sledovat myšlenkové pochody agenta a nástroje, které aktuálně volá nad aplikací.

Obrázky uživatelského rozhraní a jejich další popis je k dispozici v příloze B.

Vygenerovaný test se uloží do specifikované složky v projektu. Pokud test neodpovídá plně představám, lze využít tlačítko *Regenerate*, kam stačí vložit chybový log nebo přirozeným jazykem popsat požadovanou úpravu. Testy lze pouštět z aplikace v sekci “Test Suites”, kde každý běh vygeneruje HTML report s detailním logem, který může sloužit pro další iterace regenerace testu.

Programátorská dokumentace



Vývojové prostředí a instalace

Pro lokální vývoj je nutné mít nainstalované Node.js (v22+) a npm (9+). Zdrojové kódy se nachází ve složce `testpilotai/`. Postup pro lokální spuštění zahrnuje kroky:

1. Přechod do složky `testpilotai/`
2. Instalace závislostí pro celé monorepo: `npm install`
3. Instalace prohlížečů pro Playwright: `npx playwright install`
4. Pro spuštění v režimu vývoje (backend i frontend paralelně) využít příkaz:
`npm run dev`

Důležité NPM příkazy

V kořenovém adresáři monorepa jsou k dispozici následující skripty pro správu a sestavení:

- `npm run build`: Sestavení celého projektu (sdílené typy, backend, frontend).
- `npm run build:cli`: Sestavení finálního CLI balíčku do složky `packages/cli/dist`.
- `npm run test:api`: Spuštění unit a integračních testů backendové části v `apps/api`.
- `npm run test:client`: Spuštění testů frontendové části ve `apps/client` pomocí Vitest.

Konfigurace přímých LLM poskytovatelů

Výchozí nastavení využívá OpenAI-compatible rozhraní skrze AI Gateway, kde je v rámci `.env` proměnných zadán klíč a URL pro tuto gateway (viz příloha E). Pro přechod na přímé napojení na nativní SDK poskytovatelů (OpenAI, Anthropic, Google) je potřeba upravit implementaci v `apps/api/src/llm/llm.service.ts`.

1. Instalace příslušných balíčků v `apps/api`: `npm install @ai-sdk/openai @ai-sdk/anthropic @ai-sdk/google`.
2. Upravení metody `generate()` tak, aby využívala nativní providery na základě prefixu modelu:

```
1 import { openai } from '@ai-sdk/openai';
2 import { anthropic } from '@ai-sdk/anthropic';
3 import { google } from '@ai-sdk/google';
4
5 // Logic inside LLM service
6 // Determine provider based on model prefix
7 // and initialize corresponding SDK client
8 let aiModel;
9 if (model.providerPrefix === 'openai') {
10   aiModel = openai(model.shortId);
11 } else if (model.providerPrefix === 'anthropic') {
12   aiModel = anthropic(model.shortId);
13 } else if (model.providerPrefix === 'google') {
14   aiModel = google(model.shortId);
15 }
```

Následně je potřeba do `.env` souboru přidat klíče pro jednotlivé poskytovatele, například:

```
ANTHROPIC_API_KEY=<YOUR_ANTHROPIC_API_KEY>
OPENAI_API_KEY=<YOUR_OPENAI_API_KEY>
GOOGLE_API_KEY=<YOUR_GOOGLE_API_KEY>
```

Obsah přílohy



Níže je zobrazen obsah přiloženého zip souboru, který obsahuje všechny relevantní materiály a zdrojové kódy spojené s touto diplomovou prací.

```
└─ A24N0104P_prilohy.zip
   └─ Aplikace_a_knihovny
      ├── testpilota
      ├── testing_app
      └─ Poster
         ├── poster.pdf
         └── poster.pub
      └─ Text_prace
         └── dp.pdf
      └─ Readme.txt
```

Složka `Aplikace_a_knihovny` obsahuje zdrojové kódy systému TestPilot AI (složka `testpilota`) a testovací aplikace (složka `testing_app`), která byla použita pro účely testování a demonstrace funkcionalit nástroje.

Složka `Poster` obsahuje digitální verzi posteru.

Složka `Text_prace` obsahuje tento text diplomové práce ve formátu PDF.

Soubor `Readme.txt` poskytuje stručný popis o obsahu a struktuře přiložených materiálů.

Bibliografie

1. WANG, Junjie et al. *Software Testing with Large Language Models: Survey, Landscape, and Vision*. 2024. Dostupné z arXiv: 2307.07221 [cs.SE].
2. DOS SANTOS, Carlos Alberto; BOUCHARD, Kevin; PETRILLO, Fabio. AI-Driven User Story Generation. In: *2024 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA)*. 2024, s. 1–6. Dostupné z doi: 10.1109/ACDSA59508.2024.10467677.
3. DIMA, Alina; MAASSEN, Maria Alexandra. From Waterfall to Agile software: Development models in the IT sector, 2006 to 2018. Impacts on company management. *Journal of International Studies*. 2018, roč. 11, s. 315–326. Dostupné z doi: 10.14254/2071-8330.2018/11-2/21.
4. SENARATH, Udes S. Waterfall methodology, prototyping and agile development. *no. June*. 2021.
5. ALMEIDA, Fernando. Challenges in migration from waterfall to agile environments. *World Journal of Computer Application and Technology*. 2017, roč. 5, č. 3, s. 39–49.
6. SASSA, Adrielle Cristina; ALMEIDA, Isabela Alves de; PEREIRA, Tábata Nakagomi Fernandes; OLIVEIRA, Milena Silva de. Scrum: A systematic literature review. *International Journal of Advanced Computer Science and Applications*. 2023, roč. 14, č. 4.
7. HRON, Michal; OBWEGESER, Nikolaus. Scrum in practice: an overview of Scrum adaptations. In: *Hawaii International Conference on System Sciences*. Curran Associates, Inc., 2018, s. 4496–4505.
8. SACHDEVA, Sakshi. Scrum methodology. *Int. J. Eng. Comput. Sci.* 2016, roč. 5, č. 16792, s. 16792–16800.
9. FANIRAN, Victor Temitayo; BADRU, Abdulbaqi; AJAYI, Nurudeen. Adopting Scrum as an Agile approach in distributed software development: A review of literature. In: *2017 1st International Conference on Next Generation Computing Applications (NextComp)*. IEEE, 2017, s. 36–40.

10. RAHARJANA, Indra Kharisma; SIAHAAN, Daniel; FATICHAH, Chastine. User Stories and Natural Language Processing: A Systematic Literature Review. *IEEE Access*. 2021, roč. 9, s. 53811–53826. Dostupné z doi: 10.1109/ACCESS.2021.3070606.
11. AMNA, Anis R.; POELS, Geert. Systematic Literature Mapping of User Story Research. *IEEE Access*. 2022, roč. 10, s. 51723–51746. Dostupné z doi: 10.1109/ACCESS.2022.3173745.
12. POKHAREL, Prabhat; VAIDYA, Pramesh. A Study of User Story in Practice. In: *2020 International Conference on Data Analytics for Business and Industry: Way Towards a Sustainable Economy (ICDABI)*. 2020, s. 1–5. Dostupné z doi: 10.1109/ICDABI51230.2020.9325670.
13. LUCASSEN, Garm; DALPIAZ, Fabiano; WERF, Jan Martijn EM van der; BRINK-KEMPER, Sjaak. The use and effectiveness of user stories in practice. In: *International working conference on requirements engineering: Foundation for software quality*. Springer, 2016, s. 205–222.
14. BUGLIONE, Luigi; ABRAN, Alain. Improving the User Story Agile Technique Using the INVEST Criteria. In: *2013 Joint Conference of the 23rd International Workshop on Software Measurement and the 8th International Conference on Software Process and Product Measurement*. 2013, s. 49–53. Dostupné z doi: 10.1109/IWSM-Mensura.2013.18.
15. LI, Yishu et al. SimAC: simulating agile collaboration to generate acceptance criteria in user story elaboration. *Automated Software Engineering*. 2024, roč. 31, č. 2, s. 55.
16. FERREIRA, António MS; SILVA, Alberto Rodrigues da; PAIVA, Ana CR. Towards the Art of Writing Agile Requirements with User Stories, Acceptance Criteria, and Related Constructs. In: *ENASE*. 2022, s. 477–484.
17. CHANDORKAR, Adwait; PATKAR, Nitish; DI SORBO, Andrea; NIERSTRASZ, Oscar. An Exploratory Study on the Usage of Gherkin Features in Open-Source Projects. In: *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 2022, s. 1159–1166. Dostupné z doi: 10.1109/SANER53432.2022.00134.
18. ALSHAMMARI, Abdullah. *On the use of user interface specific domain Gherkin in behaviour driven development*. 2025. Dis. pr. University of Glasgow.
19. SCHWEDT, Stefan; STRÖDER, Thomas. From bugs to benefits: Improving user stories by leveraging crowd knowledge with cruise-ac. *arXiv preprint arXiv:2501.15181*. 2025.

20. JAMIL, Muhammad Abid; ARIF, Muhammad; ABUBAKAR, Normi Sham Awang; AHMAD, Akhlaq. Software Testing Techniques: A Literature Review. In: *2016 6th International Conference on Information and Communication Technology for The Muslim World (ICT4M)*. 2016, s. 177–182. Dostupné z doi: 10.1109/ICT4M.2016.045.
21. HEROUT, Pavel. *Testování pro programátory*. České Budějovice: Kopp, 2015. ISBN 978-80-7232-469-1.
22. MUKHIN, Vadym et al. The Testing Mechanism for Software and Services Based on Mike Cohn's Testing Pyramid Modification. In: *2021 11th IEEE International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS)*. IEEE, 2021, sv. 1, s. 589–595.
23. LEOTTA, Maurizio; CLERISSI, Diego; RICCA, Filippo; TONELLA, Paolo. Approaches and tools for automated end-to-end web testing. In: *Advances in Computers*. Elsevier, 2016, sv. 101, s. 193–237.
24. THE SELENIUM PROJECT. *Selenium WebDriver Documentation*. 2024. Dostupné také z: <https://www.selenium.dev/documentation/webdriver/>. Accessed: 2026-03-06.
25. ROBOT FRAMEWORK FOUNDATION. *Robot Framework User Guide*. 2024. Dostupné také z: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html>. Accessed: 2026-03-07.
26. MICROSOFT. *Playwright Library Documentation*. 2024. Dostupné také z: <https://playwright.dev/docs/intro>. Accessed: 2026-03-06.
27. CYPRESS.IO. *Cypress Key Differences*. 2024. Dostupné také z: <https://docs.cypress.io/guides/overview/key-differences>. Accessed: 2026-03-07.
28. BUTT, Shimza; KHAN, Saif Ur Rehman; HUSSAIN, Shahid; WANG, Wen-Li. A conceptual model supporting decision-making for test automation in Agile-based Software Development. *Data & Knowledge Engineering*. 2023, roč. 144, s. 102111.
29. MINAEE, Shervin et al. Large language models: A survey. *arXiv preprint arXiv:2402.06196*. 2024.
30. WANG, Zichong et al. History, development, and principles of large language models: an introductory survey. *AI and Ethics*. 2025, roč. 5, č. 3, s. 1955–1971.
31. VASWANI, Ashish et al. Attention is all you need. *Advances in neural information processing systems*. 2017, roč. 30.

32. GALLÉ, Matthias. Investigating the Effectiveness of BPE: The Power of Shorter Sequences. In: INUI, Kentaro; JIANG, Jing; NG, Vincent; WAN, Xiaojun (ed.). *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, s. 1375–1381. Dostupné z DOI: 10.18653/v1/D19-1141.
33. LEVY, Mosh; ELYOSEPH, Zohar; RAVFOGEL, Shauli; GOLDBERG, Yoav. State over Tokens: Characterizing the Role of Reasoning Tokens. *arXiv preprint arXiv:2512.12777*. 2025.
34. RENZE, Matthew. The effect of sampling temperature on problem solving in large language models. In: *Findings of the association for computational linguistics: EMNLP 2024*. 2024, s. 7346–7356.
35. SHEN, Zhuocheng. Llm with tools: A survey. *arXiv preprint arXiv:2409.18807*. 2024.
36. GIM, In et al. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*. 2024, roč. 6, s. 325–338.
37. HORÍNEK, Milan. *Generování jednotkových testů s využitím LLM [online]*. 2024 [cit. 2026-03-08]. Dostupné také z: <https://theses.cz/id/qi2was/>. Diplomová práce. Západočeská univerzita v Plzni, Faculty of Applied Sciences Plzeň. SUPERVISOR: Ing. Richard Lipka, Ph.D.
38. HOLICKÝ, Petr. *Nástroj pro generování GUI testů s využitím LLM [online]*. 2025 [cit. 2026-03-08]. Dostupné také z: <https://theses.cz/id/7mlcwt/>. Diplomová práce. Západočeská univerzita v Plzni, Fakulta aplikovaných věd Plzeň. SUPERVISOR: Ing. Richard Lipka, Ph.D.
39. SINGH, Aditi; EHTESHAM, Abul; KUMAR, Saket; KHOEI, Tala Talaei. Enhancing AI systems with agentic workflows patterns in large language model. In: *2024 IEEE World AI IoT Congress (AIIoT)*. IEEE, 2024, s. 527–532.
40. AHMADI, Arash; SHARIF, Sarah; BANAD, Yaser M. Mcp bridge: A lightweight, llm-agnostic restful proxy for model context protocol servers. *arXiv preprint arXiv:2504.08999*. 2025.
41. OPENAI. *A Practical Guide to Building Agents [online]*. 2025. [cit. 2026-03-12]. Technical Report. OpenAI. Dostupné z: <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>.
42. SARKAR, Anjana; SARKAR, Soumyendu. Survey of llm agent communication with mcp: A software design pattern centric review. *arXiv preprint arXiv:2506.05364*. 2025.

43. YAN, Xiangjun; YANG, Xincong; JIN, Nan; CHEN, Yu; LI, Jiaqi. A general AI agent framework for smart buildings based on large language models and ReAct strategy. *Smart Construction*. 2025, roč. 2, č. 1, s. 1–2.
44. BIERMAN, Gavin; ABADI, Martín; TORGERSEN, Mads. Understanding typescript. In: *European Conference on Object-Oriented Programming*. Springer, 2014, s. 257–281.
45. RICHARDS, Gregor; ZAPPA NARDELLI, Francesco; VITEK, Jan. Concrete types for TypeScript. In: *29th European Conference on Object-Oriented Programming (ECOOP 2015)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2015, s. 76–100.
46. CHEN, Songtao; THADURI, Upendar Rao; BALLAMUDI, Venkata Koteswara Rao. Front-end development in react: an overview. *Engineering International*. 2019, roč. 7, č. 2, s. 117–126.
47. MUNIROV, JJ. NestJS vs Express. js: A Comprehensive Comparison. *Modern World Education: New Age Problems–New solutions*. 2025, roč. 2, č. 6, s. 33–39.
48. PATIL, Omkar; KULKARNI, Himani; POTTAVATHINI, Rohit; DHOLE, Ishaan; SALGAONKAR, Krishna. Real time inventory management system powered by generative user interface. *Preprint Research Gate*. 2024.
49. TALAKOLA, Swetha. Automated end to end testing with Playwright for React applications. *International Journal of Emerging Research in Engineering and Technology*. 2024, roč. 5, č. 1, s. 38–47.

Seznam obrázků

2.1	Vizualizace Scrum procesu a jeho klíčových fází [9].	6
2.2	Pyramida testování Mika Cohna.	10
2.3	Vzor ReAct: Spojení uvažování a jednání pro autonomní agenty.	21
2.4	Ukázková architektura Client-Host-Server v rámci Model Context Protocolu.	22
3.1	Doménový model systému.	30
3.2	Vývojářské flow s využitím local-first přístupu.	32
3.3	Koncepční architektura systému.	33
3.4	Schéma Single-agent vs Multi-agent architektury.	35
3.5	Návrh uživatelského rozhraní.	37
4.1	Schéma komunikace pomocí SSE vs WebSocketů.	41
5.1	Zjednodušený diagram vrstvených závislostí backendových modulů.	45
5.2	Schéma validační pipeline a samoopravné smyčky.	60
5.3	Vizualizace běhu agenta zobrazená komponentou Activity Feed.	63
6.1	Detekce chyby modelem Claude Sonnet 4.5 s konkrétními poznámkami v Activity logu.	74
B.1	Náhled na uživatelské rozhraní aplikace.	95
B.2	Detail formuláře pro specifikaci parametrů uživatelského příběhu.	96
B.3	Modální okno pro výběr LLM modelu.	97
B.4	Konfigurace povolených MCP nástrojů.	97
B.5	Prezentace výsledku spuštěného testu včetně zdrojového kódu a metrik exekuce.	98
B.6	Přehled vygenerovaných testů strukturovaný podle příslušných projektů.	99
B.7	Automaticky generovaný HTML report nástrojem Playwright.	99

Seznam tabulek

2.1	INVEST principy pro tvorbu kvalitních user stories [14].	7
2.2	Druhy testů dle [21].	10
2.3	Přehled běžně používaných druhů tokenů v LLM.	16
2.4	Vybrané modely vydané od H2 2025 a jejich cenová politika.	18
3.1	Specifikace funkčních požadavků na systém.	28
3.2	Specifikace nefunkčních požadavků na systém.	29
5.1	Modely registrované v LlmService a jejich ceny.	53
5.2	Architektura a klíčové sekce systémového promptu.	58
6.1	Přehled injektovaných chyb a jejich mapování na hash parametry. . . .	67
6.2	Výsledky hodnocení kvality generování testů pro pozitivní scénáře (škála 0-3, viz 6.2.1).	70
6.3	Úspěšnost detekce zanesených chyb (D=Detected, FN=False Negative, E=Error).	72
6.4	Celkové náklady pro generování kompletní sady testů.	76

Seznam výpisů

2.1	Standardní šablona uživatelského příběhu.	7
2.2	Ukázka testovacího scénáře zapsaného pomocí klíčových slov jazyka Gherkin.	8
2.3	Ukázka akceptačních kritérií zapsaných formou strukturovaného seznamu.	9
3.1	Ukázka dotazu v jazyce JQL pro filtrování uživatelských příběhů. .	25
5.1	Ukázka sdílených doménových entit v balíčku @testpilotai/shared.	47
5.2	Ukázka DTO objektů pro vytváření a aktualizaci user stories. . . .	48
5.3	Struktura test-pilot.json.	49
5.4	Struktura .testpilot-cache.json.	49
5.5	Ukázkové volání LLM přes Vercel AI SDK.	50
5.6	Zjednodušené napojení Vercel AI SDK na vlastní AI Gateway . . .	51
5.7	Datová struktura pro konfiguraci LLM modelů.	52
5.8	Výpočet ceny LLM dotazu.	54
5.9	Zkrácený proces inicializace MCP.	55
5.10	Struktura vnitřního stavu exekuce agenta.	56
5.11	Zjednodušená řídicí smyčka agenta.	57
5.12	Inicializace a generování komponent knihovny Shadcn UI.	62
5.13	Definice událostí streamu vizualizovaných v klientském rozhraní.	62
6.1	Ukázka sdíleného kódu pro přihlášení vygenerovaného modelem GPT-5.	70
6.2	Ukázka detekce chyby modelem Gemini 2.5 Pro.	73
6.3	Ukázka výstupu nástroje browse_snapshot.	75
6.4	Porovnání manuálního řešení a přístupu modelu GPT-5 při ověřování nativní validace formuláře.	78
A.1	Plné znění systémového promptu s vyznačenými dynamickými bloky.	89
A.2	Plné znění regeneračního promptu s vyznačenými dynamickými bloky.	92

101011000011100010 1100001
1010110001 10



11010011101101001
01100001 101
111000101011 101